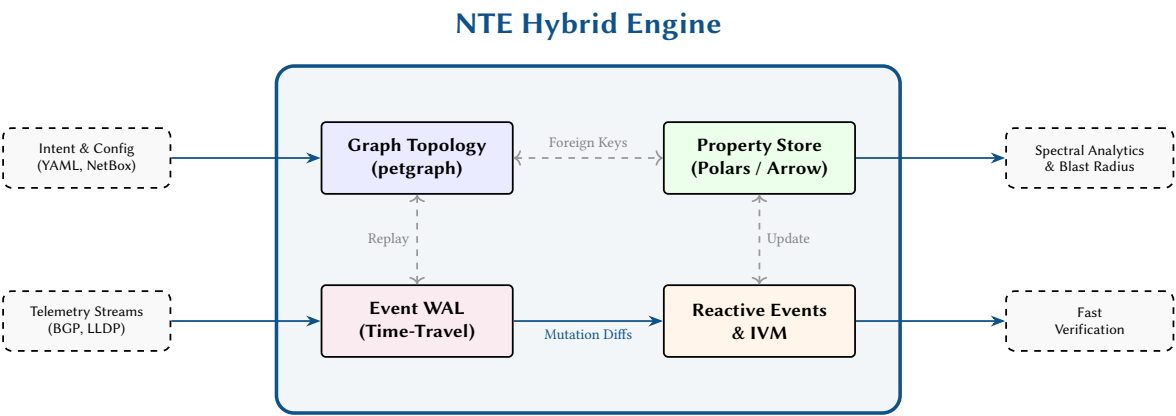


NTE: Network Topology Engine

A Hybrid Graph-Relational Engine for Network Topology Modelling

Simon Knight, Ph.D.
Principal Architect, AutoNetKit Research
March 2026 • Version 2.8.0
Last updated: March 18, 2026



Abstract. NTE (Network Topology Engine) is a hybrid graph-relational engine for large-scale structural and property analytics in network topology modelling. Its central design principle is a dual-write consistency model that aims to keep topology structure and columnar properties aligned. This report documents the engine’s policy and validation approach, filter-first query planner, and layered data model that supports configuration synthesis, graph analysis, and live telemetry ingestion.

Contents

I ARCHITECTURE & INSIGHTS

1 Introduction	2
1.1 Why a Bespoke Engine?	2
1.2 Running Example: Data Centre Migration.	3
1.3 Ecosystem: Embeddable Engine vs. Remote Server	3
2 Design Principles of Network Modeling	3
2.1 The Fallacy of the Simple Graph (The Endpoints Model)	3
2.2 The Fractal Nature of Nodes	4
2.3 Property Flattening (Algorithmic Accessibility)	5
2.4 Orthogonal Layering (The Multi-Plane Abstraction).	5
3 Data Model	6
3.1 Formal Definition	6
3.2 Vertex Types	6
3.3 Edge Kinds.	8
3.4 Composite Patterns.	10
3.5 Layer Hierarchy	11
3.6 Multi-Layer Topology Invariants	12
3.7 Topology Diffing	12
3.8 Property Maps.	13
4 Semantic Query Translation	13
4.1 Orthogonal Layering: Layers vs. Kinds	13
4.2 Syntax to Data Model Mapping	13
4.3 Virtual Cypher Expansion	14
4.4 Parallel Expansion: The LAG Scenario.	14
4.5 Context Projections and Sub-Selects	15
4.6 Bridging the Gap: The Benefits of Virtual Cypher	15
4.7 NTE-QL: Projections and State Mapping	15
4.8 Programmatic Mapping: Python and Starlark	16
5 Policy Engine	16
5.1 Design Rationale	17
5.2 Policy Specification.	17
5.3 The Starlark Policy Engine	17
5.4 Policy Evaluation (CEL & Starlark)	18
5.5 Shadow Topologies and DeltaNet Validation.	19
5.6 DeltaNet: Incremental Policy Validation	19
5.7 Worked Policy Examples.	20

II OPERATIONS & USAGE (ADVANCED CONCEPTS)

6 Building the Digital Twin: Ingestion & Identity	21
6.1 Bimodal Execution: Strict vs. Relaxed	21
6.2 Multi-Source Topology Aggregation	21
6.3 Flexible Ingestion Patterns	22
6.4 Semantic Identity Across Snapshots.	22
6.5 Shadow Graph Inference.	23
6.6 Cross-Layer Lineage: Vertical Impact Analysis	23
7 Structural Transformations	23
7.1 Split	24
7.2 Merge.	24
7.3 Explode (The Hub-and-Spoke Dissolver)	24
7.4 Aggregate (Graph Summarisation).	24
8 Operational Case Studies	25
8.1 Case Study 1: Transparent to Routed Migration	25
8.2 Case Study 2: Multi-layer Policy Validation	25
8.3 Case Study 3: Blast Radius Analysis.	25

9	Blast Radius & Failure Domain Isolation	25
10	Advanced Analytics	25
10.1	Differential Reachability: Imperative vs. Declarative Approaches	25
10.2	Spectral Graph Signatures	26
10.3	Probabilistic Risk Modelling: Monte Carlo Analysis	27
III	ENGINE IMPLEMENTATION (DEEP DIVE)	
11	Implementation Architecture	27
11.1	Workspace Layout	27
11.2	Crate Dependency Hierarchy	27
12	Transactions	28
12.1	"What-If" Branching	28
12.2	Dual-Write Atomicity: The DualWriteGuard	29
12.3	The Dual-Write Transaction Model	29
12.4	ACI (Durability via WAL) Transaction Model	30
12.5	Isolation Levels	31
12.6	Transaction Snapshots	31
12.7	Python Transaction API	31
12.8	Conflict Detection	32
13	Persistence and Snapshots	32
13.1	Write-Ahead Log (WAL) and Crash Recovery	32
13.2	Historical Reconstruction (Event-Sourced Temporal Queries)	33
13.3	Named Snapshots	33
13.4	File Serialisation	33
13.5	Topology Diffing & Semantic Hashing	34
13.6	CSR Serialisation for Archival	34
13.7	Memory-Mapped Loading (Out-of-Core Execution)	35
13.8	Use Cases for Historical Querying	35
14	Reactive Events	35
14.1	Reactive Policy Loops: Streaming Graph Ingestion	35
14.2	Event Lifecycle and Async Dispatch Architecture	36
14.3	Closed-Loop Automation Cycle	37
14.4	Event Types	37
14.5	Python Subscription API	37
14.6	Subscription Patterns	38
14.7	Use Cases	38
15	Distributed Deployment	39
15.1	When to Use Clustering	39
15.2	Raft Consensus via OpenRaft	39
15.3	Configuration	40
15.4	Leader and Follower Roles	40
15.5	Snapshot Transfer	41
15.6	Security and Multi-Tenancy	41
16	Storage Architecture	41
16.1	Rationale for Dual-Write	41
16.2	petgraph StableDiGraph	41
16.3	Polars Columnar Store	42
16.4	Memory Fragmentation and Defragmentation	42
16.5	Dual-Write Consistency Protocol	43
16.6	Foreign Function Interface & Zero-Copy Boundaries	44
16.7	EventStore Ring Buffer	45
16.8	Pluggable Backends	45
17	Schema & Type System	45
17.1	Pydantic-to-DataFrame Column Mapping	45
17.2	Wide Columnar Storage	45
17.3	Schema Validation	46
17.4	Current Limitations	46

18	Algorithm Reference	46
18.1	Worst-Case Optimal Joins (Leapfrog Triejoin)	46
18.2	Betweenness Centrality (Brandes)	47
18.3	CSR Memory Layout	47
18.4	Filter-First Query Planning	48
18.5	Incremental View Maintenance (IVM)	48
18.6	GPU SSSP as Wavefront Relaxation	48
18.7	SPOF Discovery (Tarjan Bridges and Articulation Points)	49
18.8	Redundancy Capacity (Max-Flow / Min-Cut)	49
19	Query System	50
19.1	Design Goals	50
19.2	Query Representations	50
19.3	Execution and Optimisation	50
19.4	User-Facing Interfaces	56
19.5	End-to-End Query Trace	60
20	NTE-QL Language Reference	61
20.1	Implemented Syntax	61
20.2	Supported Operators	65
20.3	Relationship to QuerySpec and PatternNode	65
20.4	Implementation Status	65
21	GPU Acceleration	66
21.1	Architecture Overview	66
21.2	Hardware Acceleration State	66
21.3	SSSP Kernel	66
21.4	Connected Components Kernel	67
21.5	CPU Fallback	67
21.6	When to Use GPU Acceleration	67
21.7	Invoking GPU Algorithms from Python	67
22	Graph Algorithm Plugins	68
22.1	Plugin Architecture	68
22.2	Built-In Algorithm Plugins	68
22.3	Domain-Specific Graph Kernels	69
23	Production Diagnostics	69
23.1	Health and Memory Endpoints	69
23.2	Chaos Engineering Endpoints	69
23.3	TUI Diagnostics Dashboard	70
IV	REFERENCE & EVALUATION	
24	API Reference and Usage Guide	70
24.1	End-to-End Example	70
24.2	Core Method Reference	71
24.3	Error Handling	72
24.4	Threading and Concurrency	72
24.5	Common Patterns	72
25	Related Work	72
25.1	Native Graph Databases	72
25.2	Relational and Hybrid-Relational Approaches	73
25.3	Network-Specific Tools	73
25.4	Infrastructure-as-Code and Intent-Based Networking	73
25.5	Graph Processing Frameworks	73
25.6	Positioning Summary	74
26	Limitations and Future Work	74
26.1	Current Limitations	74
26.2	Recently Completed Improvements	74
26.3	Planned Capabilities	75
26.4	Short-Term Roadmap	76
26.5	Long-Term Vision	76

27 Performance Characteristics	76
27.1 Complexity of the Hybrid Pipeline	77
27.2 Internal Scaling Characterisation	77
27.3 Latency Breakdown.	77
27.4 Incremental View Maintenance: Preliminary Results	77
27.5 Representative Micro-benchmarks	78
28 Lessons Learned	78
29 Conclusion	80
29.1 Generalisable Insights	80
A Cargo Workspace Crates	81
B Legacy Configuration Support	81
C NTE-QL Grammar (EBNF)	82
C.1 Worked Examples	82
Revision History	84

1 Introduction

Modern network infrastructure is a graph of physical and logical dependencies, but the data used to operate that graph is property-heavy and relational.

Engineers need to answer two kinds of questions:

1. **Structural traversal:** "Is there a physically diverse path between Router A and Router B within four hops?"
2. **Relational filtering:** "Which routers in the Sydney data centre have an MTU below 9000 bytes?"

General-purpose systems handle one class of question well and the other poorly.

Why graph databases are not enough. Native graph databases such as Neo4j [1] store properties in row-oriented record chains. A broad predicate scan—"all routers where pop = "SYD"—requires pointer-chasing through each candidate node's property chain, producing $O(V)$ random reads even when the result set is a tiny fraction of the graph. In network topologies, where 90% of queries begin with a property constraint (site, vendor, role, AS number), this cost dominates execution time. Columnar stores eliminate this overhead via SIMD-accelerated linear scans over contiguous memory, but they cannot natively express multi-hop structural traversals.

Why relational databases are not enough. Relational databases can express graph traversals through recursive Common Table Expressions (CTEs). However, recursive CTEs suffer from intermediate materialisation: each recursion level produces a full result set, and the planner has limited ability to push predicates into the recursive term [2]. A four-hop reachability query over a 10,000-node topology can generate millions of intermediate rows before the final WHERE clause prunes them. The query plan explosion grows combinatorially with path depth, making deep structural queries impractical in a relational engine.

The hybrid opportunity. NTE (Network Topology Engine) combines these two workloads in one engine: it stores topology structure in a graph adjacency list for $O(1)$ -per-hop traversal, and properties in a columnar store for SIMD-accelerated predicate evaluation. The resulting hybrid pipeline reduces the dominant traversal cost by the predicate selectivity factor (Proposition 1). Mutations are coordinated across both stores through a dual-write transaction model that guarantees consistency.

This report makes four main contributions:

1. A **dual-write storage model** that keeps graph structure and columnar properties atomically synchronised, with RAIL-guarded rollback on partial failure (Section 12.3).
2. A **network-specific topology model** based on explicit endpoints and layered structure, enabling port-level failure simulation that naive $G = (V, E)$ models cannot express (Section 2).
3. A **filter-first query pipeline** that lowers semantic Cypher-like queries onto the structural model, executing property predicates via Polars before graph traversal to achieve sub-linear candidate reduction (Section 19).
4. A **policy and branching workflow** for validating proposed topology changes against structural invariants before commit, using copy-on-write shadow topologies and a Starlark-based policy engine (Section 12.1).

1.1 Why a Bespoke Engine?

NTE is built for network automation workloads that demand low latency, structural fidelity, and deterministic execution:

- **Embedded execution:** NTE runs in-process via PyO3 or native Rust linking, so automation pipelines avoid network round-trips to an external database. This eliminates the serialisation overhead and operational complexity of a client-server graph database.
- **Filter-first execution:** Many network queries begin by excluding most entities on property predicates before any traversal starts. NTE performs these filters on columnar data via SIMD-accelerated scans, then traverses only the reduced subgraph.
- **Deterministic compilation:** Configuration synthesis and policy validation depend on stable, reproducible behaviour. An embedded library with no external state dependencies provides this guarantee naturally.

Table 1 positions NTE against several adjacent systems by architecture and execution model. For a detailed discussion of related work, see Section 25.

Table 1. Architectural Positioning: NTE vs. Adjacent Systems. This table summarises high-level trade-offs; it is not a benchmark claim.

Feature	Neo4j	PostgreSQL	Batfish	NTE
Primary Model	Graph (Row)	Relational (Row)	Simulation	Hybrid
Query Execution	Cypher	SQL (CTE)	Declarative	Filter-First Graph
Deployment	Standalone DB	Standalone DB	CLI / Service	Embedded Library
“What-If” Branches	Medium (Clone)	Slow (Snapshot)	Fast	Instant (CoW)

1.2 Running Example: Data Centre Migration

To ground the abstractions described in this report, we use a recurring scenario: an operator planning the migration of a regional data centre (pop = “SYD-1”) to a new facility (SYD-2). The migration involves physically relocating routers and re-homing BGP sessions. Before committing, the operator must verify that no single point of failure is introduced and that all OSPF areas remain contiguous.

This scenario exercises several NTE subsystems in sequence:

1. **Property query** (Section 19): Identify all affected devices with `MATCH (n:Router) WHERE n.pop = "SYD-1" RETURN n`. The columnar store evaluates the pop predicate via a SIMD scan; no graph traversal is needed.
2. **Structural traversal** (Section 19): Discover all BGP sessions reachable from the affected routers with `MATCH (n:Router {pop:"SYD-1"})-[:Connectivity*2]-(peer) RETURN peer`. The filter-first pipeline applies the property predicate before expanding the two-hop neighbourhood.
3. **What-If branching** (Section 12.1): Create a copy-on-write shadow topology, apply the proposed mutations (`SET n.pop = "SYD-2"` for each relocated router), and verify invariants against the shadow without affecting the live state.
4. **Policy validation** (Section 5): Run a Starlark policy rule that asserts OSPF area contiguity. The policy engine calls `algo.clustering.connected_components` on the OSPF-layer subgraph; if any area is partitioned, the shadow commit is rejected.
5. **Vulnerability analysis**: Call `algo.vulnerability.spoof` on the post-migration topology to confirm that no new single points of failure have been introduced.

This end-to-end workflow—property scan, structural traversal, shadow mutation, policy check, vulnerability analysis—completes in a single transactional boundary. The individual subsystems are described in the sections that follow.

1.3 Ecosystem: Embeddable Engine vs. Remote Server

NTE supports two execution modes.

Embedded mode: NTE runs inside the host process.

- **Python (ank_pydantic):** PyO3 bindings expose the engine to data-science and automation workflows.
- **Rust (ank_netcfg):** The engine links directly into the configuration compiler.

Remote mode: NTE runs as a standalone service.

- **n-te-server** exposes HTTP and WebSocket interfaces. Arrow Flight for bulk columnar transfer is planned (see Section 26).
- **Remote clients** can execute the same queries against a separate server instance.

2 Design Principles of Network Modeling

A naive graph model defines a network as $G = (V, E)$, where V are devices and E are the cables between them. That model is simple, but it cannot express many of the structural constraints that matter in network engineering. Networks are shaped by attachment points, containment hierarchies, layers, and rich properties. NTE therefore uses a stricter topology model designed for algorithmic evaluation.

2.1 The Fallacy of the Simple Graph (The Endpoints Model)

In the standard $G = (V, E)$ abstraction, cables plug into routers. In practice, cables plug into transceivers, which sit in line cards, which sit in chassis slots.

If a line card fails, a naive graph cannot determine which links fail with it unless the system adds extra structure outside the graph itself.

Design Rule 1: The Attachment Point Ontology

To provide a universal algorithmic canvas, NTE requires all physical and logical connections to be mediated by an explicitly modelled **Endpoint**. A connection between two devices is never a single edge, but a strict three-hop composition: Node Structural Endpoint Connectivity Endpoint Structural Node.

This normalisation lets algorithms such as shortest path or failure simulation operate on a uniform topology without vendor-specific parsing.

Concrete payoff: line-card failure simulation. Consider a chassis router with two line cards, each hosting four Ethernet ports. In a naive $G = (V, E)$ model, removing a line card requires external logic to determine which of the router's eight edges correspond to ports on the failed card—information that the graph itself does not encode. In NTE's endpoint model, the line card is a node whose ports are endpoints connected via *Structural* edges. Simulating the failure is a single graph operation: remove the line card node, and all *Structural* edges to its endpoints are severed, which in turn severs the *Connectivity* edges that traverse those endpoints. The blast radius—exactly which remote peers lose connectivity—is determined by the graph structure alone, with no out-of-band metadata lookup. This is the difference between “the graph *contains* the answer” and “the graph *requires* an answer from elsewhere.”

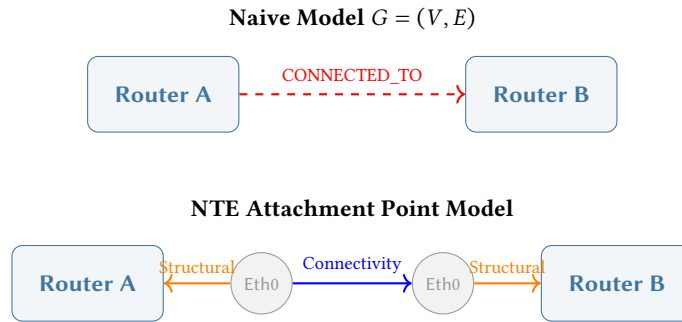


Figure 1. The Naive Graph vs. The NTE Endpoints Model. The naive model abstracts away physical attachment points, making it impossible to simulate port-level failures. NTE enforces a strict 3-hop composition to preserve algorithmic blast-radius fidelity.

2.2 The Fractal Nature of Nodes

Network devices are not atomic. A chassis may contain switching fabrics, redundant route processors, line cards, and Virtual Routing and Forwarding (VRF) instances.

NTE treats topology as **fractal**. The same graph primitives used to model a wide-area network are also used to model the containment hierarchy inside a single device. With a consistent internal and external structure, an algorithm written to trace a WAN path can also trace a path through a modular chassis.

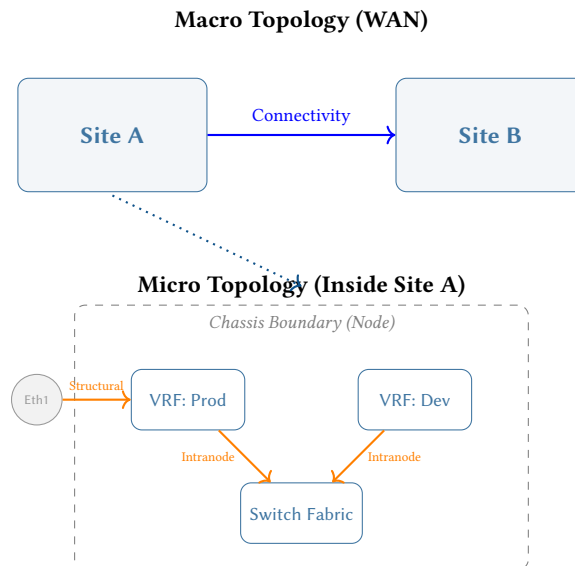


Figure 2. The Fractal Nature of Topology. The exact same graph primitives (Nodes, Endpoints, Structural/Intranode edges) used to construct a macro WAN are used to construct the micro containment hierarchy inside a single modular chassis.

2.3 Property Flattening (Algorithmic Accessibility)

Many network automation systems store domain semantics in deeply nested JSON or XML trees.

When data is buried in nested structures, generic graph algorithms cannot use it directly. A traversal engine does not know how to parse a vendor-specific JSON block to recover interface speed or routing metric.

Design Rule 2: Property Flattening

NTE decouples domain semantics from graph structure by flattening properties into key-value pairs attached directly to topological primitives. This turns nested vendor data into a form that generic algorithms can filter and evaluate.

Users remain free to define their own schemas, but once those schemas are projected onto the structural model, generic algorithms can evaluate predicates such as "traverse all links where speed $\geq 100G$ " without custom parsers.

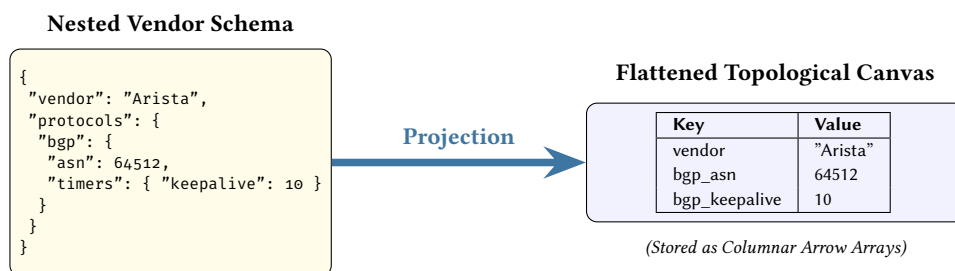


Figure 3. Property Flattening. Deeply nested vendor-specific data models are projected into a flat, key-value tabular structure attached directly to topological primitives, making them accessible to SIMD-accelerated generic graph algorithms.

2.4 Orthogonal Layering (The Multi-Plane Abstraction)

Networks operate in multiple layers. A physical fibre cut (Layer 1) may trigger an IP adjacency drop (Layer 3), which may bring down a BGP session, which may in turn withdraw a VPN route.

NTE models layers as orthogonal graph planes linked by strict hierarchical dependencies (Parent edges). This lets the engine treat the BGP overlay as its own graph for routing algorithms while preserving the lineage needed to trace cascading physical failures.

Crucially, edges are strictly segregated by their topological role:

- **Intra-Layer Edges (Connectivity):** These edges map peer-to-peer relationships *strictly within the same layer*. An IP endpoint can only have a Connectivity edge to another IP endpoint; a physical port to a physical port.

- **Inter-Layer Edges (Parent):** These edges map the vertical stack *across different layers*. A BGP session endpoint connects downwards via a Parent edge to the IP interface that hosts it.

This typing ensures that layer-specific algorithms do not cross into other layers unless the query or algorithm asks them to.

3 Data Model

Axiom 1 (Dual-Write Atomicity). *Every mutation to the topology must be reflected atomically in both the graph adjacency store and the columnar property store. A transaction is only committed if both representations are successfully updated.*

Design Rule 3: Graph-Relational Equivalence

The set of all nodes V and edges E reachable via graph traversal is equivalent to the set of rows in the primary Polars DataFrames. v DiGraph, !row DataFrame mapping to v . This invariant is enforced by the dual-write transaction model (Section 12.3), not proven as a formal theorem.

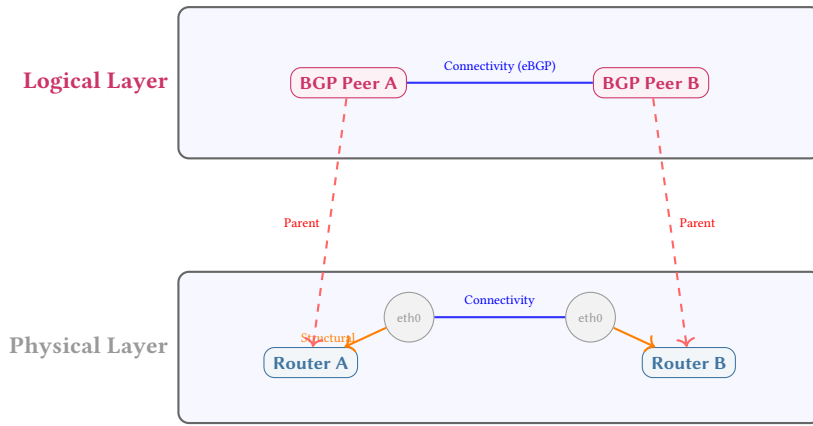


Figure 4. Core Data Model Overview. The topology is divided into isolated Layer planes. Nodes and Endpoints reside within layers, connected by intra-layer *Connectivity* edges and bound together by intra-node *Structural* edges. *Parent* edges provide inter-layer lineage linking logical overlays to physical infrastructure.

3.1 Formal Definition

A topology in NTE is a typed, directed, attributed multi-graph $G = (V, E, V, E, V, E, L,)$ where:

$$\begin{aligned}
 V &= V_{\text{node}} \cup V_{\text{endpoint}} \quad (\text{disjoint node and endpoint sets}) \\
 E &\subseteq V \times V \quad (\text{directed edges}) \\
 V &: V \rightarrow V \quad (\text{type label, e.g. "Router", "Switch"}) \\
 E &: E \rightarrow \{\text{Connectivity, Structural, Intranode,}\} \quad (\text{edge kind}) \\
 V &: V \rightarrow (\text{String JSON}) \quad (\text{property maps}) \\
 E &: E \rightarrow (\text{String JSON}) \quad (\text{edge property maps}) \\
 L & \quad (\text{ordered set of named layers}) \\
 &: V \rightarrow L \quad \{\} \quad (\text{optional layer assignment})
 \end{aligned}$$

Nodes are assigned integer IDs that are globally unique within a topology instance. NTE uses i32 as the external ID type. IDs are chosen by the caller; NTE does not auto-assign them (though helpers exist for auto-increment).

3.2 Vertex Types

NTE recognises two top-level vertex categories and one specialisation.

Terminology note

NTE uses “node” specifically for *network devices* and “endpoint” for their connection points. This differs from graph-theory usage where “node” means any vertex. In this document, “vertex” is used when the graph-theoretic meaning is intended.

3.2.1 Nodes

A **Node** represents a network *device* — a router, switch, server, firewall, optical transponder, etc. (Figure 5). Nodes carry a string `node_type` and an optional layer membership. Property data is stored as an arbitrary JSON object, populated by `ank_pydantic` from Pydantic model fields.

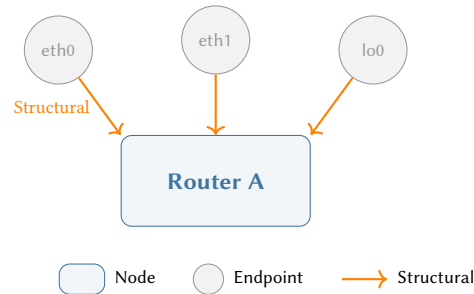


Figure 5. A Node with three Endpoints. Each **Structural** edge points *from* the Endpoint *to* its owning Node, expressing the ownership relationship.

3.2.2 Endpoints

An **Endpoint** is a connection point on a node (Figure 6). Endpoints fall into three broad categories:

Physical endpoints represent hardware attachment points where a cable terminates: Ethernet ports, fibre connectors, transceiver slots. They correspond to tangible components on a device.

Logical endpoints represent protocol-level connection points that exist in software rather than hardware: loopback interfaces, VLAN sub-interfaces, tunnel endpoints (e.g. GRE, VXLAN [3] VTEPs). A single physical port may host multiple logical endpoints via sub-interfacing.

Generic endpoints are used when the physical/logical distinction is irrelevant or unknown — for example, a telemetry source reporting an interface name without classifying it. Generic endpoints allow ingestion pipelines to populate the topology without forcing premature classification.

Each endpoint carries an optional `endpoint_name` (e.g. `GigabitEthernet0/1`, `lo0`, `Vlan100`) for human identification, alongside a type label (e.g. "Endpoint", "Interface") and an arbitrary property map.

Design Rule 4: Endpoint Ownership Invariant

Every endpoint is owned by exactly one node via a Structural edge. An endpoint cannot exist unattached. This invariant is enforced at creation time and preserved by all mutation operations.

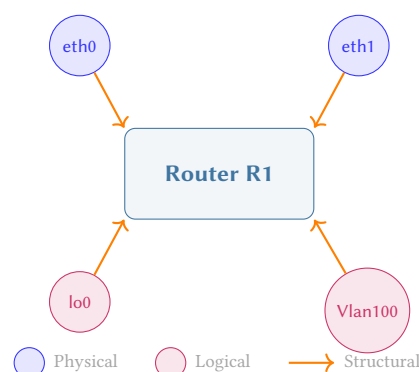


Figure 6. A device with mixed physical and logical endpoints. Physical endpoints (`eth0`, `eth1`) represent hardware ports; logical endpoints (`lo0`, `Vlan100`) represent software-defined connection points. All are owned by the same node via Structural edges.

3.2.3 Logical Nodes

A **LogicalNode** (Figure 7) is a specialised node that *always* belongs to a layer and represents protocol-level or virtual topology objects that do not correspond to physical hardware. Typical examples include BGP [4]

session state machines, OSPF [5] or IS-IS [6] areas, VLAN broadcast domains, and VPN routing/forwarding (VRF) instances.

Key distinction from regular Nodes. A LogicalNode *requires* a layer assignment (non-optional), whereas a regular Node may exist without one. This mandatory binding ensures that every logical object is anchored in a specific plane of abstraction, enabling the engine to reason about cross-layer dependencies.

Query visibility. LogicalNodes are excluded from default query results to avoid polluting device-centric queries. Targeting them requires explicitly matching their layer — for example, `MATCH (s:BGPSession) WHERE s.layer = "bgp"` in NTE-QL. This opt-in visibility prevents protocol-state objects from appearing in infrastructure queries that expect only physical devices.

Common patterns. A LogicalNode in the BGP layer linked via a Parent edge to a physical Router represents “this router participates in this BGP session.” Similarly, an OSPF area LogicalNode may have Parent edges to all routers in that area, providing a single query target for area-wide analysis.

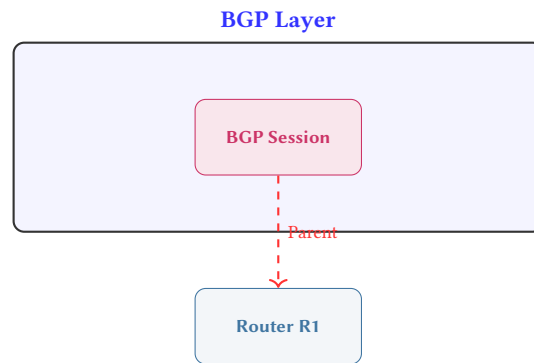


Figure 7. A LogicalNode (BGP Session) residing within a layer plane, linked to its parent physical Node via a **Parent** edge. Logical nodes are excluded from default queries unless their layer is explicitly targeted.

3.3 Edge Kinds

NTE defines three principal external edge kinds. The engine also uses several structural edge kinds (Parent, Child, Root, Origin) that are fully accessible from Python via methods such as `add_parents()`, `get_parent_nodes()`, and `add_roots()`.

Edge Terminology Mapping

The Rust engine uses the names Connectivity, Structural, and Intranode internally. The Python API exposes these as `add_inter_edges` (Connectivity), `add_intra_edges` (Structural), and `add_intranode_edges` (Intranode) for historical compatibility with `ank_pydantic`.

3.3.1 Structural (Ownership)

A **Structural** edge (Figure 8) runs *from* an endpoint *to* the node that owns it. It expresses the “this port belongs to this device” relationship. A node may own any number of endpoints; each endpoint has exactly one owning node. The Python method is `add_intra_edges`.

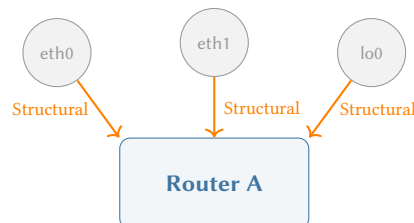


Figure 8. Structural (Ownership) edges. Three Endpoints are owned by a single Node. Arrows point *from* Endpoint *to* Node, expressing the “belongs to” relationship.

3.3.2 Connectivity

A **Connectivity** edge (Figure 9) connects two endpoints. It models a physical or logical link between two ports on different devices. Connectivity edges are bidirectional by convention: NTE stores them as a pair of directed edges. The Python method is `add_inter_edges`.



Three-hop path: Router A—Structural—eth0—Connectivity—eth0—Structural—Router B

Figure 9. Connectivity edges between two devices. Each device connects via its Endpoint. A bidirectional link is stored as two explicit directed edges. The three-hop traversal path is shown below.

3.3.3 Intranode

An **Intranode** edge connects two nodes within the same logical chassis or container. Where Structural edges express *ownership* (an endpoint belongs to a device), Intranode edges express *co-location* (two nodes share a physical chassis or logical container). These are fundamentally different relationships: ownership is hierarchical and one-to-many; co-location is peer-to-peer and many-to-many.

Use cases.

- **Multi-VRF routers:** each VRF is a separate Node, but all inhabit the same physical chassis. Intranode edges link VRF nodes to the Global Routing Table node (see Figure 10).
- **Modular chassis:** line cards are child Nodes connected to the chassis supervisory module via Intranode edges.
- **Virtual router instances:** multiple VR instances inside a hypervisor, sharing the same compute host but appearing as independent nodes.

Direction convention. Intranode edges are directed from child to parent (e.g. VRF → Global RT), following the same “points toward the owner” convention as Structural edges. Bidirectional peer relationships (e.g. VRF A ↔ VRF B for route leaking) are modelled as two explicit directed edges.

Implementation Note

The current Rust implementation unifies Intranode under the Structural edge kind in petgraph, with the semantic distinction tracked via event types (IntraEdgeCreated vs IntranodeEdgeCreated). This pragmatic simplification avoids a separate edge-kind enum variant whilst preserving the conceptual separation at the API layer. The Python method is `add_intranode_edges`.

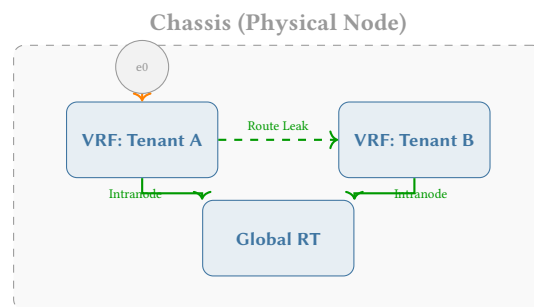


Figure 10. Intranode edges within a chassis. VRF sub-nodes connect to the Global Routing Table via **Intranode** edges. Each sub-node can optionally own its own Endpoints (shown on VRF: Tenant A). The dashed edge represents a route-leak relationship.

3.3.4 Device Shortcut (DeviceLink)

When a Connectivity edge is created between two endpoints, NTE automatically adds a **DeviceLink** shortcut edge between the owning devices. This avoids the three-hop traversal ($DeviceA \rightarrow PortA \rightarrow PortB \rightarrow DeviceB$) in the common case where callers only want device-to-device connectivity. The shortcut is maintained in `device_shortcuts: HashMap<(i32, i32), Vec<EdgeIndex>` inside `NteGraph`.

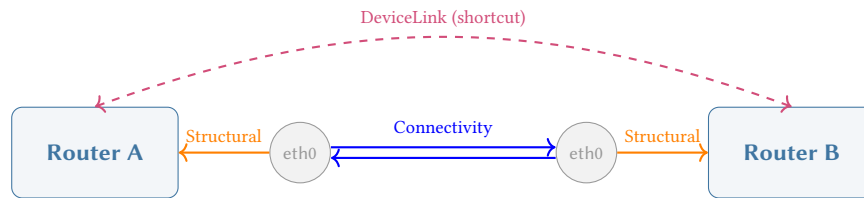


Figure 11. The DeviceLink shortcut. The dashed purple edge bypasses the three-hop Structural–Connectivity–Structural path, providing direct device-to-device adjacency for common queries.

3.3.5 Semantic (Named Virtual Edges)

A **Semantic** edge carries a named label (e.g. `Semantic("bgp")`, `Semantic("ospf_area_0")`) and represents a user-defined logical relationship that exists purely at a named layer. Unlike Connectivity edges, which connect endpoints, Semantic edges connect *nodes directly* and bypass the Endpoints model entirely.

Use case. Semantic edges overlay protocol relationships that do not correspond to physical cables. For example, an OSPF adjacency between two routers that share a broadcast domain, or a management-plane “monitors” relationship between a collector and its targets. In each case, the relationship is meaningful at the logical layer but has no single physical attachment point worth modelling.

Trade-off. Semantic edges sacrifice the blast-radius precision of the Endpoints model for query ergonomics. Because they skip the three-hop pattern, failure propagation through Semantic edges cannot isolate individual port-level failures. They should be used when the physical attachment point is irrelevant to the analysis.

Python API. The method `add_semantic_edges(layer, source_ids, target_ids)` creates Semantic edges within the specified layer. The `layer` parameter must name an existing registered layer.

3.4 Composite Patterns

The vertex types and edge kinds defined above combine into recurring structural patterns that appear throughout real network topologies.

3.4.1 The Three-Hop Pattern

The most fundamental traversal in NTE is the *three-hop pattern*: to walk from one device to another, the path always passes through Device A—**Structural**—Endpoint—**Connectivity**—Endpoint—**Structural**—Device B. This pattern is a direct consequence of the Endpoint model and is the reason DeviceLink shortcuts exist (see Figure 11).

Abstraction Leak

The necessity of the three-hop pattern highlights an abstraction leak: the physical reality of network ports complicates basic queries. This is precisely why NTE-QL introduces **Virtual Cypher Translation** (Section 4), which automatically collapses these endpoint traversals when a user executes queries at the device level, removing the cognitive burden from the engineer.

3.4.2 Endpoint Lifecycle

In practice, endpoints are created and wired through a predictable sequence:

1. **Creation:** Endpoints are typically created alongside their owning node in a single batch operation (e.g. `add_nodes` with an `endpoints` parameter). The engine atomically creates the node, its endpoints, and the Structural ownership edges in one transaction.
2. **Wiring:** Connectivity edges are added between endpoints on different nodes to establish links (`connect_endpoints`).
3. **DeviceLink shortcut:** NTE automatically creates the DeviceLink shortcut edge when a Connectivity edge is added, requiring no explicit action from the caller.
4. **Validation:** In Strict mode, endpoint creation requires specifying the parent node and layer. The engine validates ownership consistency and layer compatibility before committing.

3.4.3 Internal Structure (Chassis, VRFs, Line Cards)

Nodes can optionally expose internal sub-structure. A router may be treated as an atomic device, or it may be decomposed into child nodes representing line cards, forwarding engines, or VRF instances. This decomposition is modelled as child Nodes connected via Intranode edges within a chassis boundary (see Figure 10).

The internal structure is entirely optional: a node with no Intranode edges behaves as an opaque, atomic device. When sub-structure is present, each child node can own its own Endpoints, enabling per-line-card or per-VRF connectivity modelling.

Because **Intranode** edges represent structural ownership rather than external connectivity, they are implemented in the Rust backend as a subset of the `Structural` edge kind. In NTE-QL, users can explicitly query these internal sub-structures by matching against the `:Structural` relationship:

```
// Find all VRFs contained within a specific physical chassis
MATCH (chassis:Router)-[:Structural]->(vrf:VRF)
WHERE chassis.hostname = "core-1"
RETURN vrf.name, vrf.route_target
```

Because this query specifically requests a `Structural` edge rather than a semantic relationship type (like `:eBGP`), the Query Planner recognises this as an explicit internal query and **bypasses Virtual Cypher expansion**. This allows operators to directly dive into the sub-components of a node without switching query languages.

3.4.4 Cross-Layer Relationships

A topological digital twin must track how a high-level logical construct (like a BGP session) relies on lower-level physical infrastructure (like a fibre cable). NTE achieves this via **Parent** edges, which span across the independent Layer planes.

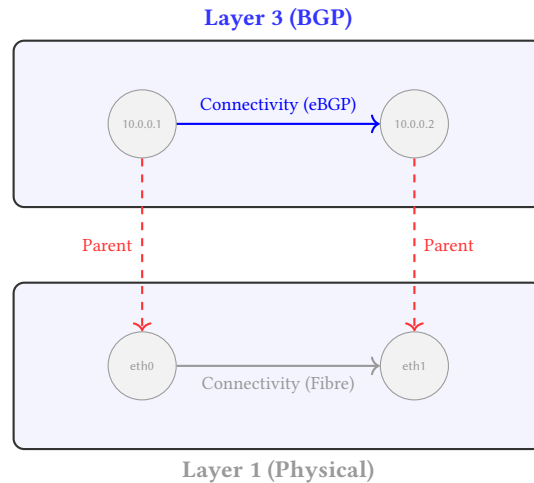


Figure 12. Cross-Layer Relationships. L3 BGP Endpoints are bound to L1 Physical Endpoints via **Parent** edges. If `eth0` fails, the engine traverses the incoming Parent edge to deterministically tear down the dependent BGP session.

3.5 Layer Hierarchy

Layers partition the topology into independent sub-graph planes (Figure 13). Each layer defines its own set of nodes and endpoints that form an independent Connectivity graph. Common examples include a physical layer (hardware links), an ip layer (IP adjacencies), a bgp layer (BGP sessions), and an mpls [7] layer. Layers can share the same physical nodes — a Router may appear in both the Physical and BGP layers — but each layer’s edges are independent.

Layer dependency DAG. Layers declare parent-child dependencies (e.g. BGP depends on IP, IP depends on Physical). NTE validates at registration time that these dependencies form a directed acyclic graph; cycles are rejected with a `LayerCycleError`. This DAG drives the cross-layer traversal primitives (`get_ancestor_in_layer`, `get_descendant_in_layer`) described in Section 3.6.

Interaction with the Endpoints model. Within a single layer, the standard three-hop pattern applies: `Device`—`Structural`—`Endpoint`—`Connectivity`—`Endpoint`—`Structural`—`Device`. Across layers, Parent edges connect a higher-layer endpoint or node to its lower-layer counterpart. For example, a BGP session endpoint at L3 has a Parent edge pointing to the physical Ethernet endpoint at L1 that carries it (see Figure 12).

Strict vs Relaxed enforcement. In **Strict** mode, `connect_endpoints` validates that both endpoints belong to the same layer. In **Relaxed** mode, this constraint is lifted to support ad-hoc telemetry ingestion where layer metadata may be incomplete or inconsistent. The mode is configured per-topology at creation time.

Base layer. A special base layer acts as a cross-cutting plane for nodes that must appear in all layers — for example, a management collector that aggregates data across every protocol plane. Nodes assigned to the base layer are implicitly visible in every other layer’s query scope.

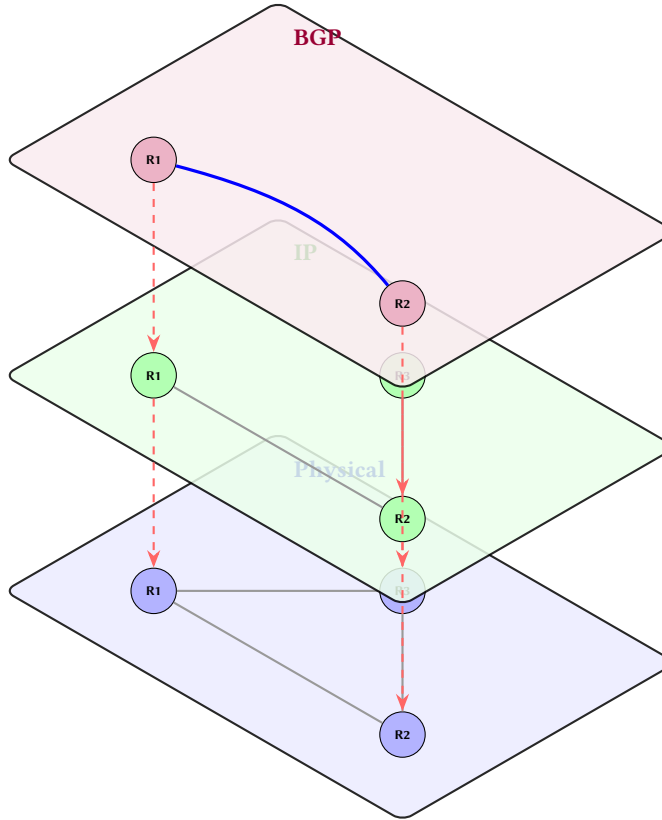


Figure 13. The 3D layered topology model. Each plane uses a distinct colour (blue = Physical, green = IP, purple = BGP). Node labels (R1, R2, R3) are visible inside each circle. Red dashed arrows show cross-layer structural dependencies projecting logical overlays down onto their physical underlays.

Layers are stored internally in a `LayerDependencyGraph` (a DAG of layer names). At edge-creation time, NTE validates that nodes on incompatible layers are not mixed (raising `LayerMismatchError` if so). The topological ordering of layers also determines the default iteration order for cross-layer algorithms.

3.6 Multi-Layer Topology Invariants

A fundamental strength of the NTE engine is its ability to maintain structural invariants across the vertical stack. Most network models treat physical and logical layers as isolated graphs; NTE formalises their relationship through **Ancestor/Descendant** mappings.

Design Rule 5: Cross-Layer Structural Integrity

Every logical node n_L in layer L_i must have a structural mapping $\mathcal{M} : n_L \rightarrow n_P$ where n_P is an underlay node in layer $L_j (j < i)$. If a physical node is removed, all its logical descendants must be either re-parented or marked as “orphaned” to maintain referential integrity.

NTE provides optimised primitives for “tunnelling” through the stack:

- `get_ancestor_in_layer(node_id, layer)`: Efficiently walks the hierarchy to find the physical hardware (e.g., Switch Port) supporting a high-level session (e.g., BGP Peer).
- `get_descendant_in_layer(node_id, layer)`: Projects a physical failure up the stack to identify all impacted logical services.

This cross-layer awareness ensures that queries like “Find the physical switch port supporting this BGP session” are resolved in $O(H)$ time, where H is the height of the layer stack, typically 5.

3.7 Topology Diffing

NTE supports structural comparison between topology instances via `diff_against()`, which computes a `TopologyDelta` describing added, removed, and modified nodes and edges. This is used by `ank_pydantic` to detect configuration drift and to drive change-analysis workflows.

3.8 Property Maps

Both nodes and edges carry arbitrary key-value properties stored as JSON objects. Properties are serialised from Pydantic model instances by `ank_pydantic` and stored in Polars DataFrames as individual columns (one column per field name). The Polars representation enables fast columnar scans, type-aware comparisons, and SIMD acceleration.

Warning

Property columns are created lazily as new field names appear. This means queries against fields that have never been set on any node return an empty result rather than an error. Check column existence before relying on absent-field semantics.

4 Semantic Query Translation

A central problem in network topology databases is the tension between structural fidelity and query ergonomics. To model physical failure domains accurately, NTE uses the strict **Endpoints Model** (Section 3). A naive graph such as `(Router)-[CONNECTED_TO]->(Router)` cannot express which port carries a link or which connections fail when a line card loses power.

In NTE, physical cables and logical sessions are represented as three-hop graph traversals explicitly tracking the attachment points: `(Device) <-[Structural]- (Endpoint) -[Connectivity]-> (Endpoint) -[Structural]-> (Device)`.

Engineers should not need to write that structure out by hand for every query. If the intent is to find an eBGP session, the natural query is `(Router)-[eBGP]->(Router)`. NTE-QL therefore uses **Virtual Cypher Translation** (Figure 15) to lower semantic queries onto the structural model.

4.1 Orthogonal Layering: Layers vs. Kinds

Network engineering spans multiple layers. A single Connectivity edge represents a connection at one specific layer. The NTE-QL grammar therefore separates **Layer** (which subgraph to traverse) from **Type/Kind** (which schema class of relationship to match).

Consider how different layers map to the Endpoints Model:

Example A: The Physical Layer (L1)

Nodes: Router, Switch

Endpoints: Physical transceiver ports (e.g., `GigabitEthernet0/1`)

Connection Kind: wire (with properties like `length`, `media_type`)

Structural reality: `[Router:Core1] <-Structural- [Endpoint:Gi0/1]`

`-Connectivity(wire)- [Endpoint:Te1/1] -Structural-> [Switch:Dist1]`

Example B: The Data Link Layer (L2)

Nodes: Switch, Server

Endpoints: Logical switchports or sub-interfaces (e.g., `eth0.100`)

Connection Kind: trunk or access (with properties like `vlan_id`)

Structural reality: `[Switch:Dist1] <-Structural- [LogicalEndpoint:Vlan100]`

`-Connectivity(trunk)- [LogicalEndpoint:eth0.100] -Structural-> [Server:Web1]`

Example C: The Routing Protocol Layer (L3+)

Nodes: Router

Endpoints: Loopback interfaces or routed IP addresses (e.g., `100`)

Connection Kind: eBGP or iBGP (with properties like `local_asn`)

Structural reality: `[Router:Core1] <-Structural- [IP:10.0.0.1]`

`-Connectivity(eBGP)- [IP:10.0.0.2] -Structural-> [Router:Peer1]`

4.2 Syntax to Data Model Mapping

To maximize expressiveness without compromising structural fidelity, NTE-QL maps its Cypher-like syntax directly to these internal constructs. In a standard graph query like `-[variable:Label {properties}]->`, the label and properties target different mechanical components in NTE:

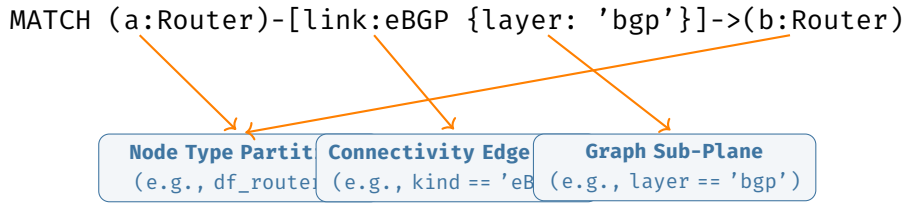


Figure 14. Syntax to Data Model Mapping. NTE-QL explicitly separates the specific relationship type (Kind) from the execution sub-graph (Layer). This allows users to query for *any* connection on the BGP layer, or specifically for *eBGP* connections across any layer.

This mapping allows several useful query forms:

- `MATCH (a)-[:eBGP]->(b)`: Finds eBGP sessions across *any* layer.
- `MATCH (a)-[{layer: 'bgp'}]->(b)`: Finds *any* connection explicitly on the bgp layer.
- `MATCH (a)-[:eBGP {layer: 'bgp'}]->(b)`: Finds specifically eBGP sessions on the bgp layer.

4.3 Virtual Cypher Expansion

When a user submits a query, they do not need to name Structural edges or Endpoint nodes unless the query targets port-level properties.

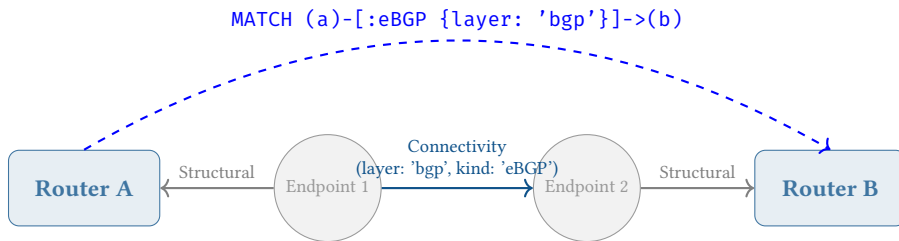


Figure 15. Virtual Cypher Translation. The user specifies a semantic device-to-device relationship (dashed line). The query planner intercepts this and automatically compiles it into the three-hop endpoint traversal (solid lines) required by the structural data model.

The NTE-QL mapping rules are:

- **Cypher Node Label** (e.g., `:Router`): Maps to the `node_type` in the partitioned Polars DataFrame.
- **Cypher Edge Label** (e.g., `:eBGP`): Maps to the `kind` property of the underlying Connectivity edge.
- **Cypher Edge Property** (e.g., `{layer: 'bgp'}`): Defines the sub-graph boundary, instructing the planner to strictly traverse Connectivity edges belonging to the BGP partition.

For example, to find all active eBGP sessions, the user writes:

```
MATCH (a:Router)-[session:eBGP {layer: 'bgp', status: 'active'}]->(b:Router)
RETURN a.hostname, b.hostname
```

The query planner intercepts the Abstract Syntax Tree (AST). If an edge is drawn directly between two device types, the planner inserts the missing endpoint traversal automatically. The resulting execution plan is: find Router nodes → follow Structural edges to endpoints → follow matching Connectivity edges on the bgp layer → follow Structural edges back to target Router nodes.

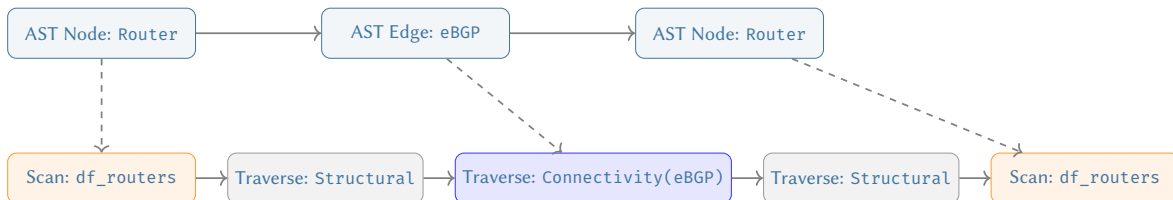


Figure 16. AST Compilation to Execution Plan. The semantic Cypher AST (top) is mathematically expanded by the Query Planner into a deterministic traversal plan (bottom) across the normalised Endpoints graph and partitioned DataFrames.

4.4 Parallel Expansion: The LAG Scenario

The Virtual Cypher abstraction is especially useful for multi-homed connectivity such as Link Aggregation Groups (LAGs) or Equal-Cost Multi-Path (ECMP) routing.

In a physical data centre, two core routers might be joined by four separate 100G fibre links. In the NTE Endpoints Model, this appears as four distinct Connectivity edges between four endpoint pairs.

However, when an engineer queries `MATCH (a:Router)-[link:LAG]->(b:Router)`, they usually want to treat that bundle as one logical connection. The planner handles this through **Parallel Expansion**. It can collapse the four parallel Connectivity paths into one semantic relationship and aggregate underlying endpoint properties such as total bandwidth.

4.5 Context Projections and Sub-Selects

To avoid repeating `{layer: '...'} in complex queries, NTE-QL defines a context-scoping directive, USE LAYER. This instructs the planner to traverse one topological plane for the duration of the query.`

The `USE LAYER` directive is fully implemented in the parser and compiler (see Section 20.4 for the complete language status). It provides a clean way to scope queries without inline property filters.

```
USE LAYER 'physical'
MATCH (a:Switch)-[link:trunk]->(b:Switch)
```

This layer abstraction is especially useful for **cross-layer queries**, where a user wants to trace logical state back to physical infrastructure.

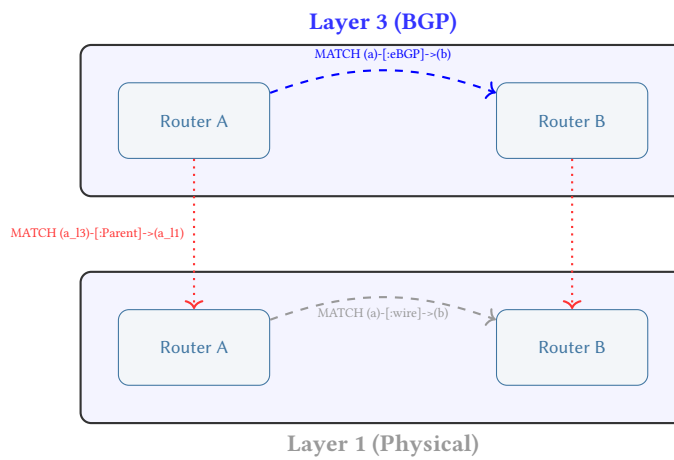


Figure 17. Cross-Layer Semantic Querying. Users can independently query the semantic graph of each layer and trace dependencies across them using the Parent edges, entirely abstracted from the underlying port complexities.

```
// Cross-Layer Query Example
MATCH (r1:Router)-[session:eBGP {layer: 'bgp'}]->(r2:Router)
MATCH (r1)-[:Parent]->(r1_phys)
MATCH (r2)-[:Parent]->(r2_phys)
MATCH (r1_phys)-[cable:wire {layer: 'physical'}]->(r2_phys)
RETURN session.status, cable.media_type
```

4.6 Bridging the Gap: The Benefits of Virtual Cypher

The combination of the Endpoints Model and Virtual Cypher Translation gives the engine four practical benefits:

1. **Uniform structure for algorithms:** Graph algorithms operate on the fully materialised Endpoints Model and can therefore reason about ports, line cards, and shared conduits directly.
2. **Shorter user queries:** Users do not need to write explicit Structural hops when their intent is device-to-device.
3. **No stored shortcut edges:** Translation happens in the planner. The engine does not need to materialise and maintain duplicate semantic edges.
4. **Better partition pruning:** Semantic labels map cleanly onto partitioned columnar data, so queries can skip irrelevant entity classes early.

This planner-driven translation keeps the graph normalised while preserving a concise, domain-specific query language.

4.7 NTE-QL: Projections and State Mapping

Beyond graph traversal, NTE-QL also defines projection features for deriving logical state or aggregating existing metrics. When a query returns or creates a derived view, the engine must still preserve the mapping back to the underlying topology.

4.7.1 Attribute Projection

When executing a RETURN or WITH clause, users can project attributes from the matched path and rename them to fit the output schema. In the columnar engine, these projections are array-selection operations over the relevant columns.

```
MATCH (src:Router)-[link:wire]->(dst:Router)
RETURN
    src.hostname AS origin_device,
    link.bandwidth AS capacity,
    dst.asn AS target_asn
```

4.7.2 Creating Virtual Nodes (Target Mapping)

During complex transformations, or when feeding data to external vis tools, users may need to create new virtual nodes from existing data.

When NTE-QL creates a new virtual node via a CREATE or MERGE statement in a projection, it can track the structural lineage.

CREATE and MERGE statements are fully implemented in the parser, compiler, and executor (Section 20.4). The example below illustrates how a new node can be enriched with `_origin_ids` pointing back to the source nodes that spawned it, preserving the lineage needed to trace a logical summary node back to the physical source.

```
// Aggregate OSPF neighbors into a logical Area Node
MATCH (r:Router {ospf_area: '0.0.0.0'})
WITH collect(r) as backbone_routers
CREATE (area:OSPFArea {name: 'Backbone', target_type: 'LogicalGroup'})
// Lineage mapping happens implicitly via the engine's projection rules
```

4.8 Programmatic Mapping: Python and Starlark

While NTE-QL provides a declarative string interface, NTE is also designed to sit inside larger automation systems. The same semantic translation model therefore appears in the **Python SDK** and the embedded **Starlark Policy Engine**.

4.8.1 Python SDK Execution

In the Python SDK, queries can be built through the fluent builder API. The translation rule is the same: the user specifies a device-to-device relationship, and the Rust core performs the endpoint traversal.

```
# Find all core routers connected via 100G trunks
query = (
    topo.query()
    .match_node("core", node_type="Router", role="core")
    .match_edge("link", edge_type="trunk", layer="physical", bandwidth="100G")
    .match_node("peer", node_type="Router", role="core")
)
results = query.execute()
```

4.8.2 Starlark Policy Invariants

NTE embeds a Starlark runtime to evaluate design rules before mutations commit. Because Starlark is sandboxed, it does not execute arbitrary string queries. Instead, it relies on semantic graph projections exposed through context bindings.

When a Starlark policy checks an invariant such as "all edge switches must have redundant uplinks", it uses the same semantic mapping layer to traverse the underlying physical graph.

```
# Starlark Policy: Verify Redundant Uplinks
def validate_uplinks(ctx):
    # Context bindings automatically abstract the Structural/Connectivity hops
    edge_switches = ctx.find_nodes(type="Switch", layer="physical", role="edge")

    for switch in edge_switches:
        # get_neighbors automatically traverses the 3-hop physical connection
        uplinks = switch.get_neighbors(edge_type="wire", layer="physical")
        if len(uplinks) < 2:
            ctx.fail("Switch {} lacks redundant uplinks", switch.id)

    return ctx.findings()
```

By using the same translation layer in NTE-QL, the Python SDK, and the Starlark policy engine, NTE provides one high-level abstraction across its query interfaces without relaxing the Endpoints Model.

5 Policy Engine

5.1 Design Rationale

Network topologies must satisfy design rules: every edge device must have at least two uplinks, no two devices in the same rack should connect to the same spine, and all transit links must provide at least 100 Gbps. Encoding such rules as ad-hoc Python checks is fragile and hard to maintain.

NTE moves these constraints into a policy layer [8] (Figure 19). This separates *what the rule is* from *when it is evaluated*. Policies can run on demand, before a mutation in a what-if branch, or as part of a CI validation step.

Design Rule 6: Shadow Topology Isolation

NTE leverages its copy-on-write (CoW) architecture to support “What-If” analysis via Shadow Topologies. A mutation can be applied to an ephemeral child snapshot, validated against the full policy engine, and discarded without ever modifying the primary production state. This provides an isolated validation workflow for network-wide changes.

5.2 Policy Specification

NTE supports more than one policy format. The Starlark engine provides the most mature execution path, while NTE-QL serves as the declarative query form. Most NTE-QL constructs used in this section are fully implemented; exceptions (notably the `count{}` sub-query syntax) are marked explicitly.

At its simplest, a policy is a named rule that returns the part of the topology that violates a constraint.

For example, a policy that requires all devices to meet a 100G speed threshold can be expressed as a query for non-compliant nodes:

Listing 1. Policy expressed as an NTE-QL violation query

```
// P001: All devices must support 100G
MATCH (n:Device)
WHERE n.speed_gbps < 100
RETURN n.id, n.speed_gbps
```

Structural invariants can be expressed in the same style:

Listing 2. Structural policy: transit exit redundancy (planned syntax)

```
// P002: Every PoP must have at least 2 transit exits
MATCH (p:PoP)
WHERE count { (p)-[:HAS_EXIT]->(e:TransitExit) } < 2
RETURN p.id, count { (p)-[:HAS_EXIT]->(e:TransitExit) }
```

The `count{}` sub-query syntax used in P002 is a planned NTE-QL extension (Section 20.4). This policy can currently be expressed using the Starlark engine or the Python fluent query API.

When integrated into a CI/CD pipeline, these rules are executed by the `PolicyEngine`. Any non-empty result set becomes a policy finding. For legacy integration and configuration management, a YAML-based specification is also supported (see Appendix B).

attribute Evaluated per node/edge. The expression is checked against each entity’s property map independently.

structural Evaluated once against global topology metrics (counts, ratios, aggregate properties).

5.3 The Starlark Policy Engine

For policies requiring programmatic logic beyond simple comparisons, NTE includes a Starlark-based policy engine. Starlark is a Python-like scripting language designed for safe, deterministic execution in embedded environments.

Design Rule 7: Lazy Host Bindings

To eliminate the $O(N)$ serialization bottleneck when evaluating policies against large topologies, the engine exposes the `NteGraph` directly to Starlark via lazy host bindings. Instead of pre-copying all node properties and adjacencies into the evaluation context, the Starlark runtime performs $O(1)$ lookups into the core Rust memory buffers only when a script explicitly requests a property or neighbor.

- **Sandboxed:** no filesystem access, no network access, no mutable global state.
- **Deterministic:** no random number generation, no timing functions.
- **Familiar:** syntax is a subset of Python, comfortable for network engineers.

- **Fast:** zero-copy memory access to the topology substrate.

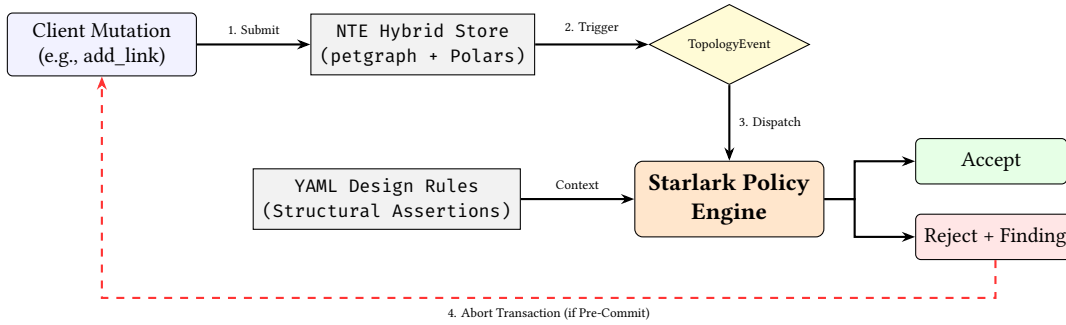


Figure 18. Reactive Policy Pipeline. Structural mutations trigger events that invoke the Starlark runtime. In a pre-commit context (e.g., What-If Branching), a policy violation automatically aborts the transaction before it is persisted.

Listing 3. Starlark policy example: uplink redundancy

```

# Policy: every leaf switch must have at least 2 uplinks
def validate_leaf_uplinks(node_id, uplink_count, required):
    if uplink_count < required:
        fail("Node {} has only {} uplinks, need {}".format(
            node_id, uplink_count, required
        ))
    return True

# Host function bound from Rust: check_uplinks(node_id, required)
# Returns True if the node has >= required uplinks
result = check_uplinks("leaf-01", 2)
result # Starlark scripts return the value of their last expression

```

5.4 Policy Evaluation (CEL & Starlark)

The PolicyEngine pre-compiles policy expressions at load time. To balance performance and expressiveness, NTE uses a tiered execution model:

CEL (Common Expression Language) Used for high-performance attribute and structural checks. CEL is a declarative language designed for safe, fast evaluation. It is suited to simple property comparisons and existence checks.

Starlark Used for complex procedural logic that requires loops, conditionals, or custom functions (e.g., recursive path validation).

At evaluation time, the engine performs the following steps:

1. **Global Context Injection:** Structural policies are evaluated once against a global `RuntimeContext` containing aggregate topology metrics (e.g., total node count, layer distribution).
2. **Subject Iteration:** Attribute policies are evaluated for every matching subject (node or edge). The engine binds `this` (the entity ID) and `attrs` (the property map) into the evaluation scope.
3. **Deterministic reporting:** Each violation produces a stable `Finding` report. These reports are deterministic and can be exported as JSON (v1 schema) for automated reconciliation.

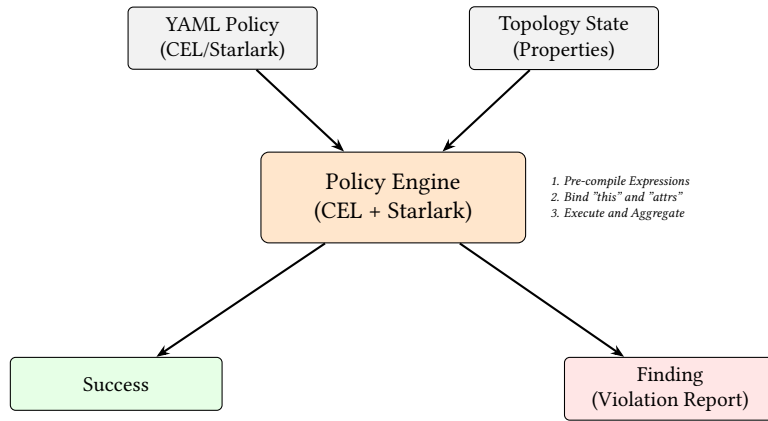


Figure 19. Policy Evaluation Pipeline. The engine pre-compiles declarative CEL expressions and procedural Starlark logic, executing them against a unified topology context to generate structured findings.

5.5 Shadow Topologies and DeltaNet Validation

NTE supports *what-if* analysis via `ShadowTopology`: a clone of the live topology onto which a proposed `TopologyDelta` is applied without affecting the real state.

Warning

Snapshot memory cost: Polars DataFrames use Arrow CoW buffers, so property cloning is effectively $O(1)$. By contrast, `petgraph::StableDiGraph` is backed by Rust Vecs. Creating a shadow topology therefore requires an $O(|V| + |E|)$ deep copy of the structural graph. For topologies above 100,000 nodes, creating a shadow or starting a transaction will introduce a millisecond-scale CPU stall.

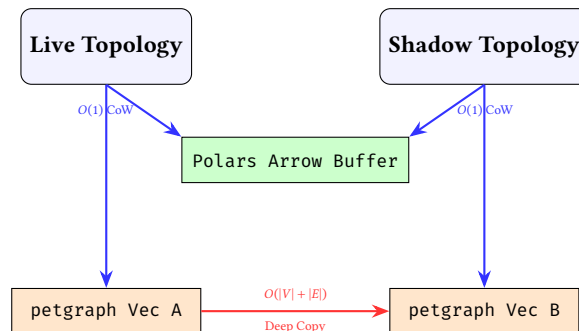


Figure 20. The Hybrid Snapshot Reality. While property DataFrames share memory efficiently via CoW pointers, the underlying petgraph adjacency lists must be physically deep-copied into new memory allocations.

5.6 DeltaNet: Incremental Policy Validation

Feature-Gated

The `DeltaNetValidator` is behind the `delta-net` feature flag. In the current implementation, Starlark validation scripts cannot yet query the simulated topology directly; they operate on pre-extracted property maps.

The `DeltaNetValidator` (in `n-te-policy`) provides an optimised alternative to global topology validation. Rather than re-evaluating every policy against the entire graph, DeltaNet uses an **Impact Radius** algorithm to identify the minimal set of nodes affected by a mutation.

Design Rule 8: Impact-Aware Validation

For any mutation, the validator computes the affected subgraph $\mathcal{G}$ using a bounded BFS traversal of depth k (where k is the maximum dependency depth of the active policies). Only nodes within $\mathcal{G}$ are re-validated, reducing validation complexity from $O(V)$ to $O(V)$.

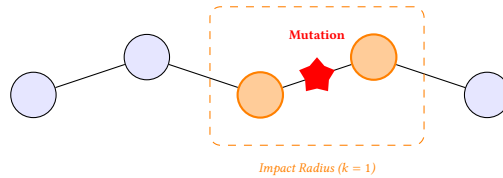


Figure 21. Incremental Validation via Impact Radius. By identifying the nodes physically and logically adjacent to a mutation, NTE avoids the $O(V)$ cost of global policy re-evaluation.

This incremental approach is essential for large-scale production networks where a single property update (e.g., a BGP weight change) should not trigger a multi-second validation sweep of 100,000 devices.

5.7 Worked Policy Examples

5.7.1 Transit Redundancy

Listing 4. Transit redundancy policy

```
# Every PoP must have at least 2 transit exit links
def check_transit_redundancy(pop_name, exit_count):
    if exit_count < 2:
        fail("PoP {} has only {} transit exits (minimum 2)".format(
            pop_name, exit_count
        ))
    return True

check_transit_redundancy(pop_name, transit_exit_count(pop_name))
```

5.7.2 Peer Diversity

Listing 5. NTE-QL policy: peer diversity (planned syntax)

```
// P003: No two BGP sessions should share the same peer AS (from a given router)
MATCH (r:Router)-[:HAS_SESSION]->(s:BgpSession)
WITH r, s.peer_as as peer_as, count(s) as session_count
WHERE session_count > 1
RETURN r.id, peer_as, session_count
```

5.7.3 OSPF Area Consistency

Listing 6. NTE-QL policy: OSPF Area consistency (planned syntax)

```
// P004: Ensure all interfaces in a broadcast domain share the same OSPF Area
MATCH (domain:BroadcastDomain)-[:Parent]-(ep:Endpoint)
MATCH (ep)-[:Connectivity {layer: 'ospf'}]->(proto:OspfState)
WITH domain, count(DISTINCT proto.area_id) as area_count
WHERE area_count > 1
RETURN domain.id, area_count
```

P003 and P004 use WITH, count(), and DISTINCT, all of which are fully implemented in the NTE-QL executor. The count{} sub-query syntax is a planned extension; however, COUNT() as an aggregation function is available (Section 20.4). These policies can also be expressed using the Starlark engine or the Python fluent query API.

5.7.4 MPLS Path Diversity

Listing 7. NTE-QL policy: MPLS RSVP-TE path diversity

```
// P005: Primary and Backup LSPs must be SRLG-disjoint (Shared Risk Link Group)
MATCH (p:LSP {kind: 'primary'})-[link:wire]->(target:Router)
MATCH (b:LSP {kind: 'backup', headend: p.headend, tailend: p.tailend})
WHERE p.srlg_id = b.srlg_id
RETURN p.id, b.id, p.srlg_id
```

5.7.5 No Single Point of Failure

Listing 8. No single point of failure policy

```
# All routers in the critical path must have >= 2 uplinks
def check_no_spof(critical_nodes):
    for node_id in critical_nodes:
        uplinks = check_uplinks(node_id, 2)
        if not uplinks:
```



```

    fail("Critical node {} is a single point of failure".format(node_id))
    return True

# critical_path_nodes() is a host function that returns the set of
# articulation points in the physical topology
check_no_spof(critical_path_nodes())

```

OPERATIONS & USAGE (ADVANCED CONCEPTS)

6 Building the Digital Twin: Ingestion & Identity

6.1 Bimodal Execution: Strict vs. Relaxed

While NTE can act as a deterministic compilation engine for configuration synthesis (via `ank_netcfg`), restricting the system exclusively to rigid, top-down generation would significantly limit its utility. Modern network operations increasingly demand ad-hoc, real-time analysis of active network states (e.g., ingesting streaming BGP telemetry, or analysing live routing tables).

To support both design-time generation and run-time analysis, NTE implements a **Bimodal Execution Architecture** (Figure 22):

Strict Mode (The Design Authority) In this mode, NTE enforces the full blueprint aggregation model. All structural mutations are subjected to pre-commit schema validation and invariant checks. If a user attempts to draw a logical edge between two nodes that lack underlying physical Connectivity connectivity, the transaction is forcefully aborted. This mode is used when NTE acts as the source-of-truth compiler generating configuration artefacts.

Relaxed Mode (The Telemetry Sink) In this mode, structural invariants and schema requirements are dynamically bypassed. Users can rapidly stream ad-hoc properties (e.g., live SNMP counters or streaming telemetry) directly into the Polars DataFrames, or draw arbitrary graph edges representing discovered control-plane states, without satisfying the strict underlying Endpoints Model requirements. This transforms NTE into a high-performance graph-analytical sink for live operational data.

This bimodal capability means that the same query engine (NTE-QL) and vis tooling can be used to analyse both the “intended” network (Strict) and the “actual” network (Relaxed) with closely related semantics.

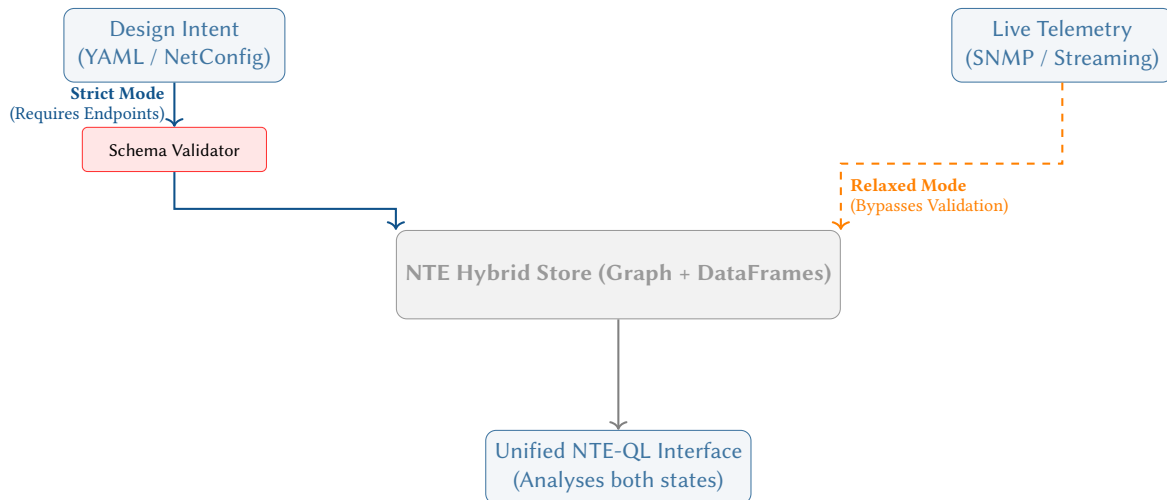


Figure 22. Bimodal Execution Architecture. Strict mode passes design intent through schema validation to enforce the Endpoints model. Relaxed mode bypasses parts of that validation so that live telemetry can enter the engine more directly. Both modes share the same NTE-QL interface.

6.2 Multi-Source Topology Aggregation

A core capability of NTE is **multi-source aggregation** (Figure 23). Topology intent from heterogeneous sources—design diagrams, IP address management spreadsheets, configuration files, or live telemetry—is aggregated into a single, unified structural representation that serves as the engine’s source of truth.

Design Rule 9: Structural Isomorphism

The internal graph acts as the single source of absolute structural truth. The mapping from various input sources into this formal model is a deterministic aggregation. The engine then manages this unified

model, fanning it out into the multi-layered topology overlays (L1, L2, L3) that make up the internal database.

This principle decouples *input format* from *structural representation*. Once the topology is materialised, the NTE engine maps it into its isolated layer planes. This same unified graph can equally drive configuration synthesis, network analysis, or live telemetry correlation.

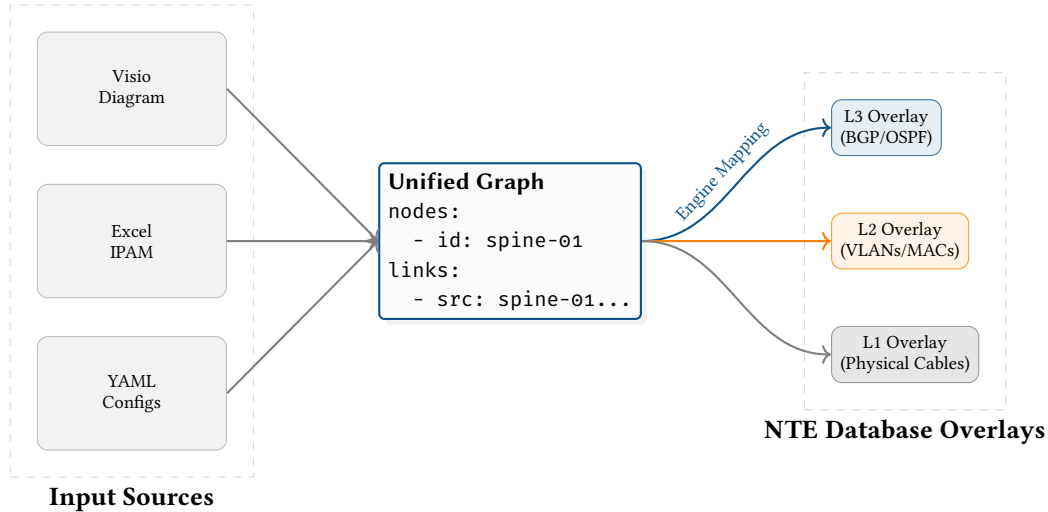


Figure 23. Multi-source aggregation: diverse input sources are normalised into a unified structural graph, which the engine evaluates and fans out into rigorous, multi-layered topology overlays.

6.3 Flexible Ingestion Patterns

To accommodate both top-down design and bottom-up discovery, NTE supports two primary data ingestion patterns. These patterns correspond to the **Bimodal Execution** modes described in Section 6.1.

Pattern A: The Explicit Endpoints Model Used primarily in **Strict Mode**, this pattern requires explicit modelling of nodes, endpoints, and structural edges. It is used when the topology must serve as a high-fidelity digital twin for physical failure simulation. The ingestion sequence follows the structural hierarchy: `add_nodes` → `add_endpoints` → `add_intra_edges` → `add_inter_edges`.

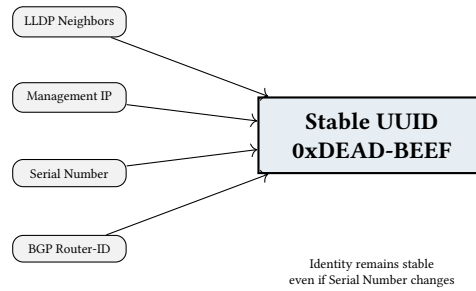
Pattern B: Ad-Hoc Direct Links Used in **Relaxed Mode**, this pattern allows drawing direct semantic edges between device nodes via the `add_direct_links` method. This bypasses the need for explicit endpoint modelling, enabling the rapid ingestion of discovered control-plane state (e.g., from an ARP table or a BGP peer list). This pattern is also suitable for modelling simple internal sub-components (like a Line Card connected directly to a Chassis) where the full complexity of physical interface pinning is not required.

While this pattern has lower structural fidelity, it remains fully compatible with the NTE-QL query engine. The Query Planner automatically selects the most efficient traversal path: if a direct edge exists, it is matched; otherwise, the planner attempts to expand the query into a 3-hop endpoint traversal.

6.4 Semantic Identity Across Snapshots

NTE separates **Semantic Identity** from volatile attributes (IP, MAC). A basic semantic hashing mechanism exists in `diff_against()`, which computes a hash covering node type, layer, and peer types to detect identity continuity across topology snapshots.

The full multi-source identity resolution described below (combining LLDP, management IP, serial number, and BGP Router-ID into a stable UUID) is planned (Section 26). Currently, semantic identity uses a simpler hash based on type, layer, and peer-type signatures.

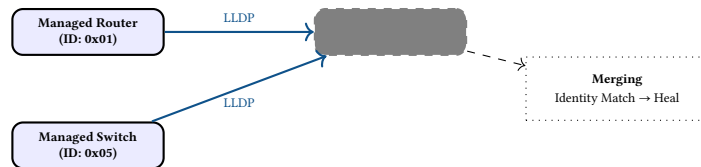


Illustrative Scenario: Zero-Downtime Inventory Migration A data-centre hardware refresh replaces all spine switches. In the full identity-resolution design, semantic identity would allow historical data and branches to map cleanly onto replacement devices, preserving lineage across hardware generations.

6.5 Shadow Graph Inference

A persistent challenge in network digital twins is the “edge of the map.” Telemetry data (LLDP, CDP, BGP peering logs) often reveals links to devices that are not managed by the central orchestrator. NTE provides **ShadowTopology** — a copy-on-write overlay for what-if simulation (Section 5.5). The concept of automatic Shadow Graph Inference, where unknown devices are dynamically created from telemetry signals, extends this foundation.

Automatic inference of shadow nodes from telemetry is planned but not yet implemented (Section 26). The ShadowTopology struct exists for explicit what-if simulation; the automatic “spawn a shadow node when an edge points to an unknown ID” workflow described below is an illustrative design goal.

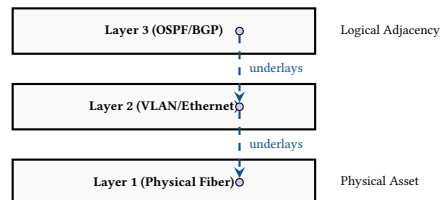


In the planned design, when an edge points to an unknown ID, the engine would spawn a “Shadow Node” tagged with a `confidence_score`. These nodes would visualise the boundary of managed territory. As more telemetry arrives, or when the device is officially provisioned, the identity-resolution layer would merge the shadow node into a fully managed node and preserve associated edges.

Illustrative Scenario: Mapping the Invisible An enterprise network relies on a third-party managed optical transport network (OTN) for inter-datacentre connectivity. While the enterprise has no management access to the OTN switches, its edge routers may still report LLDP neighbours on the optical hand-offs. In the planned shadow-node workflow, those neighbours would appear as inferred nodes, allowing the digital twin to include the previously unmodelled transport hop in path analysis.

6.6 Cross-Layer Lineage: Vertical Impact Analysis

A defining feature of a digital twin is mapping dependencies between layers. NTE uses **Vertical Lineage Links** to connect endpoints across the Physical, Data-Link, and Network layers.



Illustrative Scenario: Root-Cause of a “Silent Drop” An ISP experiences intermittent packet loss. Vertical lineage reveals an MTU mismatch on a physical fiber member of an LACP bundle. Querying `MATCH (b:BGP)-[:UNDERLAYS*]->(p:Physical) WHERE p.mtu != b.expected_mtu` immediately identifies the port causing silent drops for jumbo frames.

7 Structural Transformations

While standard graph databases focus on CRUD (Create, Read, Update, Delete) operations, NTE provides high-level **Structural Transformations** that allow complex topological manipulation with single API calls. These are atomic mutators implemented purely in Rust (in `n-te-topology`) and exposed securely via the Python SDK.

Structural transforms matter because network evolution rarely involves one node or one edge at a time. Migrating a Layer-2 bridge to a routed fabric, inserting a new aggregation tier, or collapsing redundant chassis into a single logical device—all require coordinated changes to nodes, endpoints, edges, and properties. Expressing these as sequences of low-level mutations is error-prone and difficult to roll back. Each transformation below executes inside a `DualWriteGuard` (Section 12.2), ensuring atomicity across both storage planes.

7.1 Split

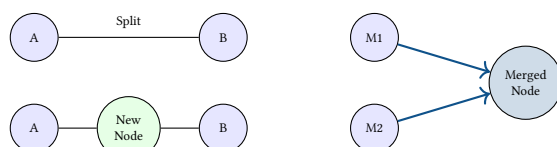
The **Split** operation (`split_edge`) inserts a new node into the middle of an existing edge, automatically handling the creation of new endpoints and rewiring of connectivity edges. The original edge is removed and two new edges are created: one from the source endpoint to a newly created endpoint on the inserted node, and one from a second new endpoint on the inserted node to the original destination endpoint.

All properties on the original edge are preserved on the source-side segment by default. The inserted node inherits the layer of the original endpoints, ensuring layer consistency is maintained without manual intervention.

7.2 Merge

The **Merge** operation (`merge_nodes`) collapses two nodes into one, inheriting all logical endpoints and physical connections from both parents. The surviving node absorbs the properties of the consumed node (with the survivor's values taking precedence on conflict). All edges incident on the consumed node are rewritten to point to the survivor, and the consumed node and its ownership edges are removed.

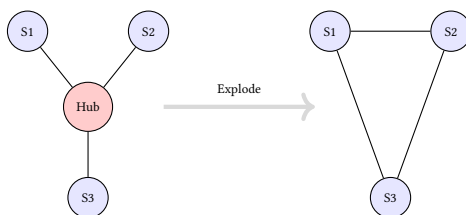
This operation is particularly useful when reconciling duplicate representations of the same device (e.g., a router that appears in both an LLDP discovery feed and a design diagram).



7.3 Explode (The Hub-and-Spoke Dissolver)

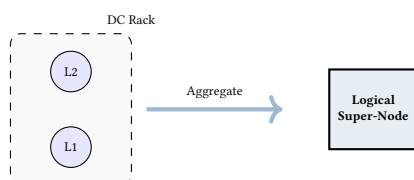
The **Explode** operation (`explode_node`) removes a central hub node and replaces it with a full mesh of direct connections between its former neighbours. For a hub with k neighbours, this produces $\binom{k}{2}$ new edges. The operation is invaluable for simulating the removal of a transparent bridge or a Layer-2 switch while maintaining logical connectivity between the devices that were formerly connected through it.

Each new edge inherits the layer of the original connectivity, and the hub node's endpoints and ownership edges are cleaned up atomically.



7.4 Aggregate (Graph Summarisation)

Aggregation algorithmically collapses a cluster of nodes (e.g., an OSPF Area or a datacentre rack) into a single logical Super-Node, preserving inter-cluster edges as edges between super-nodes.



Roadmap

The aggregate operation is tracked as a high-priority roadmap item. It is not yet implemented in the core engine. The planned design uses hierarchical clustering to generate multi-scale views of large topologies. See Section 26 for the current roadmap.

8 Operational Case Studies

This section demonstrates how NTE’s hybrid engine, structural transformations, and policy engine combine to solve complex operational challenges in network engineering.

8.1 Case Study 1: Transparent to Routed Migration

A network architect plans to migrate a “stretched” Layer-2 segment into a modern Layer-3 routed fabric. Using the **Explode** operation on the legacy Layer-2 bridge, the architect instantly visualises the resulting full mesh of routed peering required to maintain existing connectivity. Combined with What-If Branching, this allows the architect to generate a precise BGP Peering Plan by diffing the “Before” (Hub-and-Spoke) and “After” (Exploded Mesh) topologies via `diff_against()`.

8.2 Case Study 2: Multi-layer Policy Validation

In this scenario, an operator attempts to provision a new BGP peering session between two border routers. The Starlark policy engine is configured to enforce an architectural design rule: all external BGP sessions must traverse an encrypted IPsec tunnel.

When the operator submits the mutation to add the BGP Link to the topology, the reactive event system triggers the Starlark validation suite. The policy engine queries the topology using NTE-QL to verify the underlying transport layers. It discovers that the path between the two routers lacks an IPsec layer. The policy engine instantly rejects the mutation, preventing an unencrypted session from being deployed, and returns a detailed failure report to the operator outlining the missing transport requirement.

8.3 Case Study 3: Blast Radius Analysis

During a maintenance window, an engineer needs to upgrade the firmware on a core physical switch. Before proceeding, they must understand the full impact of taking this node offline.

Using NTE’s cross-layer relationship tracking, the operator executes a blast radius query. The engine traverses from the target physical switch up through the structural Parent edges to identify all impacted logical overlays. The query reveals that taking down the switch will sever a physical fibre link, which in turn brings down an OSPF adjacency, causing a BGP route reflector session to drop, and ultimately disrupting three tenant EVPN [9] overlays. By exposing this cascading failure path across multiple layers, NTE allows the engineer to proactively drain traffic from the switch before initiating the upgrade.

9 Blast Radius & Failure Domain Isolation

The primary justification for the strict verbosity of the Endpoints Model is deterministic failure propagation. When a physical or logical component fails, the engine must accurately calculate the “blast radius”—the exact subset of downstream services affected.

By modelling ownership explicitly via Structural edges, failure simulation becomes a pure mathematical reachability problem (Figure 24).

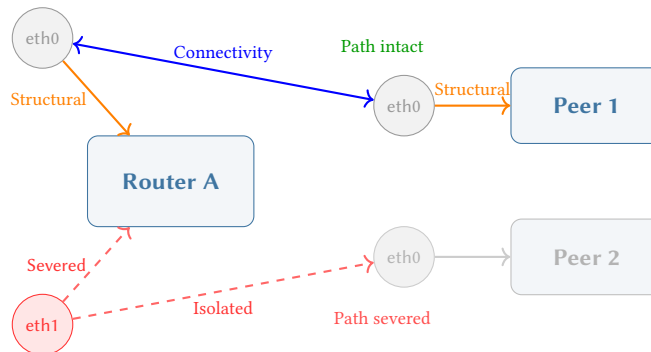


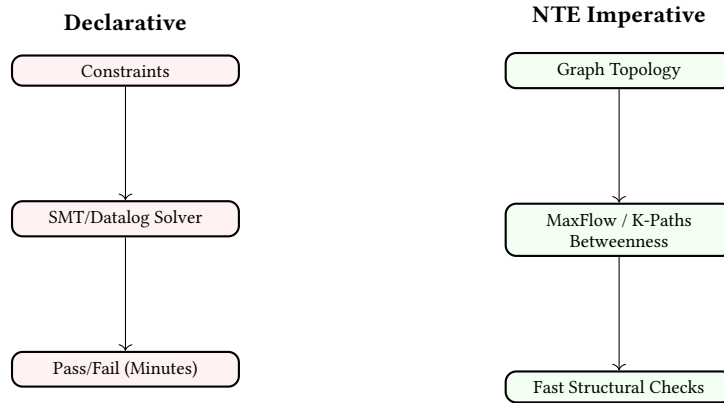
Figure 24. Blast Radius Propagation. When Line Card 2 fails, its outgoing Structural edge is severed. The connection to Peer 2 is isolated whilst Peer 1 remains unaffected because eth0 and eth1 reside on different endpoints with intact Structural edges. A naive (Router)-[CONNECTED_TO]->(Router) model cannot express this partial failure.

10 Advanced Analytics

This section describes the graph analytics capabilities that NTE provides for network verification, capacity planning, and structural anomaly detection.

10.1 Differential Reachability: Imperative vs. Declarative Approaches

A core goal of network verification is to check whether reachability policies hold before and after a change. While tools such as Batfish translate configurations into logical constraints, NTE emphasises an imperative algorithmic approach over the current topology model.



NTE exposes several graph algorithms written in Rust:

K-Shortest Paths (Yen’s Algorithm) Calculates the top K paths between two nodes. Rather than a binary “can A reach B?” answer, this produces a ranked set of alternative routes. A behavioural diff between two topology snapshots reveals shifts such as “Path A→B shifted from 2 hops to 5 hops; redundancy dropped from 3 paths to 1.” This is directly useful for maintenance-window impact assessment and traffic-engineering verification.

Max Flow / Min Cut (Dinic’s Algorithm) Efficiently calculates the theoretical maximum edge-disjoint capacity between two points. The min-cut result identifies the bottleneck link set whose removal would partition the source from the destination—essential for detecting single points of failure in transit fabrics.

Betweenness Centrality Identifies topological bottlenecks by measuring how often each node or edge lies on shortest paths between all other pairs. Nodes with high betweenness are candidates for capacity upgrades or redundancy improvements.

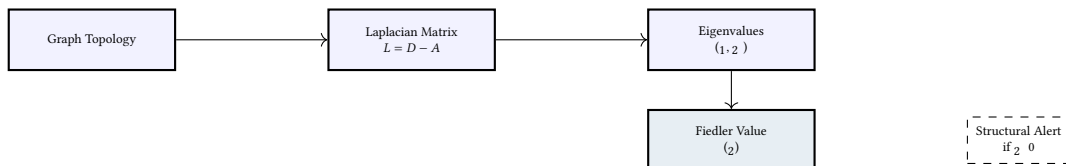
By exposing these algorithms to the Starlark policy engine, NTE can support faster structural checks for selected workloads than heavier solver-based approaches. The exact trade-off depends on the property being verified and on the topology size.

Illustrative Scenario: Real-Time Compliance. A healthcare network’s compliance policy requires that a medical-imaging VLAN retain at least two physically diverse paths across a campus backbone. During a planned fibre maintenance event, an engineer simulates a link shutdown using What-If Branching (Section 12.1). The imperative engine calculates max-flow between the imaging centres and warns the engineer that path diversity has dropped to zero. This real-time feedback loop allows the engineer to reroute traffic *before* the fibre is cut, maintaining HIPAA compliance throughout the maintenance window.

10.2 Spectral Graph Signatures

While Graph Neural Networks (GNNs) are powerful, they require training pipelines, parameter tuning, and suffer from explainability issues. NTE provides a complementary approach: deterministic, mathematically grounded anomaly detection via **Spectral Graph Theory**.

By calculating the **Graph Laplacian** matrix ($L = D - A$, where D is the degree matrix and A is the adjacency matrix), the engine extracts structural signatures from the topology’s eigenvalue spectrum.



The second smallest eigenvalue of the Laplacian (λ_2), known as the **Fiedler Value** (or Algebraic Connectivity), summarises how well-connected the entire network is in a single scalar. A value of zero indicates a disconnected graph; values near zero indicate a topology that is one or two failures away from partitioning.

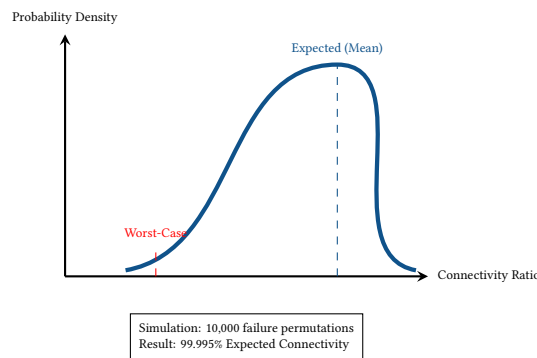
Using Rust’s high-performance linear algebra crates (e.g., `nalgebra`), NTE calculates the Fiedler value of the `AnalysisGraph` on demand. If a configuration diff shows that five nodes changed and the Fiedler value drops by 40%, the engine flags a massive structural fragmentation risk—even if no links are completely severed. The corresponding eigenvector (the Fiedler Vector) can mathematically bisect the network, identifying the specific “choke point” links that threaten to split it in two. This provides absolute, deterministic anomaly detection without relying on trained models.

Illustrative Scenario: Early Warning of Network Fragmentation. A SCADA network for a national power grid is designed for extreme resilience. During a series of cascading hardware failures in a regional substation, the network remains operational due to redundancy. However, NTE detects that the Fiedler value has dropped from 0.45 to 0.02. This indicates that the network is “algebraically brittle”—one more specific link failure will cause a complete partition. NTE alerts the grid operators to a critical loss of structural integrity hours before the final redundant link would have failed, allowing for emergency manual overrides.

10.3 Probabilistic Risk Modelling: Monte Carlo Analysis

Deterministic What-If analysis is powerful but cannot account for the vast state space of multi-link failure permutations. NTE integrates a high-performance **Monte Carlo Engine** (in `n-te-monte-carlo`) to quantify topological risk statistically.

By executing thousands of simulation iterations, the engine generates a probability distribution of network connectivity under random failure scenarios. This moves the discussion from “Is it safe?” to “How likely is it to fail, and what is the expected impact?”



Each iteration randomly removes edges or nodes according to a configurable failure probability model, then measures connectivity, path diversity, and max-flow between critical pairs. The aggregate distribution reveals hidden “blast radius” regions—clusters of infrastructure whose simultaneous failure would cause disproportionate impact.

Illustrative Scenario: Calculating Blast Radius in SD-WAN. A global retail corporation operates 5,000 SD-WAN sites. Before deploying a new routing policy, the operations team runs a Monte Carlo simulation in NTE. The engine simulates 50,000 random failure events (fibre cuts, hardware crashes) across the backbone. The resulting distribution reveals a hidden blast radius where a specific core region failure would isolate 15% of all stores. This insight allows the team to provision an additional redundant link in the identified region *before* any outage occurs.

ENGINE IMPLEMENTATION (DEEP DIVE)

11 Implementation Architecture

A dual-write engine that coordinates graph and columnar stores requires careful dependency management to prevent the two execution paths from becoming entangled. NTE enforces a strict layered crate architecture: pure domain types sit at the bottom of the stack, event-sourcing primitives are pushed down to the foundation layer, and the query engine and topology manager consume them independently through well-defined interfaces. This layering is the structural prerequisite for the reactive pipeline described in later sections.

11.1 Workspace Layout

NTE is a 17-crate Cargo workspace (16 workspace members plus the root `PyO3` crate). The core crates and their roles are detailed in Appendix A (Table 6).

Figure 25 shows how the crates relate.

11.2 Crate Dependency Hierarchy

To maintain high architectural integrity and support reactive IVM, NTE utilises a strict layered dependency model. Following the Phase 20 refactor, all shared mutational types (Events and Sequences) were migrated to `n-te-graph` to eliminate cyclic dependencies between the query engine and the business logic layer.

1. **Layer 0 (Domain):** `n-te-domain` contains pure data structures with zero external dependencies.
2. **Layer 1 (Foundation):** `n-te-graph` provides the fundamental connectivity fabric, memory layouts (CSR), and the event sourcing definitions.

3. **Layer 2 (Execution):** nte-query implements the relational and graph execution engines (WCOJ, IVM) on top of foundation types.
4. **Layer 3 (Business Logic):** nte-topology provides high-level topology management, transactions, and formal verification.
5. **Layer 4 (Bindings):** The src/ root crate provides the PyO3 interface to Python.

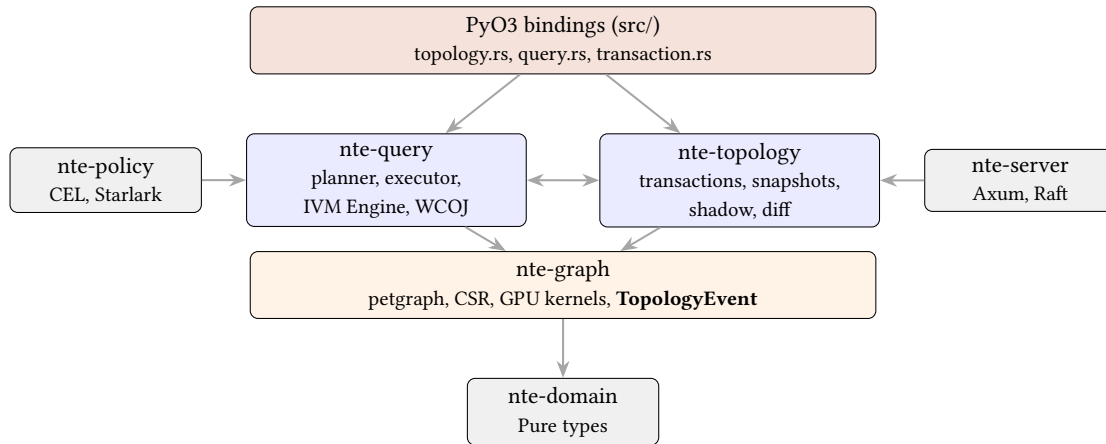


Figure 25. The Layered Crate Architecture. Shared foundation types (Events) are pushed down to nte-graph to enable clean, uni-directional dependency flow between the query engine and topology manager.

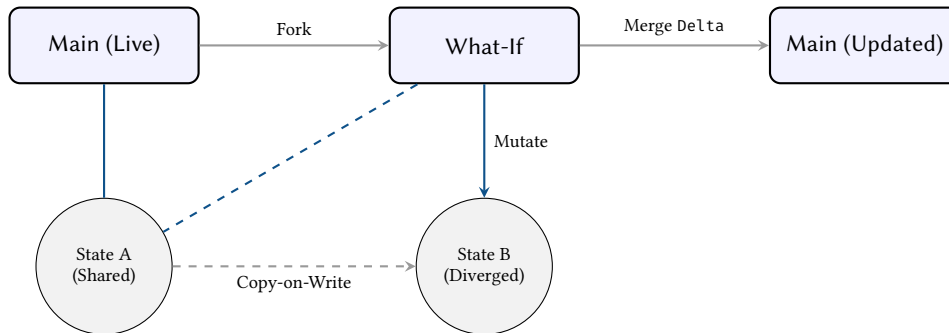
The policy engine (Section 5) operates at Layer 3, leveraging the query engine at Layer 2 to evaluate design-rule contracts against the topology.

12 Transactions

12.1 "What-If" Branching

Many network management systems operate on a single mutable current state. Simulating a failure or testing a new configuration therefore often requires a separate staging environment or an offline simulation.

NTE's hybrid storage layer uses petgraph for structure and Polars DataFrames for properties. This design allows partial copy-on-write (CoW) behaviour during branching and rollback.



When a user requests a “What-If” branch, for example to simulate a maintenance window:

1. **Memory Efficiency:** The underlying Arrow memory buffers backing the Polars DataFrames are not duplicated; they are reference-counted.
2. **Isolated Mutations:** As the user drops nodes (simulating a failure) or adds edges (simulating a new link) in the branch, only the modified data structures are copied and diverged. The SnapshotManager handles these isolated instances.
3. **Verification:** The Starlark policy engine (`validate()`) can be run against this isolated branch. If the simulation results in a Single Point of Failure (SPOF), the maintenance plan can be programmatically rejected.
4. **Patch Generation:** If the branch is successful, the `diff_against` engine computes the `TopologyDelta` between the branch and the live state, generating a precise “Change Request” payload.

Illustrative Scenario: Peak Traffic Resilience A Tier-1 ISP needs to upgrade the firmware on a core backbone router during a 4-hour maintenance window. Using NTE, they fork the live topology into a “What-If” branch and programmatically `drop_node()` the router. The Starlark policy engine rapidly detects that the resulting topology violates a `min_path_redundancy(3)` constraint for high-priority financial traffic. The

engineer adjusts the maintenance window to a time when secondary links can absorb the load, avoiding a potential multi-million dollar SLA breach.

12.2 Dual-Write Atomicity: The DualWriteGuard

The hybrid storage architecture creates a concrete consistency problem: every mutation must succeed in *both* the petgraph graph and the Polars DataFrame store, or in neither. A node that exists in the graph but is missing from the property store (or vice versa) would silently corrupt query results.

NTE solves this with the DualWriteGuard, an RAII (Resource Acquisition Is Initialisation) guard implemented in `nte-topology::topology::rollback`. The guard wraps every mutation sequence in a scope that automatically rolls back on failure:

1. **Begin:** The caller creates a DualWriteGuard holding mutable references to both stores.
2. **Record:** Each mutation (e.g., `add_nodes_with_metadata`, `remove_nodes`, `add_connectivity_edges`) writes to both stores and pushes a corresponding `UndoOp` onto a reverse-order undo stack.
3. **Commit:** The guard calls `check_dual_write_agreement()` to verify that graph node count and DataFrame row count are consistent, and that layer registrations match. If the check passes, the guard marks itself as committed.
4. **Drop:** If the guard is dropped without committing (due to an error or early return), the Drop implementation replays the undo stack in reverse order, restoring both stores to their pre-mutation state.

The `UndoOp` enum captures the inverse of each mutation type:

RemoveNodesFromGraph/DataFrame Reverses a node addition.

RestoreNodesToGraphWithMetadata Reverses a node removal by re-inserting captured metadata.

RestoreEdgesToGraph Reverses edge removal by re-inserting captured edge tuples with their original `GraphEdgeKind`.

The guard is annotated with `#[must_use]`, so the Rust compiler warns if a caller creates a guard but never commits it.

Debug Validation

In debug builds, setting the environment variable `NTE_VALIDATE_METADATA_SYNC_ON_COMMIT=1` enables an additional $O(n)$ metadata synchronisation check at commit time. This validates that every node's type and layer in the graph metadata matches the corresponding DataFrame row. This flag is invaluable during development but is disabled by default in release builds.

12.3 The Dual-Write Transaction Model

NTE's core architectural feature is its **hybrid storage architecture** (Section 16), which pairs a petgraph adjacency structure with Polars columnar DataFrames. This design improves mixed graph-relational execution, but it also creates a consistency problem: the graph structure and the tabular properties must never fall out of sync.

NTE addresses this through the **Dual-Write Transaction Model** (Figure 26). Each mutation runs inside a transaction scope that coordinates updates across both storage planes.

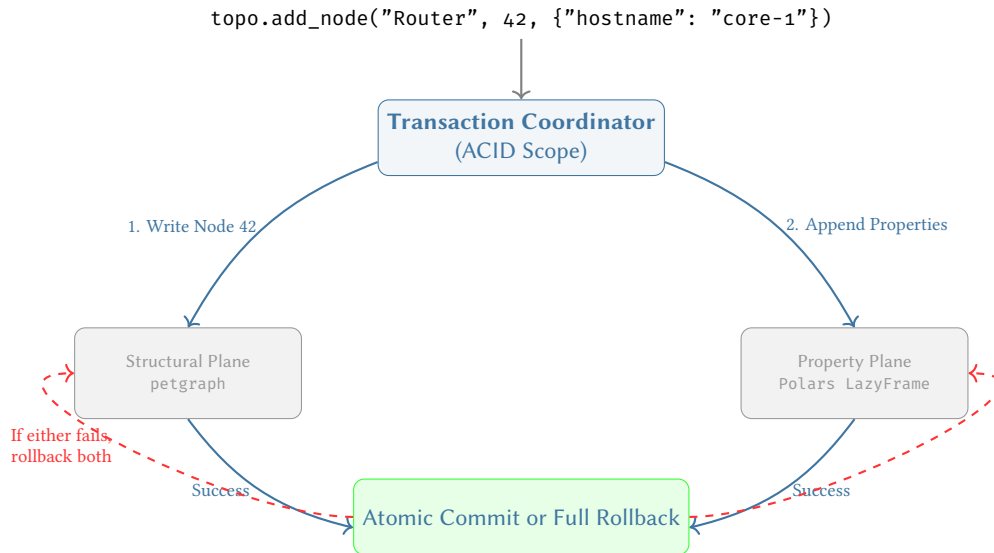


Figure 26. The Dual-Write Transaction Model. Mutations are coordinated across the graph and property stores. If one write fails, the transaction aborts and rolls back both sides.

12.4 ACI (Durability via WAL) Transaction Model

Transaction Durability

NTE provides **ACI** guarantees (Atomicity via DualWriteGuard, Consistency via post-commit invariant checks, Isolation via version-tracked conflict detection). **Durability** is available through an optional **Write-Ahead Log (WAL)**: every committed mutation is appended to a monotonically increasing journal on disk, ensuring that in-memory state can be recovered following a process crash. Structural mutations utilise copy-on-write (CoW) semantics for snapshot isolation (see Section 13.2). Note that multi-transaction MVCC is currently version-tracked at the node and edge level; full snapshot-isolated concurrent transactions are an area of ongoing development.

The transaction model (Figure 27) uses Arrow's reference-counted CoW buffers for cheap snapshot creation at transaction start and cheap rollback when a transaction aborts. When WAL is enabled, the commit phase is a two-step process:

1. **Prepare:** The mutation is validated and prepared in-memory.
2. **Commit:** The event is flushed to the WAL on disk, then applied to the primary in-memory stores.

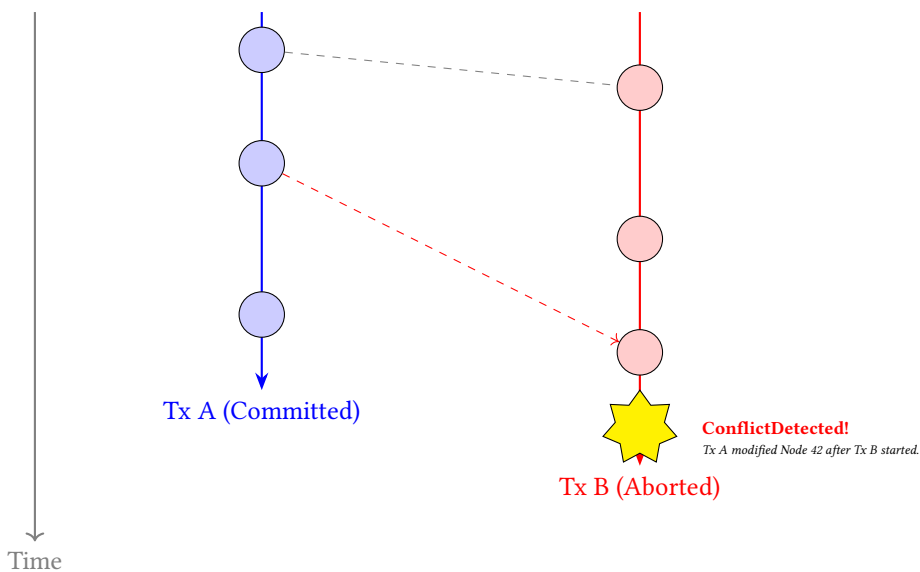


Figure 27. MVCC Transaction Timeline. Both transactions read from the same initial snapshot S_0 . Because Tx A commits a mutation to Node 42 before Tx B attempts to mutate it, the Conflict Engine safely aborts Tx B to prevent a Write-Write conflict.

12.5 Isolation Levels

NTE supports three isolation levels, following standard database semantics:

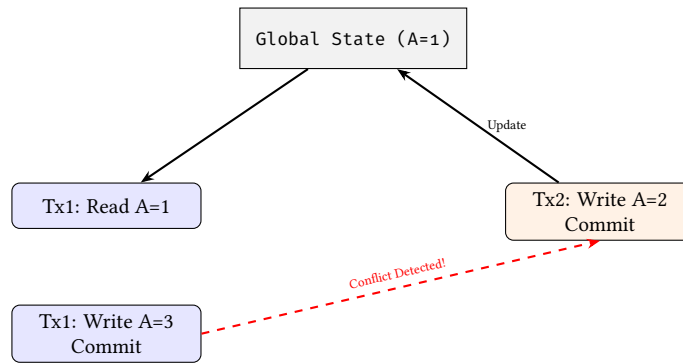


Figure 28. Serializable isolation conflict detection. Tx1 fails to commit because its read-set ($A=1$) was invalidated by Tx2's concurrent write.

Level	Behaviour
ReadCommitted	Sees only committed data; each read within the transaction sees the latest committed state.
RepeatableRead	Reads within the transaction see a consistent snapshot taken at transaction start.
Serializable	Full conflict detection: transactions that read/write overlapping data are detected and the later transaction is aborted.

12.6 Transaction Snapshots

NTE's copy-on-write architecture allows the engine to expose consistent, read-only snapshots of the topology. Each transaction creates a logical snapshot at its start time and shares structure where possible.

Reference-counted DataFrames allow a snapshot to share much of its state with the live topology. Only modified entries need to diverge, so transactions that touch a small part of the graph incur modest additional memory cost.

12.7 Python Transaction API

The PyTransaction class exposes transactions through a Python context manager:

Listing 9. Transaction API

```

from ank_nte import Topology, IsolationLevel

t = Topology()
# ... populate topology ...

# Context manager: auto-rollback on exception
with t.begin_transaction(IsolationLevel.Serializable) as txn:
    t.add_nodes(ids=[100], node_types=["Router"], layer="physical",
                data=[{"hostname": "r-new"}])
    t.add_inter_edges(sources=[10], destinations=[100])
    txn.commit() # explicit commit

# If an exception occurs before commit(), the transaction
# is automatically rolled back (both graph and DataFrame store).

# Manual rollback
with t.begin_transaction() as txn:
    t.remove_node_cascade(node_id=1)
    if not t.is_reachable(source=2, target=3):
        txn.rollback() # revert: removal would partition the graph
    else:
        txn.commit()
  
```

Durable Commit

NTE transactions can be made crash-consistent by enabling the optional Write-Ahead Log (WAL). When active, every committed transaction is flushed to a persistent on-disk journal before the in-memory state is updated. This allows the engine to recover committed state after a process crash.

12.8 Conflict Detection

When multiple transactions execute concurrently, NTE detects conflicts using a `VersionTracker` that records per-node and per-edge version numbers. At commit time, the tracker compares each transaction's read and write sets against committed versions:

Write–write conflicts (all isolation levels): if a node or edge was modified by another transaction after the current transaction began, a `ConflictError` is raised.

Read–write conflicts (Serializable only): if a node that was *read* by the current transaction was subsequently *written* by another committed transaction, a `ConflictError` is raised. Read-your-own-writes are excluded from this check.

Conflict errors carry the transaction ID, the list of conflicting node/edge IDs, and the conflict type (`WriteWrite` or `ReadWrite`), allowing callers to implement application-level retry or merge logic.

13 Persistence and Snapshots

13.1 Write-Ahead Log (WAL) and Crash Recovery

The Write-Ahead Log (WAL) provides ACID durability for the dual-write engine. Every mutation (node creation, edge removal, property update) is serialized as a `TopologyEvent` and appended to a persistent on-disk journal before being applied to the in-memory graph and relational stores.

Design Rule 10: Hardened Durability

The WAL must be resistant to partial writes and torn pages. NTE implements CRC32 checksums for every event entry. During recovery, any corrupted or incomplete JSON lines are detected and handled according to the configured safety policy (halt vs. truncate).

13.1.1 Durability Mechanics

When a mutation is initiated, the engine follows a strict write sequence:

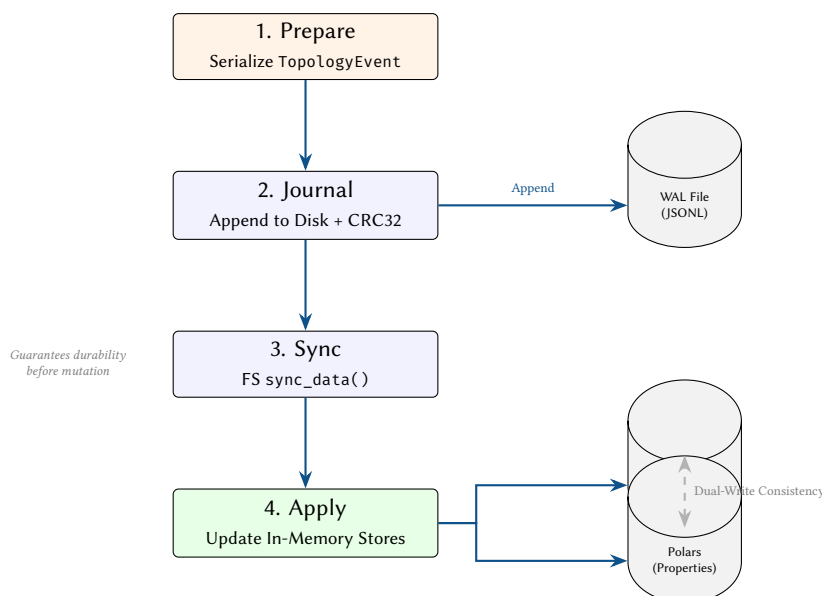


Figure 29. The Dual-Write + WAL Mutation Lifecycle. To ensure ACID durability, mutations are journaled to the WAL and flushed to disk *before* the in-memory graph and property stores are updated. This guarantees that a crash at any point after Step 3 can be fully recovered from the log.

- 1. Prepare:** The `TopologyEvent` is serialized to JSONL. ...
- 2. Journal:** The event and its CRC32 checksum are appended to the WAL file.
- 3. Sync:** A `sync_data()` call forces the OS to flush the write buffer to physical storage.
- 4. Apply:** The mutation is applied to the petgraph and Polars stores.

13.1.2 Rotation and Compaction

To prevent the WAL from growing indefinitely and to keep recovery times bounded, NTE implements automatic log rotation and compaction. When the WAL reaches a size threshold (default 100 MB), the engine:

1. Freezes the current log file.
2. Takes a full binary snapshot of the topology state (as a .nte archive).
3. Opens a fresh WAL file for new mutations.

Recovery then becomes a two-step process: load the latest stable snapshot, then replay only the events in the current active WAL.

13.2 Historical Reconstruction (Event-Sourced Temporal Queries)

The WAL also provides the basis for replay-based historical reconstruction. Because every mutation is a discrete `TopologyEvent`, the state at any point in history can be expressed as a fold: $\text{State}(t) = \text{InitialState} + \sum \text{Events}[0..t]$.



Instead of saving large numbers of discrete snapshots, NTE can reconstruct point-in-time topologies from event history. Engineers can then run standard queries against an earlier reconstructed state during failure analysis.

Illustrative Scenario: The 2:00 AM Ghost A financial-services provider experiences intermittent loss of connectivity at 2:15 AM on a Sunday. By 8:00 AM, self-healing protocols have masked the problem. By reconstructing the earlier state from WAL history, the team can inspect the topology as it existed during the incident and trace the failure to a brief maintenance window and a transient BGP misconfiguration.

Design Rule 11: Zero-Copy Isolation

Snapshots should leverage Arrow's reference-counted buffers so that historical state adds little initial memory overhead on the property side. Cloning the property schema is an $O(1)$ pointer increment. This supports transactional rollback and what-if shadow topologies.

13.3 Named Snapshots

NTE supports named in-memory snapshots managed by `SnapshotManager`. The key implementation detail is that Polars DataFrames use Apache Arrow's reference-counted buffers. Cloning a DataFrame shares the underlying Arrow buffers; a copy is only made when a buffer needs to be mutated (CoW semantics).

This means:

- Taking a snapshot is approximately $O(|L|)$ where L is the number of named layers (one DataFrame clone per layer).
- Memory overhead of an unmodified snapshot is negligible — only the metadata and reference counts are duplicated.
- Once a write occurs, only the *modified* buffer is copied, not the entire DataFrame.

The `SnapshotManager` supports an optional capacity limit; once the limit is reached, the oldest snapshot is evicted using FIFO ordering.

13.4 File Serialisation

NTE provides two file serialisation formats:

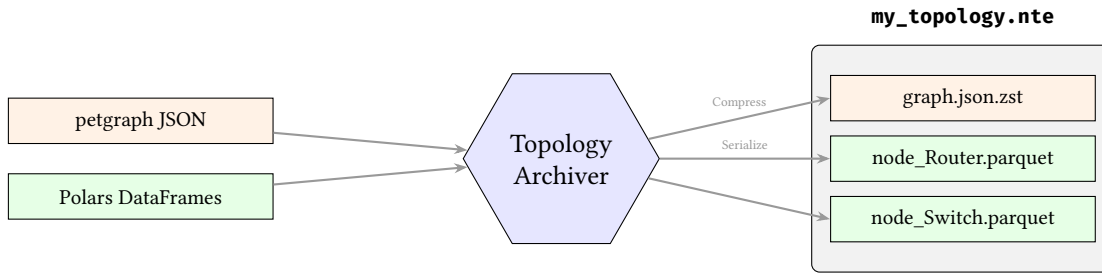


Figure 30. The .nte archive format. Structural graph data is compressed via zstd, while DataFrames are serialised directly to columnar Parquet files, all packaged into a standard ZIP archive.

.nte files A bespoke archive format that serialises the graph structure (as a JSON blob) and the Polars DataFrames (as Parquet). Files are loaded with `Topology.load(path)` and saved with `t.save(path)`. This is the recommended format for operational use.

In-memory bytes `t.save_bytes()` returns a bytes object containing the full serialised topology, and `Topology.load_bytes(data)` deserialises it. Useful for network transfer or embedding in larger artefacts.

13.5 Topology Diffing & Semantic Hashing

Comparing two large topologies to identify structural or property-level deviations is an important operation for network digital twins. NTE implements a diffing engine in `nte-topology` that identifies:

- **Structural Changes:** Added/removed nodes and edges via stable external IDs.
- **Property Changes:** Modifications to specific attributes on existing nodes or edges.
- **Identity Shifts:** Heuristic-based detection of renamed or replaced entities (Semantic Hashing).

Design Rule 12: Vectorized Property Diffing

To maintain efficient execution on large topologies, property comparison must avoid row-by-row iteration. NTE leverages Polars' native `Outer Join` operator to align two DataFrames by their primary key (external ID) and identifies differences using SIMD-accelerated bitwise comparisons of the aligned columns.

The diffing engine produces a `TopologyDelta` structure, which can be exported to Python as a structured dictionary for downstream reconciliation logic.

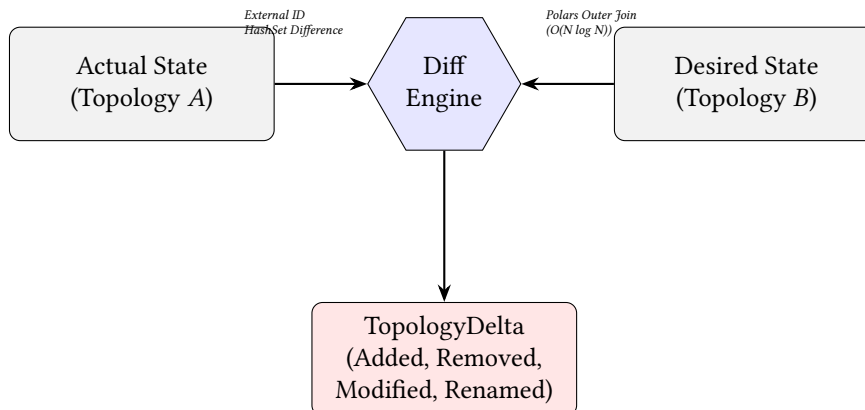


Figure 31. Topology Diffing Pipeline. The engine combines structural set operations with vectorized relational joins to identify deviations between Actual and Desired network states.

13.6 CSR Serialisation for Archival

For batch traversal and long-term archival, NTE can serialise the graph as a Compressed Sparse Row (CSR) structure using zero-copy serialisation (using frameworks like `rkyv` [10] inspired by `FlatBuffers` [11]). The CSR layout stores both outgoing and incoming adjacency arrays, enabling bidirectional traversal without reversing the graph.

The structure is serialised to a flat byte buffer with no padding or relocation, allowing it to be memory-mapped directly without deserialisation cost.

13.7 Memory-Mapped Loading (Out-of-Core Execution)

Warning

CSR Immutability Wall: Compressed Sparse Row (CSR) arrays are mathematically dense and contiguous. Consequently, they are entirely **immutable**. Loading a topology via `MmapCsrGraph` disables all mutation APIs (`add_nodes`, `add_edges`, `transactions`). Attempting to mutate an mmap'ed CSR topology will panic.

The `MmapCsrGraph` type wraps a memory-mapped file and provides a zero-copy view onto an `ArchivedCsrGraph` stored on disk. This allows the engine to access topology structure directly from the filesystem, with the operating system's page cache handling demand-paging of required segments.

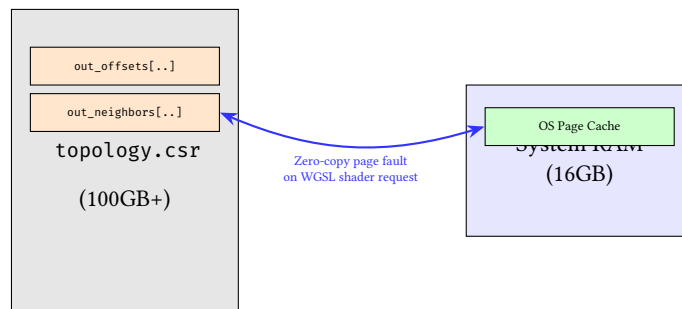


Figure 32. Mmap CSR Execution. For graphs larger than RAM, the OS pages in only the required adjacency chunks, which can support out-of-core analysis for suitable workloads.

Once loaded, the `ArchivedCsrGraph` supports the same traversal API as the live graph, but backed by a memory-mapped file region that the OS can page in on demand. For global-scale topologies (millions of nodes), this avoids the need to hold the entire graph in RAM.

Design Rule 13: Zero-Copy Persistence

To eliminate the serialisation bottleneck in performance-critical pipelines, NTE uses the `rkyv` framework for CSR archival. This makes the on-disk byte representation identical to the in-memory layout, allowing mmap-based loading without a parsing or relocation step.

13.8 Use Cases for Historical Querying

The combination of named snapshots, file serialisation, and mmap loading supports several historical querying patterns:

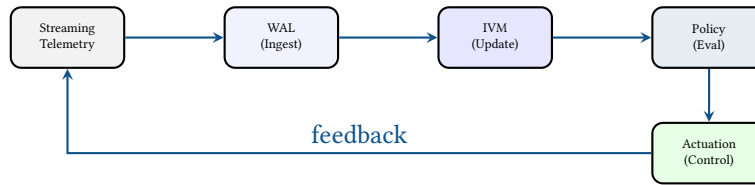
- **Before/after maintenance:** snapshot before a change window, restore and diff afterwards to verify intent.
- **Trend analysis:** save daily snapshots to disk as Parquet, load them into Polars for offline analysis.
- **Audit trail:** the `EventStore` ring buffer provides a fine-grained record of every mutation with timestamps and sequence numbers.
- **Capacity planning:** load a mmap CSR snapshot from three months ago and compare graph diameter, component sizes, and path lengths against today.

The following sections cover advanced deployment and operational topics — GPU acceleration, graph algorithm plugins, reactive events, production diagnostics, and distributed clustering. These can be read independently and in any order.

14 Reactive Events

14.1 Reactive Policy Loops: Streaming Graph Ingestion

NTE provides the basis for reactive control loops that ingest telemetry streams and emit topology events. Events enter the WAL, trigger notifications, and can be evaluated by the Starlark policy engine. Type-gated standing queries may also use the current experimental IVM fast path.



Illustrative Scenario: Sub-Second "Algebraic" Traffic Shedding An operator may monitor spectral signatures such as σ_2 . A sharp drop after a link flap could trigger a policy that sheds non-critical bulk services before a larger cascade develops.

Design Rule 14: Reactive Isomorphism

State mutations should propagate through strongly typed events so that downstream consumers can remain synchronised with the topology state. The event bus is the intended coordination mechanism for such consumers.

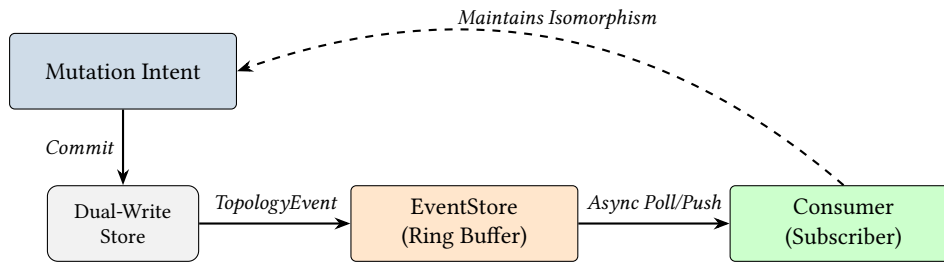


Figure 33. The Reactive Event Pipeline. Structural and relational changes are emitted as typed events, which bridge the physical storage layer back to the Intent Specification domain.

14.2 Event Lifecycle and Async Dispatch Architecture

Design Rule 15: Async Decoupling

To maintain high performance on the write plane, mutations must not block on subscriber I/O. NTE implements an asynchronous dispatch mechanism that offloads event publication to a background worker, freeing the core engine's write lock immediately after applying a mutation.

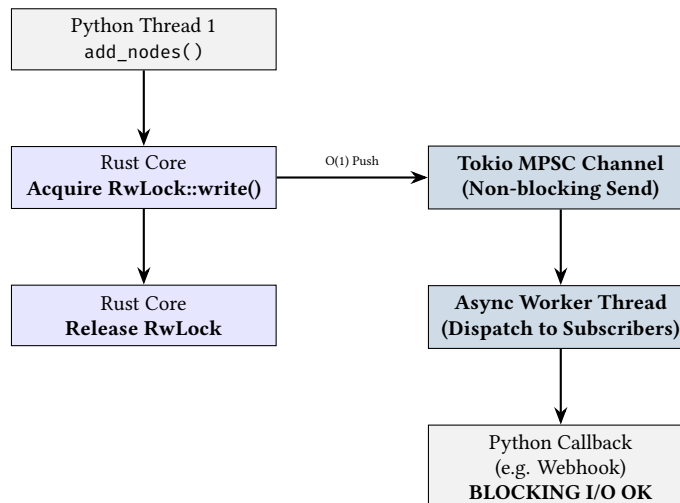


Figure 34. The Async Dispatch Pipeline. By offloading event publication to a background worker via an MPSC channel, the core engine can release its write lock immediately after applying a change, regardless of how slow individual subscribers might be.

14.2.1 Backpressure and Subscriber Health

When a subscriber (e.g., a Python webhook) is too slow to keep up with the mutation rate, the dispatch channel's capacity is reached. NTE enforces backpressure to protect the engine's memory: if the channel is full, the background worker will drop the oldest events or disconnect the slow subscriber. This prevents I/O-bound consumers from inducing head-of-line blocking on the primary database write-plane.

NTE emits a `TopologyEvent` for every topology mutation. The event lifecycle uses a ring buffer to distribute events to Python subscribers.

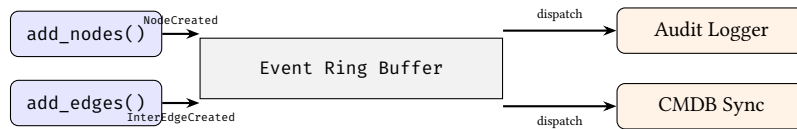


Figure 35. The Reactive Event Pipeline. Mutations in the core engine generate events which are buffered and dispatched to Python callbacks. In the current implementation, callback invocation remains synchronous on the write path.

14.3 Closed-Loop Automation Cycle

The event system is one mechanism for closing the loop between intent and observed state. By subscribing to topology mutations, external controllers can build a cycle of synthesis, deployment, and verification around the engine.

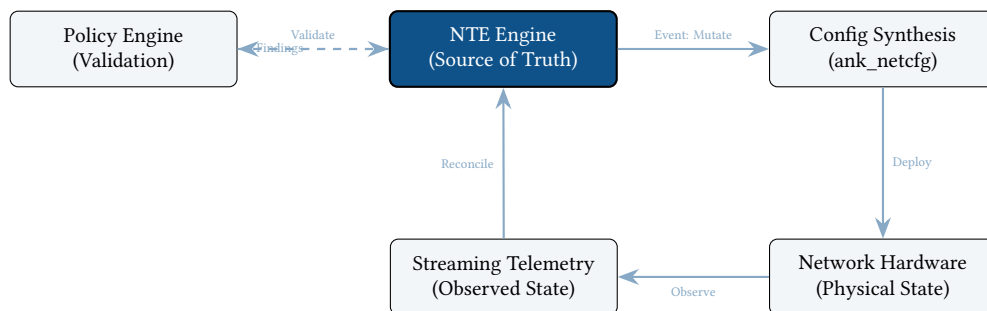


Figure 36. Illustrative closed-loop workflow. NTE can act as a central state model, with external systems triggering synthesis on mutation and reconciling incoming telemetry to detect drift.

14.4 Event Types

NTE emits a `TopologyEvent` for every topology mutation. The full event taxonomy:

Event	Trigger
<code>NodeCreated</code>	<code>add_nodes</code>
<code>EndpointCreated</code>	<code>add_endpoints</code>
<code>NodeRemoved</code>	<code>remove_nodes</code> , <code>remove_node_cascade</code>
<code>EndpointRemoved</code>	<code>endpoint removal</code>
<code>NodeUpdated</code>	<code>update_node_field</code> , <code>update_nodes_batch</code>
<code>InterEdgeCreated</code>	<code>add_inter_edges</code>
<code>IntraEdgeCreated</code>	<code>add_intra_edges</code>
<code>IntranodeEdgeCreated</code>	<code>add_intranode_edges</code>
<code>ParentEdgeCreated</code>	<code>add_parents</code>
<code>RootEdgeCreated</code>	<code>add_roots</code>
<code>InterEdgeRemoved / ...</code>	<code>corresponding removal methods</code>
<code>EdgeUpdated</code>	<code>update_link_field</code>
<code>LayerCreated</code>	<code>create_layer</code>
<code>LayerRemoved</code>	<code>remove_layer</code>
<code>GraphTransform</code>	<code>split</code> , <code>merge</code> , <code>explode</code> , <code>collapse</code>

Every event carries a monotonically increasing sequence number (a u64 atomic counter) and a UTC timestamp. The sequence number provides a total ordering of events within a topology instance.

14.5 Python Subscription API

Listing 10. Event subscription patterns

```
from ank_nte import TopologyEvent
import logging

log = logging.getLogger("topology.events")

# Basic subscription
```

```

def audit_log(event: TopologyEvent) -> None:
    log.info("AUDIT: %s seq=%d ts=%s payload=%s",
            event.event_type, event.sequence,
            event.timestamp_iso, event.to_json())

handle = t.subscribe(audit_log)

# Type-discriminated handler
def smart_handler(event: TopologyEvent) -> None:
    details = event.details()
    match event.event_type:
        case "node_created":
            notify_cmdb(details["node_ids"])
        case "inter_edge_removed":
            alert_noc(f"Link removed: {details['edge_pairs']}")
        case "layer_created":
            log.info("New layer: %s", details["layer_name"])

# Unsubscribe
t.unsubscribe(handle)

```

14.6 Subscription Patterns

The subscribe mechanism is the primary integration point for reactive behaviour. Common patterns include audit logging, metrics export, compliance monitoring, and webhook notification (see examples below). For high-frequency mutations (e.g. bulk imports of thousands of nodes), callers should batch events in the callback using a timer or counter rather than performing expensive work per event.

14.7 Use Cases

Prometheus integration

```

import prometheus_client as prom

node_gauge = prom.Gauge("nte_node_count", "Total nodes in topology")
link_gauge = prom.Gauge("nte_link_count", "Total links in topology")

def update_metrics(event: TopologyEvent) -> None:
    node_gauge.set(t.node_count())
    link_gauge.set(t.link_count())

t.subscribe(update_metrics)

```

Webhook notification

```

import httpx

def webhook_notify(event: TopologyEvent) -> None:
    if event.event_type in ("node_created", "node_removed"):
        httpx.post("https://cmdb.example.com/webhook",
                  json={"event": event.to_json()},
                  timeout=2.0)

t.subscribe(webhook_notify)

```

Compliance monitoring

```

from ank_nte import PolicyEngine

policy_engine = PolicyEngine.load("policies/design-rules.yaml")

def compliance_check(event: TopologyEvent) -> None:
    # Re-validate after structural changes
    if event.event_type in ("inter_edge_created", "inter_edge_removed",
                           "node_created", "node_removed"):

```

```

findings = policy_engine.validate(t)
if findings:
    for f in findings:
        log.warning("POLICY VIOLATION [%s] %s: %s",
                    f.severity, f.policy_id, f.message)

t.subscribe(compliance_check)

```

15 Distributed Deployment

Future Work

The distributed consensus and clustering features described in this section are currently under active research and are not yet implemented in the core engine. They are presented here to illustrate the intended architectural direction.

15.1 When to Use Clustering

A single NTE instance is intended to handle very large in-memory topologies on a modern server. Clustering is appropriate when:

- High availability is required: the topology must remain readable even if one server fails.
- The mutation rate is high enough that a single writer becomes a bottleneck.
- Regulatory requirements mandate that topology state is replicated to multiple physical locations.

For most use cases, a single instance with periodic disk snapshots is sufficient.

15.2 Raft Consensus via OpenRaft

The `nte-server` crate provides a standalone HTTP/WebSocket server (built on Axum and Tokio) with architectural scaffolding for an OpenRaft-based [12] distributed consensus layer [13]. The type configuration, request/response types, and RocksDB-backed log storage are in place; the RPC transport and state machine application logic remain incomplete. The design below describes the *intended* architecture; see the warning at the end of this section for the current implementation status.

In the target design, the topology is treated as a replicated state machine: every mutation is encoded as a `TopologyRequest` log entry, replicated to all followers, and applied only after a quorum confirms receipt.

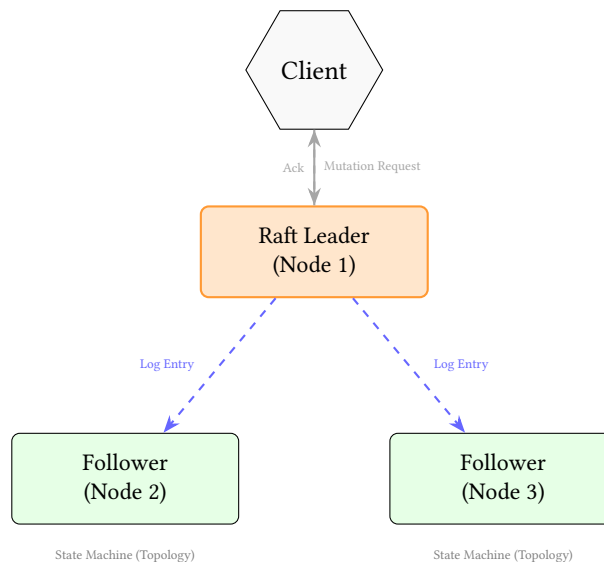


Figure 37. Distributed Raft consensus flow. Mutations are proposed to the leader, replicated to followers, and applied only after reaching a quorum.

Design Rule 16: Deterministic State Machine Replication

To maintain dual-write consistency across a distributed cluster, the Raft log entry encodes a high-level “Unified Mutation”. Upon commitment, each node in the cluster applies the same deterministic transformation to both its local petgraph instance and its local Polars DataFrames. This is intended to

keep the graph and relational representations aligned across the cluster, provided the write path remains deterministic.

The nte-server replication protocol defines a set of high-level `TopologyRequest` operations that are distributed via the Raft log. These include atomic node and edge additions, structural removals, and bulk property updates (which are shipped as JSON payloads to maintain cross-platform compatibility).

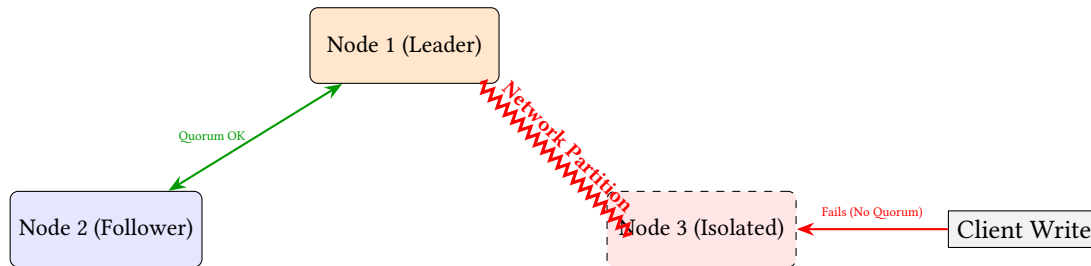


Figure 38. Split-Brain Resolution. In a partition, the isolated Node 3 rejects writes because it cannot replicate logs to a majority quorum. Nodes 1 and 2 continue processing safely.

Warning

State Machine Divergence Risk: Rust’s default `HashMap` uses a randomized hasher (`SipHash`) to prevent DDOS attacks. If any topology mutation logic iterates over a standard `HashMap` or `HashSet`, the iteration order will differ between the Raft Leader and Followers, causing the replica state machines to silently diverge. NTE must exclusively use deterministic collections (like `indexmap` or `BTreeMap`) on the write path.

The Raft type configuration uses:

- `NodeId` = `u64` — cluster member identifier.
- `D` = `TopologyRequest` — the command type replicated by Raft.
- `R` = `TopologyResponse` — the response returned to the client.
- `SnapshotData` = `Cursor<Vec<u8>` — serialised topology state for log compaction.

15.3 Configuration

A three-node cluster is the recommended minimum for Raft:

Listing 11. Cluster startup (environment variables)

```
# Node 1 (leader candidate)
NTE_NODE_ID=1
NTE_RPC_ADDR=10.0.0.1:9001
NTE_API_ADDR=10.0.0.1:8080
NTE_PEERS=10.0.0.2:9001,10.0.0.3:9001

# Node 2
NTE_NODE_ID=2
NTE_RPC_ADDR=10.0.0.2:9001
NTE_API_ADDR=10.0.0.2:8080
NTE_PEERS=10.0.0.1:9001,10.0.0.3:9001
```

15.4 Leader and Follower Roles

Leader Accepts all write requests. Replicates log entries to followers. There is exactly one leader at any time.

Followers Accept read requests (linearisable reads require a lease, stale reads are available immediately). Apply log entries from the leader.

Candidate Transient state during leader election.

Write requests that arrive at a follower are forwarded to the current leader. Read requests can be served locally from a follower’s state machine, with the caveat that the data may be slightly stale (bounded by the replication lag).

15.5 Snapshot Transfer

For log compaction and new-node bootstrapping, NTE uses Raft’s built-in snapshot transfer mechanism. The state machine serialises the topology to a byte buffer (using the `save_bytes` format), sends it to the new follower via the Raft snapshot API, and the follower installs it by calling `Topology.load_bytes`.

15.6 Security and Multi-Tenancy

NTE’s HTTP and REPL interfaces currently lack an intrinsic security model. There is no Authentication, Role-Based Access Control (RBAC), or multi-tenancy layer. Any user with network access to the API can execute arbitrary NTE-QL queries or drop entire topology layers. Deployments must place NTE behind a secure reverse proxy (e.g., NGINX, Envoy) and handle tenant isolation at the application layer. Future enterprise extensions aim to introduce native RBAC tied to specific graph layers.

Implementation Status: Architectural Proof

The distributed Raft scaffolding in `nte-server` defines the type configuration (`NteTypeConfig`), request/response types, and RocksDB-backed log storage. However, the RPC network layer currently returns stub errors, and the `RaftStateMachine::apply()` method is not yet connected to the topology mutation path. This section documents the *architectural design*, not production-ready functionality. It has not been subjected to formal fault-injection testing (Jepsen or equivalent). See Section 26 for the current known gaps.

16 Storage Architecture

Storage Architecture at a Glance

Why: Combining property filtering and graph traversal efficiently requires specialised structures.

What: A dual-write system pairing columnar DataFrames for fast property scans with adjacency lists for structural traversal.

16.1 Rationale for Dual-Write

Graph databases excel at traversal; columnar stores excel at predicate evaluation over properties. Network topology queries typically require both capabilities within a single operation. A query like “find all routers in the SYD PoP that have a BGP session to a peer with AS 65000” decomposes into two phases:

- *Property filter:* `pop == "SYD"`, best served by a columnar scan.
- *Structural filter:* connected via a BGP-layer edge to a peer with `as_number == 65000`, best served by graph traversal.

NTE’s dual-write approach keeps both representations atomically synchronised (Figure 39), allowing the query planner to select the optimal execution order for each phase of a hybrid query.

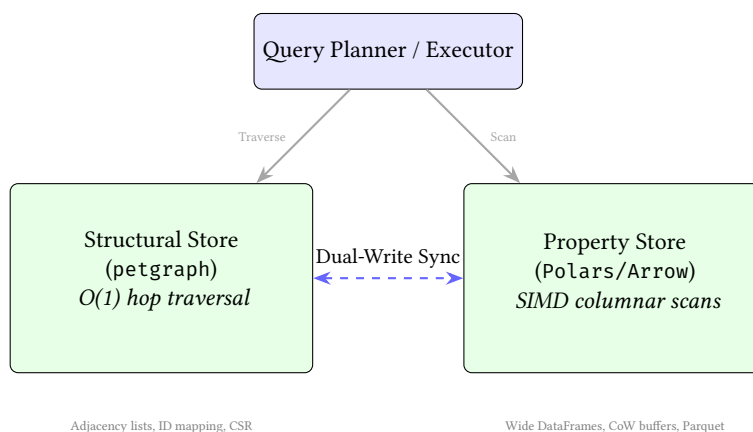


Figure 39. Dual-write architecture synergy. Structural queries target the graph engine, while property-heavy predicates target the columnar store.

16.2 petgraph StableDiGraph

The structural store uses a `petgraph::StableDiGraph<GraphNode, GraphEdge>` [14]. The stable variant was chosen over the non-stable `DiGraph` for two reasons:

- Node and edge indices remain valid after deletions. This is essential because NTE’s ID mapping (`HashMap<i32, usize>` in both directions) must not be invalidated by unrelated mutations.

- It allows $O(1)$ lookup of node weights by `NodeIndex`, which is the primary graph-layer operation.

The `NteGraph` struct wraps `StableDiGraph` and adds:

- **Bidirectional ID mapping:** `index_map: HashMap<i32, usize>` (external $\rightarrow$ internal) and `reverse_index_map: HashMap<usize, i32>` (internal $\rightarrow$ external). All Python-facing APIs use external `i32` IDs.
- **Endpoint / node ID sets:** `HashSet<i32>` for $O(1)$ “is this ID an endpoint?” checks.
- **Layer dependency graph:** `LayerDependencyGraph` for topological ordering and layer-consistency checks.
- **Device shortcut map:** `HashMap<(i32, i32), Vec<EdgeIndex>` for fast device-to-device lookups.
- **CSR cache:** a thread-safe, lazily-computed Compressed Sparse Row representation for high-performance batch traversal (described in Section 13.2).

16.3 Polars Columnar Store

The property store uses Polars [15] DataFrames, organised as a `HashMap<String, DataFrame>` keyed by node type (e.g., “Router”, “Switch”). Polars is built on the Apache Arrow columnar format, which provides three properties critical to the hybrid pipeline:

- SIMD-accelerated predicate evaluation (AVX2/AVX-512 on x86-64).
- Copy-on-Write (CoW) semantics for zero-copy snapshot cloning.
- Lazy evaluation via `LazyFrame` for filter push-down.

Properties are stored in wide columnar format (Figure 40): each unique property name becomes a column in the DataFrame. This is a deliberate trade-off. A narrow schema (one row per property) would be more flexible but would require an expensive “pivot” before columnar predicates could be applied. The wide schema means NTE scans efficiently but requires that all instances of a given node type have compatible types for a given field name.

Design Rule 17: Sparse Columnar Compression

A hyper-scale topology generates highly heterogeneous schema (e.g., a firewall has `fw_rules`, a router has `bgp_asn`). The engine must aggressively map all unique JSON keys to a wide columnar schema. Storage explosion is avoided via Apache Arrow’s validity bitmaps, ensuring that null values consume less than 1 bit per row while remaining perfectly aligned for SIMD vectorization [16].

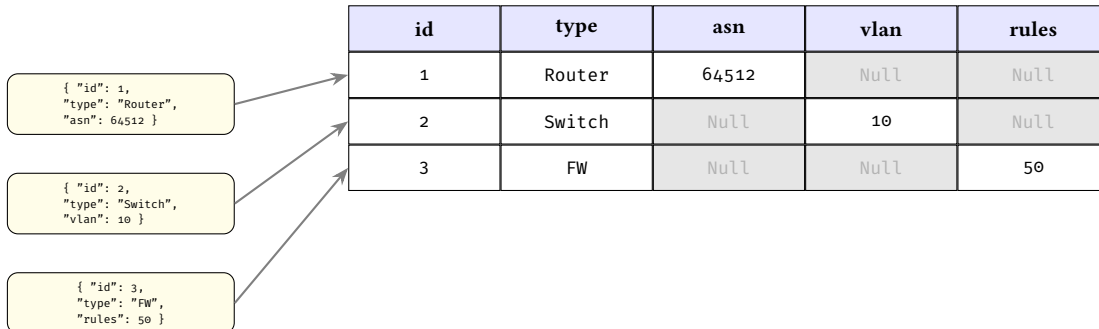


Figure 40. The Wide-Column Sparsity Matrix. Heterogeneous JSON properties are flattened into a single wide DataFrame. Arrow’s validity bitmaps efficiently compress the resulting Null regions.

16.4 Memory Fragmentation and Defragmentation

A side effect of using `StableDiGraph` for its index stability is the creation of “tombstones” (empty slots in internal arrays) when nodes or edges are deleted. In mutation-heavy workloads, such as iterative network synthesis or large-scale failure-model simulations, these holes destroy CPU cache locality and increase memory footprint.

Design Rule 18: Dynamic Memory Packing

To maintain hyper-scale performance, the engine must decouple *logical index stability* from *physical memory density*. NTE provides an atomic `pack()` operation that rebuilds the graph structure into a contiguous layout while preserving the isomorphism with the relational property store.

The `pack()` operation executes in $O(V + E)$ time by:

1. Allocating a new, dense StableDiGraph with exactly the capacity needed.
2. Mapping old node indices to new, contiguous indices.
3. Rewriting all internal state (ID maps, device shortcut caches, and edge indices embedded in weights) to reflect the new physical layout.

This operation is exposed to the Python layer as `defragment_memory()`, allowing long-running automation workers to periodically restore cache efficiency without losing data integrity.

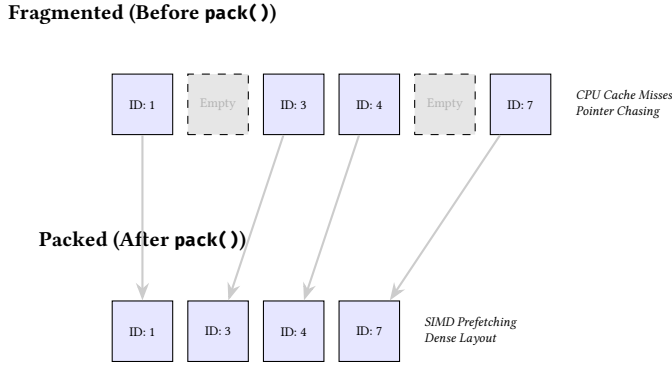


Figure 41. Memory Defragmentation (Graph Packing). The `pack()` operation removes tombstone slots from the internal adjacency arrays, restoring linear CPU cache locality and reducing memory overhead.

SIMD Vectorization. Unlike row-based graph databases that suffer from high cache-miss rates due to “pointer chasing,” NTE leverages the linear memory layout of Polars to perform SIMD-accelerated predicate evaluation. By loading property columns into 512-bit registers (AVX-512), the engine can evaluate up to 16 integer comparisons in a single CPU clock cycle.

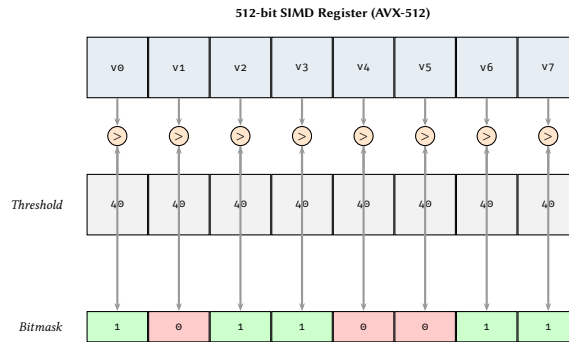


Figure 42. Hardware-Level Predicate Evaluation. A single AVX-512 instruction evaluates multiple property values in parallel, producing a bitmask that drives the subsequent graph traversal filter.

16.5 Dual-Write Consistency Protocol

Every mutation method in the `Topology` struct applies changes to both the graph and the `DataFrame` store before returning. The protocol is:

Algorithm 1 Dual-write mutation protocol

- 1: Acquire write lock on topology
 - 2: Validate inputs (type checks, layer compatibility, ID uniqueness)
 - 3: Apply mutation to `NteGraph` (structural side)
 - 4: Apply corresponding mutation to `DataFrameStore` (property side)
 - 5: If either step fails, rollback and propagate error
 - 6: Emit `TopologyEvent` to ring buffer
 - 7: Invalidate CSR cache
 - 8: Release write lock
-

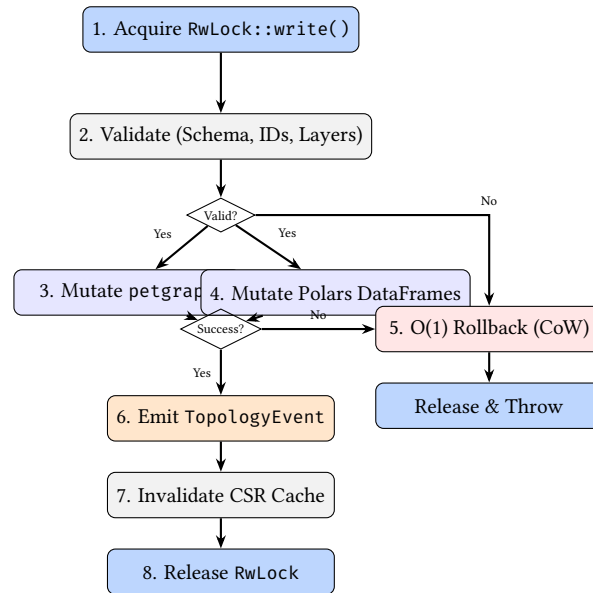


Figure 43. The dual-write transaction lifecycle. Copy-on-Write semantics allow $O(1)$ rollback if either the graph or the DataFrame mutation fails.

Note

NTE implements copy-on-write transaction semantics: if step 4 fails, the graph mutation in step 3 is rolled back via a compensating operation and the DataFrame store is restored from its pre-mutation Arrow CoW clone in $O(1)$. This ensures atomicity within a single process. Durability is provided via the **Write-Ahead Log (WAL)**: every committed mutation is appended to a persistent journal before the in-memory state is updated. Following a crash, the engine can reconstruct the exact topological state by replaying the WAL events (Section 12.1).

16.6 Foreign Function Interface & Zero-Copy Boundaries

The engine exposes a foreign function interface (FFI) primarily via PyO3 [17], which allows NTE to embed directly into a Python process. This approach is significantly faster than external RPC calls to a standalone graph database. To maximize concurrency and performance, NTE implements several boundary optimisations:

- **GIL Release:** The Topology struct is protected by a RwLock. Query operations acquire a read lock; mutations acquire a write lock. Crucially, the PyO3 binding layer releases the Python Global Interpreter Lock (GIL) before acquiring these Rust locks. This allows concurrent Python threads to execute native code or I/O while another thread holds a write lock in Rust.
- **Zero-Copy PyArrow:** When the arrow-ids feature is enabled, Polars DataFrames are shared with Python using the Arrow C Data Interface. This eliminates serialisation overhead, allowing Python data science tools to interact with NTE's columnar store directly and in zero-copy fashion.

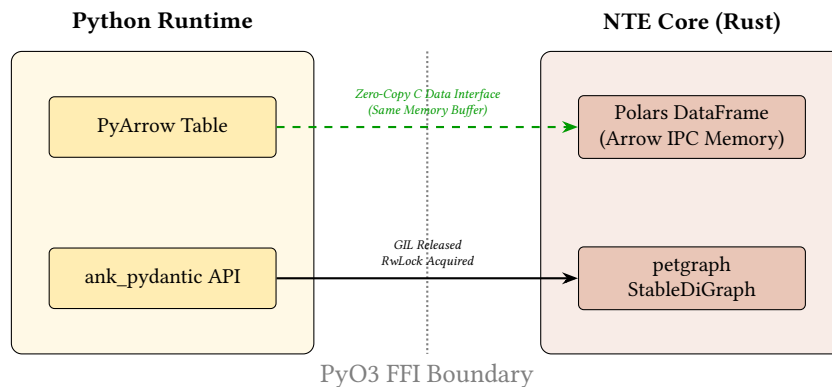


Figure 44. The FFI Zero-Copy Boundary. Unlike traditional RPC-based graph databases, NTE embeds directly in the Python process. The Arrow C Data Interface allows Python tools to read the Rust memory buffers without any serialisation overhead.

16.7 EventStore Ring Buffer

Every mutation emits a `TopologyEvent` variant into an in-memory ring buffer (a `VecDeque` managed by the `EventStore`). Events carry:

- A monotonically increasing `EventSequence` number (atomic u64).
- A UTC timestamp (`chrono::DateTime<Utc>`).
- Event-specific payload (node IDs, edge pairs, changed fields, etc.).

Events are serialisable to JSON (via `serde_json`) for external consumption. The ring buffer has a configurable capacity; once full, the oldest events are evicted.

16.8 Pluggable Backends

The storage layer is factored into three crates:

Crate	Description	Status
<code>nte-datastore-polars</code>	Polars Arrow-backed DataFrames	Default, production
<code>nte-datastore-lite</code>	Lightweight in-memory store	Development / testing

The `nte-backend` crate defines the abstract `TopologyBackend` trait that all backends implement. Selecting a backend is a compile-time feature flag in `nte-datastore`.

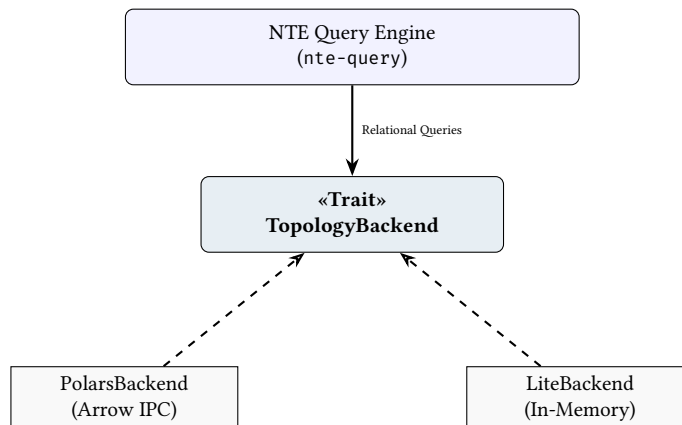


Figure 45. Trait-Based Abstraction. The core query engine interacts only with the `TopologyBackend` interface, allowing the physical storage backend to be swapped at compile time via feature flags.

17 Schema & Type System

NTE itself is schema-agnostic: it stores arbitrary key-value properties on nodes and edges without enforcing a fixed column set. Schema validation and type safety are provided by `ank_pydantic`, the Python framework that consumes NTE as its backend engine.

17.1 Pydantic-to-DataFrame Column Mapping

When `ank_pydantic` populates an NTE topology, it maps Pydantic model fields to Polars `DataFrame` columns using a `data_` prefix convention. This prevents collisions between user-defined properties and NTE’s internal metadata columns (e.g. `id`, `node_type`, `layer`).

Pydantic Field	Python Type	DataFrame Column	Polars Dtype
<code>hostname</code>	<code>str</code>	<code>data_hostname</code>	<code>Utf8</code>
<code>asn</code>	<code>int</code>	<code>data_asn</code>	<code>Int64</code>
<code>loopback</code>	<code>IPv4Address</code>	<code>data_loopback</code>	<code>Utf8 (serialised)</code>
<code>speed_gbps</code>	<code>float</code>	<code>data_speed_gbps</code>	<code>Float64</code>
<code>enabled</code>	<code>bool</code>	<code>data_enabled</code>	<code>Boolean</code>
<code>tags</code>	<code>list[str]</code>	<code>data_tags</code>	<code>List(Utf8)</code>

17.2 Wide Columnar Storage

NTE uses a *wide table* layout: each distinct property name becomes its own column in the per-type `DataFrame`. For a topology with 50 distinct property fields across all node types, the `df_routers` `DataFrame` might have 50+ columns, most of which are `null` for any given row.

This design trades storage density for query speed: a property scan (`WHERE hostname = "core-1"`) becomes a single-column SIMD scan rather than a JSON parse. Polars handles sparse columns efficiently via Arrow's validity bitmaps, so null-heavy columns incur minimal memory overhead.

Warning

Schemas with more than 10,000 distinct property keys will degrade query parsing and plan compilation performance. Avoid using dynamic keys (e.g. timestamps or UUIDs) as property field names. A "JSON overflow" column for extremely infrequent properties is planned.

17.3 Schema Validation

Schema validation is performed entirely at the Pydantic layer in `ank_pydantic`, before data reaches NTE. NTE itself accepts any JSON-serialisable property map without validation.

Design Rule 19: Schema Boundary

NTE enforces *structural* invariants (endpoint ownership, layer compatibility, dual-write atomicity) but delegates *property* validation to the consuming framework. This separation allows NTE to serve both strict design-time compilation (via `ank_pydantic`) and relaxed telemetry ingestion (via direct API calls) without schema conflicts.

17.4 Current Limitations

No schema evolution There is no built-in mechanism for migrating or versioning property schemas. Adding a new field to a Pydantic model creates the column lazily on first use; removing a field leaves the column in place (with all-null values) until the topology is rebuilt.

No NTE-native validation NTE does not validate property types at write time. If a caller writes a string to a column that previously held integers, Polars will upcast the column to a common supertype or raise a runtime error depending on the operation.

No cross-field constraints Constraints like "if role is spine, then uplink_count must be 4" must be expressed as policies (Section 5), not as schema rules.

18 Algorithm Reference

This section presents the key graph and relational algorithms that underpin NTE's query planner, structural analysis, and reactive maintenance subsystems. Each algorithm is illustrated as a state-machine diagram with explicit invariants, making the correctness argument visible at a glance.

18.1 Worst-Case Optimal Joins (Leapfrog Triejoin)

Illustrative Scenario: Hyperscale Fabric Verification In a 10,000-node Leaf-Spine data center, an operator runs a query to identify all potential 3-cycle feedback loops in BGP peering. A standard graph database generates millions of intermediate pairs before filtering for the third edge, causing high CPU usage and multi-second latency. NTE's WCOJ engine performs a simultaneous intersection of the edge iterators, returning the 45 actual loops in under 50ms, enabling real-time drift detection during automated configuration rollouts.

Code alignment: this section mirrors `LeapfrogJoin` and `TrieIterator` in `n-te-query`. The invariant "seek the minimum iterator up to max" is implemented by `LeapfrogJoin::search()` and advanced by `LeapfrogJoin::next()`.

Binary joins can create large intermediate result sets on cyclic patterns (triangles, cliques). NTE implements (and continues to integrate) a Worst-Case Optimal Join (WCOJ) path using Leapfrog Triejoin. Concretely, each variable binding step is reduced to a *multi-way intersection* over sorted iterators.

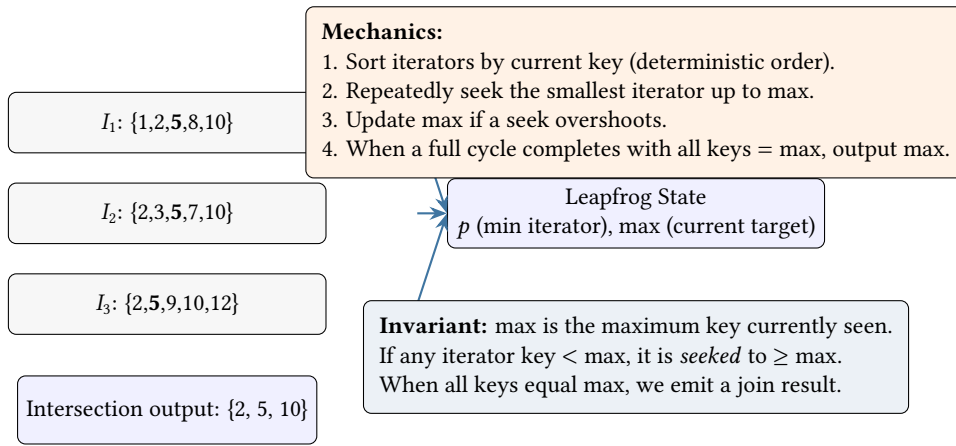


Figure 46. Leapfrog Triejoin as a stateful multi-way intersection. The algorithm maintains a single target (max) and advances the smallest iterator until all iterators align on the same key. This avoids intermediate-result explosion on cyclic patterns by intersecting all constraints simultaneously.

Why this is conceptually valuable: the join runtime is driven by iterator movement and the size of the final output, rather than by constructing and materialising large temporary relations. In NTE, the iterators are backed by CSR-sorted neighbour lists (and other filtered index views), making the intersection primitive directly usable for dense fabrics.

18.2 Betweenness Centrality (Brandes)

Code alignment: implemented as `AnalysisGraph::betweenness centrality()` using Brandes' two-phase structure: a BFS computing dist and path counts sigma, followed by the reverse delta accumulation over the predecessor lists.

Betweenness centrality is easy to state ("fraction of shortest paths passing through a node") but non-obvious to compute efficiently. NTE uses Brandes' algorithm, which can be understood as two coupled phases per source node: a BFS that builds a shortest-path DAG and counts path multiplicity (σ), followed by a reverse accumulation of dependencies (δ).

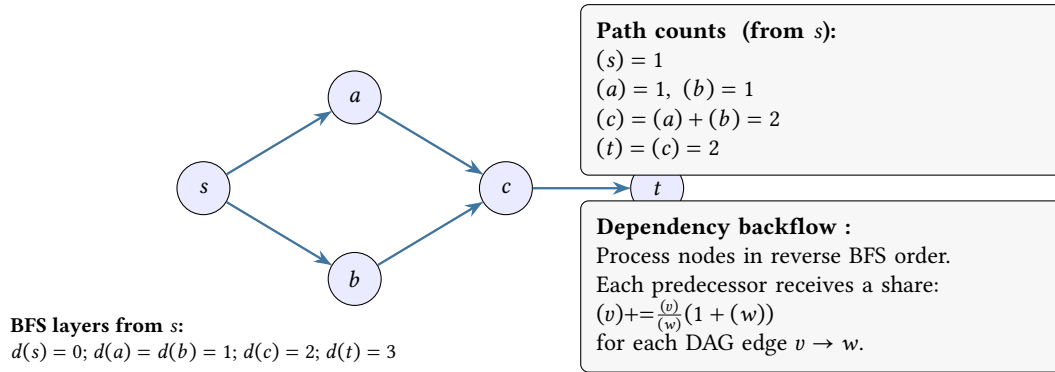


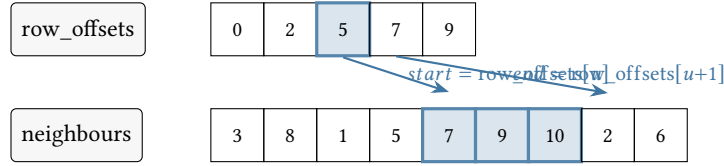
Figure 47. Brandes betweenness centrality as a conserved "flow" on the shortest-path DAG. The forward BFS counts how many shortest paths reach each node (σ). The reverse accumulation distributes dependency (δ) back to predecessors in proportion to their contribution.

Invariant to remember: for a fixed source s , the reverse pass ensures each node collects exactly the amount of "shortest-path responsibility" it carries for reaching deeper nodes. Summing this over all sources yields betweenness.

18.3 CSR Memory Layout

Code alignment: the conceptual arrays correspond to the CSR cache used for traversal in `n-te-graph`. Neighbour iteration is a slice over contiguous buffers, which is also the input shape required by the WCOJ trie iterators (sorted neighbour lists).

The CSR (Compressed Sparse Row) representation converts pointer-heavy adjacency lists into two contiguous arrays. This matters because traversal and intersection performance is dominated by memory locality: sequential reads over dense arrays are dramatically faster than chasing per-node heap allocations.



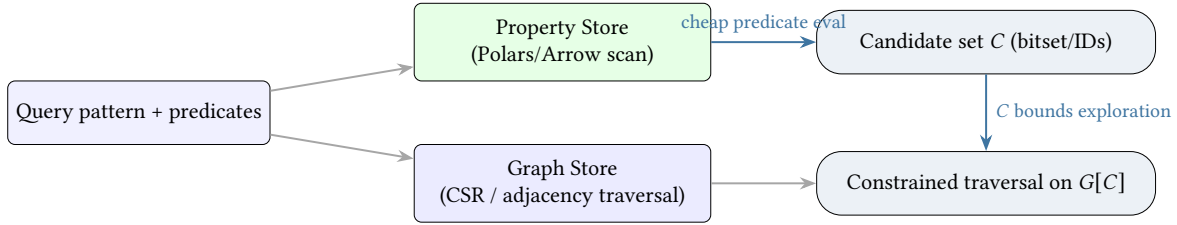
Invariant: neighbours of a node u live in a contiguous slice $\text{neighbours}[\text{start}..\text{end}]$. This enables:
 (1) BFS/DFS with sequential reads, and
 (2) set intersections (e.g. WCOJ) via merge/binary-search on sorted slices.

Figure 48. CSR layout. Two contiguous arrays encode the entire graph neighbourhood structure. A node’s adjacency list is a slice in the neighbours array, indexed by row offsets. Locality and sorted-slice operations are the performance foundation for traversal and join-style intersections.

18.4 Filter-First Query Planning

Code alignment: the planner’s costing and selectivity heuristics live in `nfe-query/src/planner/` (notably `cost.rs`). The candidate-set concept corresponds to planning decisions that push property predicates earlier to reduce the induced traversal subgraph.

NTE’s filter-first planner turns “property predicates” into cheap candidate sets before attempting expensive structural traversal. The key idea is to make the search space explicit as a *bitset* / *ID set* that monotonically shrinks.



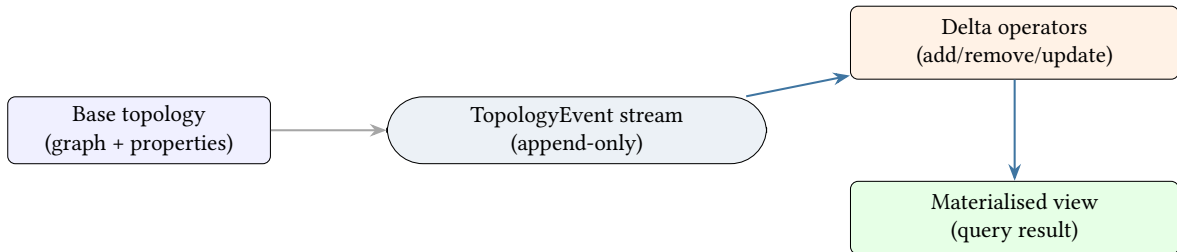
Invariant: C only shrinks (or stays equal).
 Traversal cost becomes proportional to the induced subgraph.

Figure 49. Filter-first planning as candidate-set narrowing. Property predicates are evaluated first to produce a set C of eligible nodes/edges. Structural traversal then runs against the induced subgraph $G[C]$, avoiding work on irrelevant regions of the topology.

18.5 Incremental View Maintenance (IVM)

Code alignment: the IVM loop consumes `TopologyEvent` values and updates its maintained state per event (see `process_event`). The “fold over events” invariant is the reason replay and continuous maintenance share the same semantics.

Reactive query results can be maintained incrementally by consuming the topology’s event stream. Instead of recomputing a derived view from scratch after every mutation, IVM applies small deltas to keep the view consistent.



Invariant: after processing events $0..t$, the view equals
 $V_t = f(\text{Initial}) + \sum_{i=0}^t \delta_i$.
 This turns “continuous queries” into a deterministic fold over events.

Figure 50. Incremental View Maintenance (IVM). Derived results are maintained by applying per-event deltas, rather than recomputing from scratch. This is the algorithmic bridge between the WAL/event model and reactive query semantics.

18.6 GPU SSSP as Wavefront Relaxation

Code alignment: the GPU path operates over compact buffers and iterates frontiers until convergence. This section describes the execution model (bulk relaxations), not the specific kernel API.

GPU acceleration is most effective when an algorithm can be expressed as repeated bulk operations over a *frontier*. Single-Source Shortest Paths (SSSP) in this form becomes a wavefront of distance relaxations.

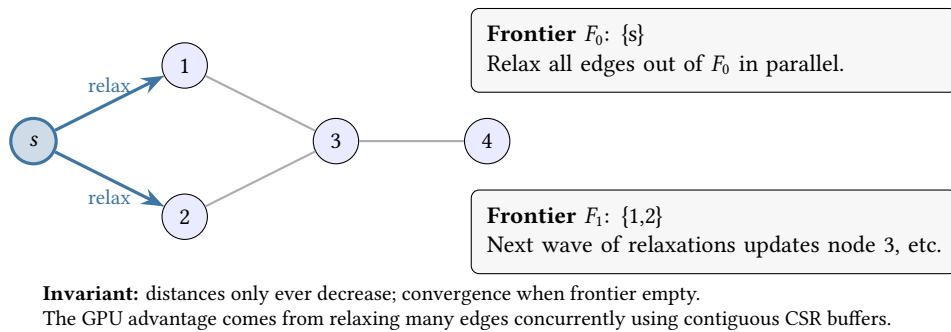


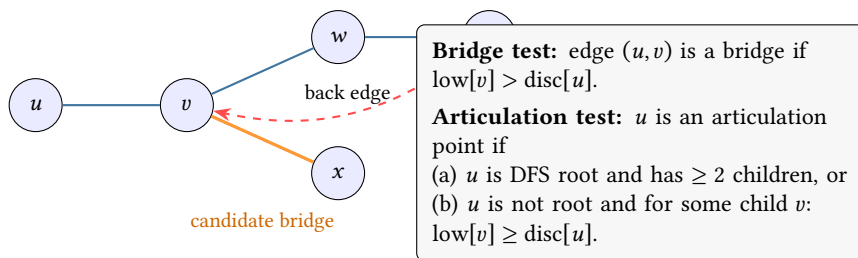
Figure 51. GPU SSSP as wavefront relaxation. Each iteration processes a frontier of active vertices and relaxes outgoing edges in parallel (drawing on parallel connectivity concepts [18]). This maps naturally onto GPU execution when adjacency is stored in compact buffers.

18.7 SPOF Discovery (Tarjan Bridges and Articulation Points)

Code alignment: bridge detection is implemented in `AnalysisGraph::bridges()` and articulation points in `AnalysisGraph::articulation_points()`. The diagram matches the `disc/low` propagation and the bridge predicate `low[v] > disc[u]`.

To identify Single Points of Failure (SPOFs) we often need two closely related primitives: *bridges* (edges whose removal disconnects the graph) and *articulation points* (vertices whose removal disconnects the graph). NTE implements Tarjan-style DFS algorithms using two timestamps per DFS vertex:

- `disc[u]`: discovery time of u in DFS.
- `low[u]`: smallest discovery time reachable from u using *zero or more* tree edges plus *at most one* back edge.



Intuition: `low[v]` answers: "can this subtree escape upwards without going through its parent?" If not, the parent edge/node is a cut. In a routing context, this identifies critical physical links whose failure would partition a BGP confederation or sever an MPLS LSP.

Figure 52. Tarjan-style SPOF primitives. Discovery times (`disc`) order the DFS tree; low-link values (`low`) summarise whether a subtree has a back-edge escape route. The bridge/articulation tests are simple comparisons once `low` is known.

In practice, this gives a deterministic, explainable structural risk signal: "these edges/nodes are cut points". This is often more actionable than a raw connectivity boolean.

18.8 Redundancy Capacity (Max-Flow / Min-Cut)

Code alignment: the user-facing entry point is `AnalysisGraph.max_flow` in the PyO3 bindings, which delegates to the underlying analysis graph implementation. This report uses the unweighted "edge-disjoint paths" interpretation as the policy-relevant mental model.

For traffic engineering and policy validation, "is there a path?" is too weak. We often want to know *how many independent ways* two nodes can stay connected. In an unweighted graph, max-flow can be interpreted as the number of edge-disjoint paths, which by Menger's theorem equals the size of a minimum cut.

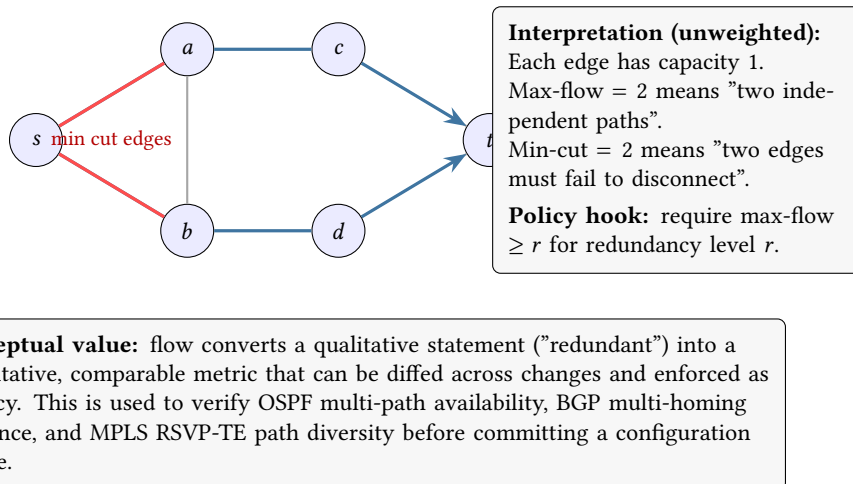


Figure 53. Max-flow/min-cut as a redundancy capacity metric. In an unweighted topology, setting each edge capacity to 1 makes the max-flow equal the number of edge-disjoint paths. The minimum cut identifies the smallest failure set that disconnects s from t .

19 Query System

Query System at a Glance

Why: Naive graph traversal is slow when properties are dense and selective.

What: A filter-first planner that drastically reduces datasets via parallel column scans before invoking structural graph traversal.

19.1 Design Goals

NTE's query system was designed around three principles:

1. **Filter-first execution:** property predicates should run before graph traversal, not after, to eliminate candidates early.
2. **Composable, typed query objects:** queries should be buildable programmatically from Python, not only via string parsing, so that tooling can construct, inspect, and serialise query plans.
3. **Honest cost estimation:** the planner should rank predicates by selectivity rather than always applying them left-to-right.

19.2 Query Representations

The nte-query crate exposes two parallel query representations:

QuerySpec A flat set of node-level filters: type, layer, kind, ID set, field equality, and expression filters. Used for "give me all nodes matching these conditions" queries. Exposed as QuerySpec in Python.

QueryPlan / PatternNode A structural graph pattern AST. Describes a subgraph shape to match (e.g. a router connected to a switch via an Connectivity edge). Used for multi-hop traversal queries. Exposed as QueryPlan and PatternNode in Python.

Design Rule 20: Predicate Pushdown

In the hybrid query planner, property filters and type assertions must be pushed down to the relational store (Polars) before any structural traversal begins. This reduces the candidate node set via SIMD vectorisation, effectively converting a complex $O(V + E)$ graph search into a constrained $O(V \bar{d}^h)$ bounded walk, where $\bar{d}$ is the predicate selectivity (Proposition 1).

19.3 Execution and Optimisation

The query compilation pipeline (Figure 63) translates high-level NTE-QL strings or Python QueryPlan objects into executable filter and traversal plans.

19.3.1 Query Compilation and Execution Planning

Before execution, high-level NTE-QL strings or Python QueryPlan objects must be translated into an executable form. This process is known as **Query Compilation**. It is during this compilation phase that the **Virtual Cypher Translation** (described in Section 4) is instantiated.

The compilation process decomposes the semantic query into its relational and structural components:

1. **AST Generation:** The NTE-QL string (e.g., `MATCH (a:Router)-[:eBGP]->(b:Router)`) is parsed into a high-level Abstract Syntax Tree (AST).
2. **Semantic Expansion:** The Query Planner recognises the device-to-device relationship. It mutates the AST, inserting the necessary Structural and Connectivity hops to satisfy the physical Endpoints Model constraints.
3. **Filter Compilation:** Cypher labels (like `:Router`) and inline properties are stripped from the AST and compiled into Polars LazyFrame execution plans.
4. **Structural Compilation:** The remaining expanded AST (representing the 3-hop traversal) is compiled into a petgraph finite state machine.

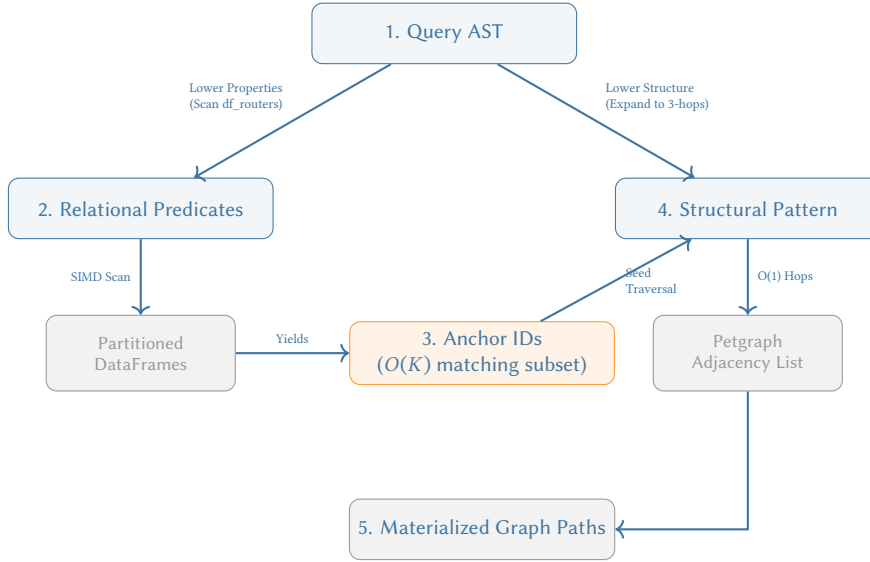


Figure 54. The Filter-First Hybrid Execution Pipeline. During the AST compilation phase, the engine splits the query. It executes highly-selective property predicates against partitioned columnar storage *first*, yielding a tiny subset of Anchor IDs. These anchors seed the expanded 3-hop structural graph traversal, guaranteeing that the engine never traces paths from irrelevant nodes.

The planner uses a **Cost-Based Execution Planning** strategy: if a query contains highly selective property filters (e.g., a specific hostname), the relational scan is executed first to prune the graph search space. If the query is purely structural, it is compiled directly for the petgraph engine.

19.3.2 Query Complexity: SIMD Pruning vs. Adjacency Traversal

The performance advantage of NTE’s hybrid approach can be quantified using Big-O analysis. Consider a property-heavy structural query (e.g., “Find all Routers where pop=’SYD’ and follow their 1-hop links”).

Naive Traversal cost $O(V + E)$. In a row-based graph store, we must visit every node V to check the property, then traverse its adjacency list E . This is dominated by pointer chasing and cache misses.

NTE Filter-First cost $O(\frac{V}{K} + E_{sub})$. where K is the SIMD vectorization factor ($K = 16$ for AVX-512) and E_{sub} is the set of edges connected only to the matching nodes.

Since $E_{sub} \ll E$ in selective queries, the query time is reduced by approximately an order of magnitude. The relational scan $O(V/K)$ is a linear scan of contiguous memory, maximizing cache locality and CPU throughput.

19.3.3 QuerySpec: The Relational Path

The QueryExecutor in nte-query compiles a QuerySpec into an ordered FilterPlan — a sequence of FilterOp variants sorted by FilterRank:

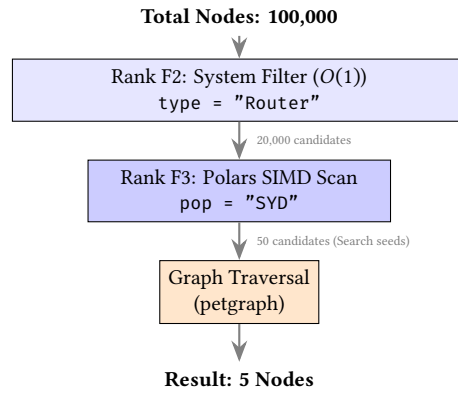


Figure 55. The Cardinality Funnel (Filter-First Execution). Cheap columnar operations aggressively prune candidates, ensuring the more expensive graph traversal engine only visits a tiny fraction of the topology.

Execution proceeds left-to-right. Each filter receives the candidate set from the previous filter and returns a (potentially smaller) subset. The Polars LazyFrame pipeline is constructed in-order and materialised with a single `collect()` call, benefiting from Polars’ internal optimiser.

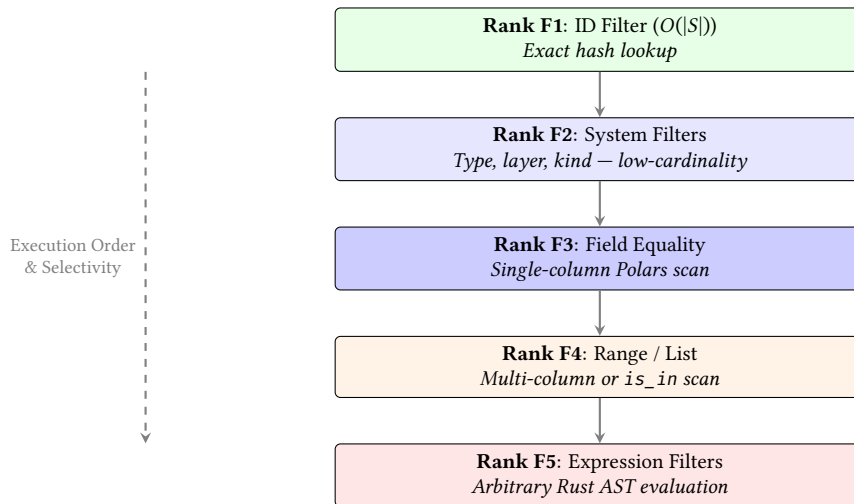


Figure 56. Query Planner Execution Ranks. Predicates are sorted and executed from cheapest (F1) to most expensive (F5) to continuously reduce the candidate set.

Filter plan example

Query: “All routers in the SYD PoP with AS number 64512”

1. F2: `type = "Router"` → scan `node_types` DataFrame
2. F3: `pop == "SYD"` → filter `data_pop` column
3. F3: `as_number == 64512` → filter `data_as_number` column

Polars executes steps 2 and 3 as a single lazy pipeline after step 1 narrows the candidate set.

19.3.4 Pattern Matching

The `PatternNode` AST describes the shape of a subgraph. Supported patterns:

`PatternNode::node(type)` Match a single vertex of the given type.

`PatternNode::binding(name, type)` Match a vertex and bind it to a name for later reference.

`PatternNode::chain(steps)` A sequence of alternating node-steps and edge-steps.

`PatternNode::union(patterns)` Union of multiple patterns.

Variable-length paths `HopSpec::exact(n)`, `HopSpec::range(min,max)`, or `HopSpec::any()`.

The pattern matcher performs a depth-first search over the graph, pruning branches when a node fails the pattern constraint. The maximum number of matches is capped at `QueryPlan::DEFAULT_MAX_MATCHES` (10,000) by default to prevent runaway traversals; the result carries a truncated flag when this cap is reached.

19.3.5 Multi-Stage Execution: WITH Pipeline

The **WITH** clause enables multi-stage query pipelines where each stage's matched bindings constrain subsequent stages. This is implemented as a sequential binding propagation mechanism in the QueryExecutor.

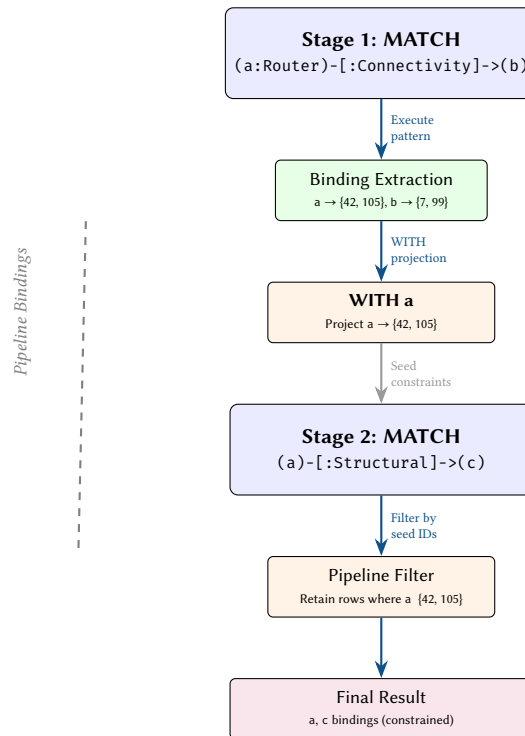


Figure 57. **WITH** clause pipeline execution. Each stage produces binding sets that are projected through **WITH** and used to constrain subsequent stages. The pipeline bindings accumulate across stages, with repeated binding names intersected to ensure consistency.

The implementation extracts matched node IDs from each stage's results, filters them through the **WITH** projection list, and merges them into a cumulative `pipeline_bindings` map. When a binding name appears in multiple stages, the ID sets are intersected. The final pattern match is then filtered to retain only rows whose bindings fall within the allowed ID sets.

19.3.6 OPTIONAL MATCH and Null-Fill

OPTIONAL MATCH implements left-outer-join semantics for graph patterns. When the optional pattern produces no matches but upstream pipeline bindings exist, the executor emits a synthetic `MatchTuple` with a `-1` sentinel value for unmatched bindings, preserving the cardinality of the driving (non-optional) stage.

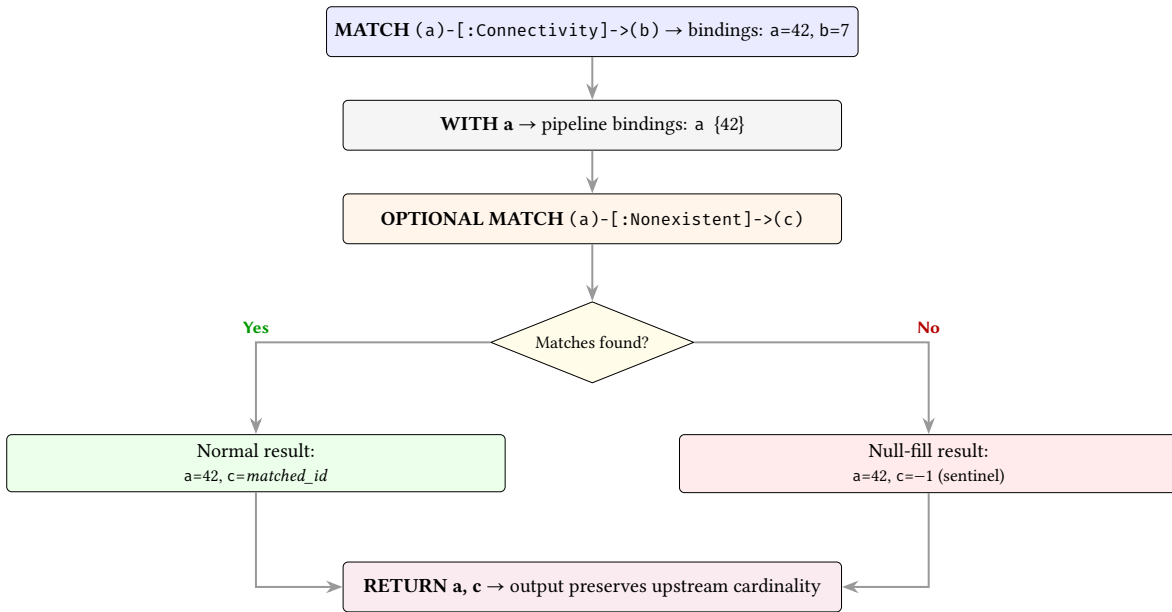


Figure 58. OPTIONAL MATCH null-fill semantics. When the optional pattern has no matches, a synthetic tuple is emitted with upstream bindings populated and new bindings set to `-1`, ensuring left-outer-join cardinality preservation.

19.3.7 NOT EXISTS Correlated Sub-queries

The `NOT EXISTS` operator filters outer matches by executing a correlated sub-query and checking whether any inner result shares bindings with the outer row. Unlike traditional seeded sub-queries (which inject candidate IDs into the inner query’s filters), NTE uses **post-filter binding correlation**: the sub-query runs un-seeded and the results are correlated after execution.

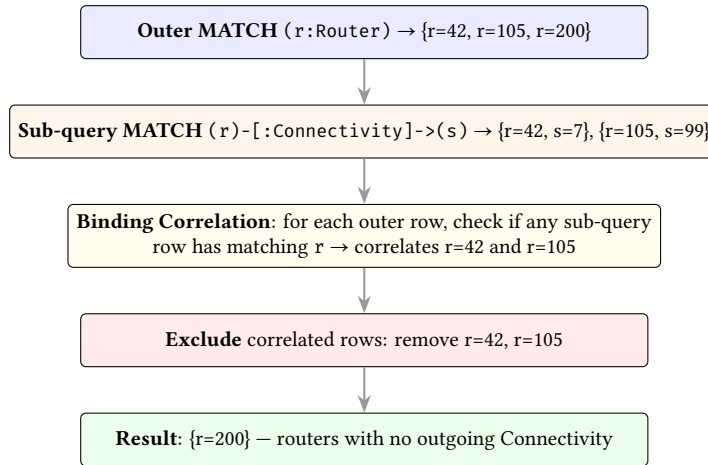


Figure 59. NOT EXISTS correlated sub-query execution. The sub-query runs independently; outer matches are retained only when no inner match shares the same binding values.

Design Rule 21: Post-Filter Correlation

The correlation-based approach avoids injecting `BindingId` filter nodes into the sub-query’s expression AST (which the Polars filter compiler does not support). Instead, all shared binding names between the outer and inner match tuples are compared after both executions complete.

19.3.8 Direction-Aware Edge Traversal

NTE’s pattern matcher respects edge direction through a three-valued `EdgeDirection` enum: `Forward`, `Backward`, and `Undirected`. The NTE-QL compiler maps arrow syntax directly:

NTE-QL Syntax	Direction	Traversal
<code>(a)-[:T]->(b)</code>	Forward	<code>filtered_neighbors()</code>
<code>(a)<-[:T]-(b)</code>	Backward	<code>filtered_predecessors()</code>
<code>(a)-[:T]-(b)</code>	Undirected	Both directions merged

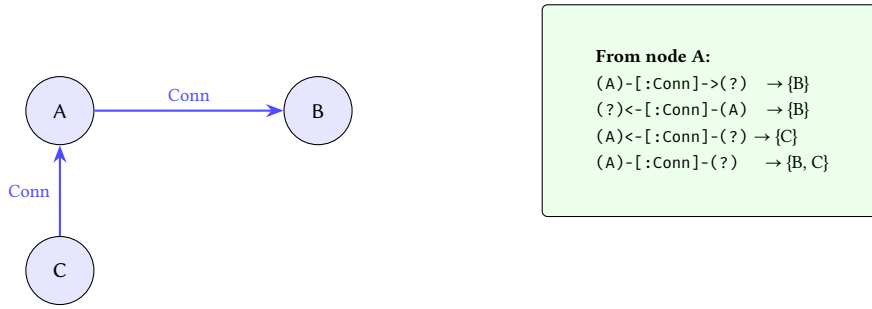


Figure 60. Direction-aware edge traversal. Forward follows outgoing edges, Backward follows incoming edges, and Undirected merges both directions.

19.3.9 DISTINCT Scoped to RETURN Bindings

The **DISTINCT** modifier deduplicates results based on the bindings listed in the **RETURN** clause, not the full match tuple. This is critical for queries where multiple structural paths lead to the same logical result.

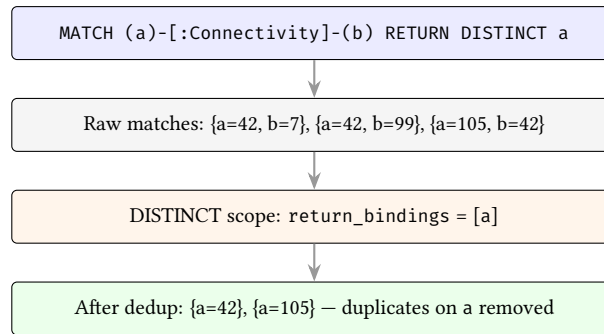


Figure 61. **DISTINCT** scoping. When `return_bindings` is non-empty, deduplication considers only the specified bindings. Without this scoping, all three raw matches would be considered distinct (since `b` differs).

19.3.10 Aggregation Execution

NTE-QL supports in-engine aggregation functions: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**, and **COLLECT**. These execute as a post-processing step over pattern match results, with group-by semantics derived from non-aggregated return items.

Design Rule 22: Vectorized Aggregation

To avoid the overhead of row-wise property lookups, the query engine offloads aggregation to the Polars relational store. For a given match set, the engine extracts the target node IDs, partitions them by type, and invokes native SIMD-accelerated Polars aggregation (`lazy.group_by().agg()`) to compute results at wire speed.

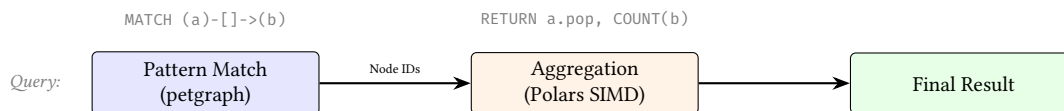


Figure 62. Aggregation execution pipeline. Match results are offloaded to Polars for vectorized aggregation, leveraging SIMD for **SUM** and **AVG** operations.

19.3.11 Cyclic Patterns and WCOJ Edge Recovery

The Leapfrog Triejoin (WCOJ) identifies node IDs that satisfy cyclic patterns by intersecting candidate sets across all positions. While node IDs are sufficient for structural matching, many queries require link-level properties (e.g., `MATCH (a)-[l:wire]->(b)-[]->(a) RETURN l.bandwidth`).

To satisfy these, the executor performs **Edge Recovery**: for every node tuple yielded by the join, it initiates a high-performance $O(1)$ adjacency lookup on the petgraph store to retrieve the specific `EdgeIndex` values connecting the nodes, effectively materializing the full path without a global graph scan.

19.3.12 Selectivity Ranking in Detail

The five filter ranks correspond to the following `FilterOp` variants:

ID Filtering (`FilterRank::Id`) — `HashSet<i32>` lookups are the fastest and always execute first.

System Filtering (`FilterRank::System`) — Queries on Type, Layer, or Kind use low-cardinality columns.

Equality Filtering (`FilterRank::Equality`) — Single-field property equality scans.

Range Filtering (`FilterRank::Range`) — Set membership (IN) or numerical range checks.

Expression Filtering (`FilterRank::Expression`) — Arbitrary `ExprNode` AST evaluations, always executed last.

Field equality filters on list values are promoted from Equality to Range rank because they require a Polars `is_in` operation, which is costlier than a direct equality predicate.

19.3.13 Polars Native Expression Pushdown

A key advantage of integrating deeply with Polars is the ability to push complex mathematical and string evaluations down into the SIMD-accelerated relational engine. Instead of materialising all node properties into Rust objects and evaluating abstract syntax trees node-by-node, NTE compiles `ExprNode` trees into native Polars `Expr` objects via the `compile_to_polars` pipeline.

String Operations Expressions like `CONTAINS`, `STARTS WITH`, and `REGEX MATCH` are compiled into `expr.str.contains()` and evaluated via Polars' native Arrow string kernels.

Arithmetic Operations Mathematical trees (e.g., `a.speed_gbps * 1000 > 40000`) are compiled into nested Polars arithmetic expressions, enabling vectorised evaluation across the entire column simultaneously without Python or Rust iterative overhead.

If an expression involves properties across different nodes that require post-join correlation, the planner falls back to the `FilterRank::Expression` stage (executed in Rust after traversal).

19.3.14 Query Profiling

Setting `QueryHints{enable_profiling: true}` in a `QuerySpec` records timing data for each filter step. The profiler captures per-step candidate counts and wall-clock durations, enabling identification of expensive filters. The profile is returned alongside the result set.

19.3.15 Plan Cache

NTE maintains a two-tier LRU cache system:

query_cache Caches query result ID sets for `QuerySpec` queries (capacity configurable via `ANK_NTE_QUERY_CACHE`, default 128 entries).

pattern_cache Caches pattern matching results for structural graph queries (capacity configurable via `ANK_NTE_PATTERN_CACHE`, default 64 entries).

The cache key for `query_cache` is a `CacheKeyMaterial` struct containing sorted, canonicalised representations of all filter fields (to avoid cache misses from `HashMap` ordering non-determinism). Cache keys are computed lazily via `OnceLock` to avoid repeated sorting allocations.

Note

Both caches store query results (ID sets and matched pattern data respectively), not compiled plans. All caches are invalidated on any topology mutation to ensure consistency.

19.4 User-Facing Interfaces

NTE exposes the query engine through multiple interfaces, each targeting a different user persona and integration context.

19.4.1 Query API Decision Matrix

NTE provides three distinct query paradigms. To avoid fragmentation, callers should select their interface based on the following matrix:

Interface	Primary Audience	When to use
Programmatic (QuerySpec)	Integrators	Internal system-to-system queries where parameters are highly dynamic or built from ASTs.
Fluent Builder	Data Scientists	Interactive Python notebooks; chaining complex property filters.
NTE-QL (Cypher)	Network Engineers	Human-readable graph pattern matching, CLI/TUI usage, and external dashboarding.

19.4.2 NTE-QL String Interface

In addition to the programmatic query API, NTE supports a Cypher-inspired string query language called NTE-QL. Queries are parsed by a parser combinator grammar implemented with the `chumsky` library and compiled into `QueryPlan` / `QuerySpec` objects before execution.

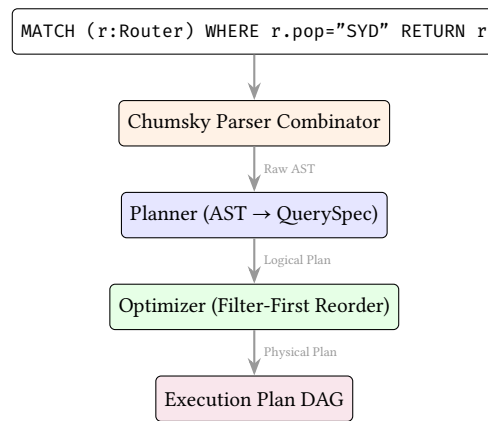


Figure 63. The NTE-QL compilation pipeline. String queries are parsed, converted into programmatic QuerySpec structures, and fed into the same optimizer as the Python APIs.

The grammar supports a variety of expressions. Here is a brief survey of supported query patterns:

Simple node scan

```

MATCH (r:Router)
WHERE r.pop = "SYD" AND r.as_number = 64512
RETURN r.hostname
  
```

Two-hop traversal: direct BGP peers

```

MATCH (r:Router)-[:Connectivity]->(s:Router)
WHERE r.hostname = "core-01"
RETURN r.hostname, s.hostname, s.as_number
  
```

Bounded-hop reachability: within 3 hops

```

MATCH (src:Router)-[*1..3]->(dst:Router)
WHERE src.hostname = "edge-01"
RETURN dst.hostname
  
```

Aggregation: node degree

```

MATCH (r:Router)-[:Connectivity]->(s:Router)
WHERE r.pop = "SYD"
RETURN r.hostname, COUNT(s) AS peer_count
ORDER BY peer_count DESC
  
```

NOT / exclusion: isolated routers

```

MATCH (r:Router)
WHERE NOT EXISTS {
  MATCH (r)-[:Connectivity]->(r:Router)
}
RETURN r.hostname
  
```

Path finding: shortest path

```

MATCH (src:Router)-[]-(dst:Router)
RETURN shortestPath(src, dst)
  
```

WITH clause: intermediate result piping

```
MATCH (a:Router)-[:Connectivity]->(b:Router)
WITH a
MATCH (a)-[:Structural]->(p)
RETURN a.hostname, p
```

OPTIONAL MATCH: left-outer-join pattern

```
MATCH (r:Router)
OPTIONAL MATCH (r)-[:Connectivity]->(s:Router)
RETURN r.hostname, s.hostname
```

Fully Implemented

All of the above examples are fully implemented in the NTE-QL parser, compiler, and executor: aggregations (COUNT/ORDER BY), shortestPath(), WITH piping, OPTIONAL MATCH, and NOT EXISTS correlated sub-queries. NOT EXISTS uses post-filter binding correlation (Section 19.3.7).

19.4.3 The ExprNode AST

Complex filter predicates are represented as an ExprNode abstract syntax tree. The full set of node types:

Variant	Description
Field { name, binding: Option<String> }	Property access, optionally scoped to a named binding
Literal(value)	Constant (null, bool, int, float, string, list)
Not(expr)	Logical negation
Logical(op, left, right)	AND / OR
Comparison(op, left, right)	=, ≠, <, >,
Arithmetic(op, left, right)	+, −, ∙, ÷
StringOp(op, field, pattern)	contains, startsWith, endsWith, matches
IsNull(expr) / IsNotNull	Null checks
IsIn(field, values)	Membership test

19.4.4 Python Fluent Query Builder

In addition to the programmatic QuerySpec / PatternNode APIs and the NTE-QLstring interface, NTE provides a fluent Python query builder that does not require `ank_pydantic`. The builder is inspired by Polars' expression API and compiles down to the same QuerySpec objects used by the Rust executor.

Fluent query builder: node queries

```
from ank_nte import Topology
from ank_nte.query import QueryNamespace, Expr

t = Topology()
# ... populate topology ...
q = QueryNamespace(t)

# Chainable node query
syd_routers = (
    q.nodes()
    .of_type("Router")
    .in_layer("physical")
    .filter(Expr.field("pop") == "SYD")
    .sort("hostname")
    .ids()
)

# Expression DSL supports operators
fast_ports = (
```

```

    q.nodes()
      .of_type("Endpoint")
      .filter(Expr.field("speed_gbps") > 40)
      .count()
  )

  # String operations
  juniper_devices = (
    q.nodes()
      .of_type("Router")
      .filter(Expr.field("vendor").contains("Juniper"))
      .collect()
  )

```

Fluent query builder: link queries

```

# Link queries for edge filtering
transit_links = (
  q.links()
    .of_type("Connectivity")
    .in_layer("physical")
    .filter(Expr.field("speed_gbps") >= 100)
    .ids()
)

# Between-nodes filtering
r1_links = (
  q.links()
    .between(source_ids=[1], target_ids=[2])
    .collect()
)

```

The expression DSL supports the full ExprNode AST via Python operator overloading: `==`, `!=`, `>`, `>=`, `<`, `<=` for comparisons; `&` (and), `|` (or), `~` (not) for logic; `+`, `-`, `*`, `/` for arithmetic; and methods `.contains()`, `.startswith()`, `.endswith()`, `.matches()` for string operations. Terminal methods include `.ids()`, `.collect()`, `.count()`, `.exists()`, `.first()`, and `.one()`.

19.4.5 Query Plan Visualisation

NTE can render query execution plans as Mermaid flowchart diagrams. This is useful for understanding how the planner decomposes a query and for identifying optimisation opportunities. The `nte-query` crate provides a recursive generator that traverses the ExecutionPlan tree to emit Mermaid flowchart syntax, mapping NodeScan, EdgeScan, WcojJoin, and SemiJoin operations to visual blocks.

The visualisation is accessible in two ways:

1. **NTE-QL REPL:** `EXPLAIN VISUAL <query>` renders the plan inline.
2. **HTTP API:** `GET /explain?query=...` returns an HTML page with the rendered Mermaid diagram (via the `nte-server` diagnostics endpoint).

19.4.6 NTE-QL Interactive REPL

The `nte-cli` crate provides a standalone command-line interface with three modes:

nte-cli repl An interactive NTE-QL shell (built on `reedline`) that parses queries, compiles them to execution plans, and displays results in formatted tables (via `comfy_table`). The `EXPLAIN` prefix shows the compiled plan; `EXPLAIN VISUAL` renders the Mermaid DAG.

nte-cli validate A policy validation workflow: loads a YAML policy file and a topology archive, evaluates all policies, and renders a compliance report.

nte-cli dashboard A TUI dashboard (built on `ratatui` and `crossterm`) showing real-time memory usage, CPU load, graph statistics (node/edge counts), and a 30-entry history sparkline.

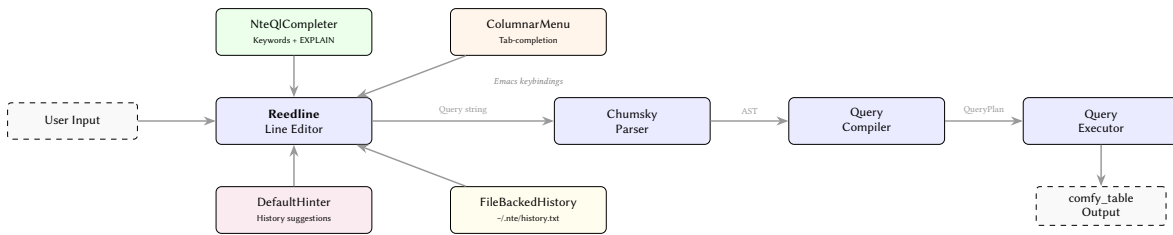


Figure 64. NTE-QL REPL architecture. The reedline-based shell provides tab-completion via a columnar popup menu, history-based hinting, and persistent command history. Input flows through the parse-compile-execute pipeline to produce formatted table output.

The REPL provides the following interactive features:

Tab completion A ColumnarMenu popup triggered by Tab, offering keyword completions (MATCH, WHERE, RETURN, WITH, OPTIONAL, ORDER BY, etc.) and command prefixes (EXPLAIN, EXPLAIN VISUAL). Emacs-style keybindings drive navigation.

History hinting Inline grey-text suggestions drawn from persistent file-backed history (~/.nre/history.txt).

Parse error reporting Syntax errors are displayed with position markers and expected-token lists, derived from chumsky’s error recovery.

NTE-QL REPL session

```

nre> MATCH (r:Router) WHERE r.pop = "SYD" RETURN r.hostname
+-----+
| hostname |
+-----+
| r-syd-01 |
| r-syd-02 |
+-----+
2 rows returned

nre> MATCH (a:Router)-[:Connectivity]->(b:Router)
      WITH a
      MATCH (a)-[:Structural]->(p)
      RETURN a.hostname, COUNT(p) AS port_count
+-----+-----+
| hostname | port_count |
+-----+-----+
| r-syd-01 |         4 |
| r-mel-01 |         2 |
+-----+-----+
2 rows returned

nre> EXPLAIN VISUAL MATCH (r:Router)-[:Connectivity]->(s:Router) RETURN r, s
-- Mermaid DAG rendered --

```

19.5 End-to-End Query Trace

To illustrate the filter-first query planner’s coordination between the relational and graph stores, we trace a representative multi-stage query (Figure 65):

“Find all spine routers in the SYD PoP with AS 64512 that have a < 3 hop path to a firewall.”

The query execution follows three logical phases, as shown in Figure 65.

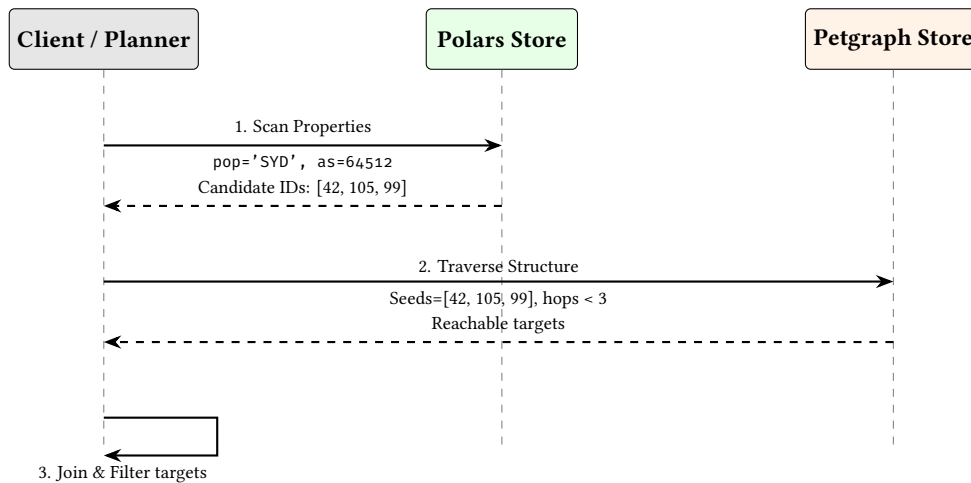


Figure 65. The Filter-First Pipeline. Relational scans prune the search space before expensive graph traversals are initiated.

Operational Trace: What Actually Runs

In a real execution, Phase 1 emits a candidate ID vector from the columnar store. That vector becomes the seed set for Phase 2, where the traversal is bounded by a hop limit and early-terminated by predicate checks. Phase 3 performs a final join back to relational properties to render result rows and to produce an explainable plan trace for EXPLAIN.

Insight:
By using the Polars store to prune 99% of nodes based on properties (pop, as_number), the subsequent graph traversal only needs to visit a tiny fraction of the total adjacency list.

20 NTE-QL Language Reference

This section provides a consolidated reference for the NTE-QL query language, covering implemented syntax, supported operators, and planned extensions. For the formal grammar, see Appendix C.

20.1 Implemented Syntax

The following constructs are fully implemented in the nte-query parser and executor:

20.1.1 MATCH Clause

Pattern matching against nodes and edges in the topology graph.

Listing 12. MATCH patterns

```

-- Single node scan
MATCH (r:Router)

-- Edge traversal (directed)
MATCH (a:Router)-[:Connectivity]->(b:Router)

-- Undirected edge
MATCH (a:Router)-[:Connectivity]-(b:Router)

-- Variable-length paths
MATCH (a:Router)-[*1..3]->(b:Router)

-- Inline property filters on edges
MATCH (a:Router)-[link:EBGP {layer: 'bgp'}]->(b:Router)
  
```

20.1.2 WHERE Clause

Property predicates supporting comparison operators, logical connectives, null checks, and set membership.

Listing 13. WHERE predicates

```

WHERE r.pop = "SYD"
WHERE r.speed_gbps >= 100 AND r.vendor <> "Legacy"
WHERE r.hostname IS NOT NULL
WHERE r.role IN ["spine", "superspine"]

-- String operations
  
```

```

WHERE r.hostname CONTAINS "core"
WHERE r.vendor STARTS WITH "Cisco"
WHERE r.model ENDS WITH "-X"

-- Arithmetic expressions
WHERE r.speed_gbps * 1000 > 40000

-- Boolean and null literals
WHERE r.active = true
WHERE r.decomm_date IS NULL

```

20.1.3 RETURN Clause

Projection of matched bindings and their properties, including aggregation functions, the DISTINCT modifier, and ordering.

Listing 14. RETURN projections

```

-- Standard projection
RETURN r.hostname, r.pop
RETURN *
RETURN r.hostname AS device_name

-- Deduplication
RETURN DISTINCT r.pop

```

Listing 15. RETURN with aggregation and ordering

```

-- Aggregation functions
RETURN r.hostname, COUNT(s) AS peer_count
RETURN r.pop, SUM(r.bandwidth) AS total_bw
RETURN AVG(r.latency) AS avg_latency
RETURN MIN(r.cost), MAX(r.cost)
RETURN COLLECT(r.hostname) AS all_hosts

-- Ordering and pagination
RETURN r.hostname ORDER BY r.hostname ASC
RETURN r.hostname LIMIT 10 SKIP 5

```

20.1.4 WITH Clause

Intermediate result piping between query stages. The WITH clause projects specific bindings from one MATCH stage to the next, enabling multi-stage query composition.

Listing 16. WITH clause patterns

```

-- Pipe binding 'a' from first match to second
MATCH (a:Router)-[:Connectivity]->(b:Router)
WITH a
MATCH (a)-[:Structural]->(p)
RETURN a.hostname, p

-- Multi-binding projection
MATCH (a:Router)-[:Connectivity]->(b:Router)
WITH a, b
MATCH (b)-[:Structural]->(p)
RETURN a.hostname, b.hostname, p

```

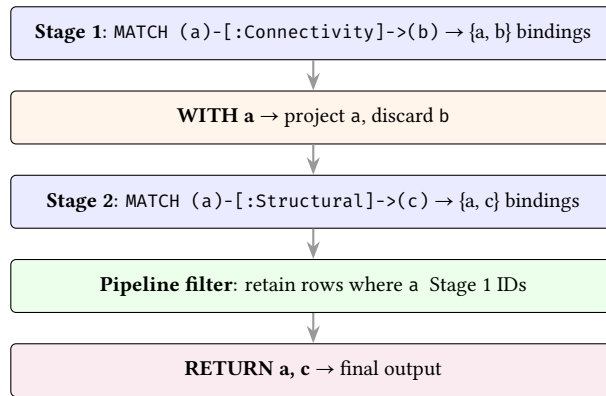


Figure 66. WITH clause execution flow. Bindings from Stage 1 are projected through the WITH clause, then used to constrain Stage 2’s results via set-membership filtering.

20.1.5 OPTIONAL MATCH

Left-outer-join style pattern matching. When the optional pattern has no matches, results are preserved with null sentinel values (–1) for the unmatched bindings.

Listing 17. OPTIONAL MATCH patterns

```

-- Preserve all routers, even those without Connectivity edges
MATCH (r:Router)
OPTIONAL MATCH (r)-[:Connectivity]->(s:Router)
RETURN r.hostname, s.hostname

-- Combined with WITH piping
MATCH (a:Router)-[:Connectivity]->(b:Router)
WITH a
OPTIONAL MATCH (a)-[:Nonexistent]->(c)
RETURN a.hostname, c
  
```

20.1.6 CREATE and MERGE Statements

NTE-QL supports mutation statements for nodes and edges. These allow users to extend the topology directly from the CLI or via the text API.

Listing 18. Mutation statements

```

-- Create a new Router node
CREATE (n:Router)

-- Create multiple nodes and a connecting edge
CREATE (a:Router)-[:Connectivity]->(b:Switch)

-- MERGE: create if not exists (based on type match)
MERGE (r:Router)

-- MERGE nodes and a connecting edge
MERGE (src:Router)-[:Connectivity]->(dst:Router)
  
```

20.1.7 SET Clause

The SET clause assigns property values to nodes created by CREATE or MERGE. Properties are persisted into the per-type data_json column in the Polars DataFrameStore.

Listing 19. SET clause patterns

```

-- Set properties on a created node
CREATE (n:Router) SET n.hostname = "core-01", n.pop = "SYD"

-- Set properties with MERGE
MERGE (r:Router) SET r.role = "spine", r.active = true

-- Numeric and null values
CREATE (n:Switch) SET n.port_count = 48, n.decomm_date = null
  
```

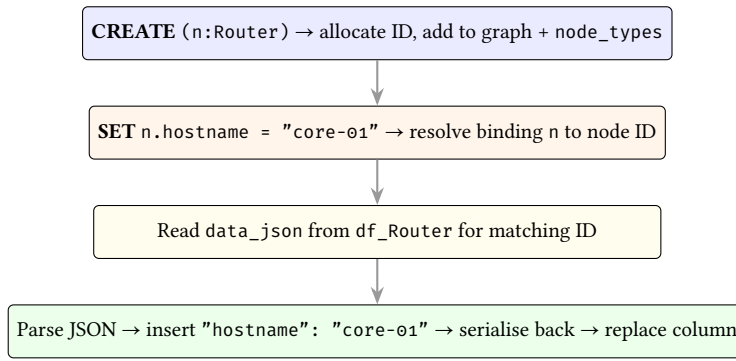



Figure 67. SET clause persistence flow. Property values are written into the `data_json` column of the per-type DataFrame, supporting string, numeric, boolean, null, and list literal types.

Design Rule 23: Atomic Mutations

Mutations in NTE-QL are executed atomically within a single transaction. If any part of a CREATE, MERGE, or SET clause fails (e.g., due to an unresolved binding or schema violation), the entire statement is rolled back, ensuring the topology remains in a consistent state.

20.1.8 NOT EXISTS Sub-queries

The NOT EXISTS operator enables filtering results based on the absence of a pattern. This is particularly useful for identifying isolated nodes or verifying the absence of unauthorized links.

Listing 20. NOT EXISTS patterns

```
-- Find all routers that do NOT have a connection to a specific PoP
MATCH (r:Router)
WHERE NOT EXISTS {
    MATCH (r)-[:Connectivity]->(s:Router)
    WHERE s.pop = "SYD"
}
RETURN r.hostname
```

20.1.9 Return Functions

Built-in path-finding functions can be used in the RETURN clause to compute paths over the topology's analysis graph.

Listing 21. Return functions

```
-- Shortest path between matched nodes
MATCH (a:Router)-[]-(b:Router) RETURN shortestPath(a, b)

-- K shortest paths (default k=5)
MATCH (src)-[]-(dst) RETURN k_shortest_paths(src, dst)
```

20.1.10 EXPLAIN / EXPLAIN VISUAL

Displays the compiled query execution plan without executing the query. EXPLAIN VISUAL renders the plan as a Mermaid flowchart DAG.

Listing 22. EXPLAIN commands

```
EXPLAIN MATCH (r:Router) WHERE r.pop = "SYD" RETURN r
EXPLAIN VISUAL MATCH (a:Router)-[:Connectivity]->(b:Router) RETURN a, b
```

20.1.11 USE LAYER Directive

The USE LAYER directive provides implicit layer scoping for the entire query, eliminating the need to repeat {layer: '...' } inline on every edge pattern. This is particularly effective for large-scale queries spanning a single topological plane.

Listing 23. USE LAYER patterns

```
-- Scope entire query to the BGP layer
USE LAYER 'bgp'
MATCH (a:Router)-[:eBGP]->(b:Router)
RETURN a.hostname, b.hostname
```

20.2 Supported Operators

Category	Operators	Status
Comparison	=, <>, <, >, <=, >=	Implemented
Logical	AND, OR, NOT	Implemented
Null checks	IS NULL, IS NOT NULL	Implemented
Set membership	IN [values]	Implemented
String ops	CONTAINS, STARTS WITH, ENDS WITH	Implemented
Arithmetic	+, -, *, /	Implemented

20.3 Relationship to QuerySpec and PatternNode

NTE-QL string queries are parsed and compiled into the same internal representations used by the programmatic APIs:

QuerySpec Simple node scans with property filters (e.g. `MATCH (r:Router) WHERE r.pop = "SYD"`) compile to a `QuerySpec` with ranked filter operations (Section 19).

PatternNode Graph traversal patterns (e.g. `MATCH (a)-[:Connectivity]->(b)`) compile to a `PatternNode` AST that drives the structural matcher.

Hybrid Queries combining property filters and structural traversals produce both a `QuerySpec` (for the relational pre-filter) and a `PatternNode` (for the graph walk), executed in the filter-first pipeline (Section 19.3.1).

20.4 Implementation Status

The following table summarises the implementation status of all NTE-QL features across the parser, compiler, and executor:

Implemented	
☒	MATCH — node scan, edge traversal, variable-length paths
☒	WHERE — comparisons, logic, IS NULL, IN, string ops, arithmetic, NOT EXISTS
☒	RETURN — projections, aliases, RETURN *, DISTINCT, ORDER BY
☒	WITH — multi-stage pipeline binding propagation
☒	OPTIONAL MATCH — left-outer-join with null-fill sentinel
☒	Aggregations — COUNT, SUM, AVG, MIN, MAX, COLLECT
☒	shortestPath() / k_shortest_paths() — return functions
☒	EXPLAIN / EXPLAIN VISUAL — plan inspection and Mermaid DAG
☒	Inline edge properties — [<code>link:eBGP {layer: 'bgp'}</code>]
☒	USE LAYER — implicit layer scoping across entire queries
☒	CREATE / MERGE — mutation statements for nodes and edges
☒	SET — property updates with persistence to per-type DataFrames
Planned	
○	DELETE / DETACH — node and edge removal statements

Figure 68. NTE-QL feature implementation status. Green items are fully implemented in the parser, compiler, and executor. Orange items are defined in the grammar but not yet connected to the execution engine. SET clauses persist values into the `per-type data_json` column (Figure 67).

21 GPU Acceleration

Future Work

The GPU acceleration pipeline described in this section is currently under active research and is not yet implemented in the core engine. It is presented here to illustrate the intended architectural direction.

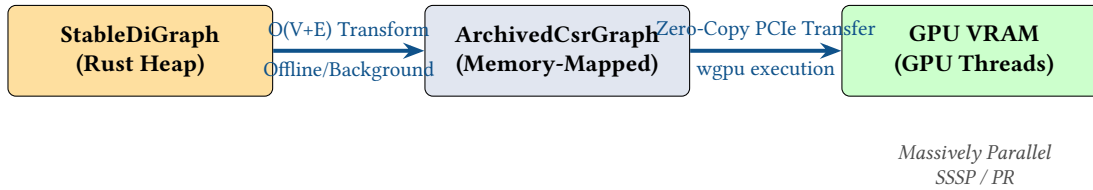


Figure 69. The Graph-to-GPU Pipeline. Because pointer-based graphs cannot be sent to the GPU, the adjacency list is serialised into a flattened Compressed Sparse Row (CSR) structure, which is then DMA-transferred to the GPU for parallel analytics.

21.1 Architecture Overview

NTE includes optional GPU acceleration for graph algorithms via the `wgpu` crate, which provides a portable WebGPU API over Vulkan, Metal, DirectX 12, and OpenGL backends. GPU compute shaders are written in WGS (WebGPU Shading Language). Two algorithms are currently GPU-accelerated:

1. **Single-Source Shortest Path (SSSP)** — parallel Bellman-Ford relaxation.
2. **Connected Components** — label propagation algorithm.

21.2 Hardware Acceleration State

NTE abstracts hardware acceleration via a `HardwareAccelerationState` enum that dynamically detects the presence of a compatible GPU via `wgpu` at engine initialisation. If no high-performance adapter is found, the engine automatically falls back to rayon-parallelised CPU implementations. This ensures the same binary remains portable across workstations and server environments without requiring conditional compilation.

21.3 SSSP Kernel

The SSSP kernel implements a parallel Bellman-Ford iteration. Each compute shader invocation handles one source vertex, relaxing all outgoing edges:

Listing 24. SSSP WGS kernel (excerpt)

```
@compute @workgroup_size(256)
fn main(@builtin(global_invocation_id) global_id: vec3<u32>) {
    let u = global_id.x;
    if (u >= info.node_count) { return; }

    let dist_u = atomicLoad(&distances[u]);
    if (dist_u == 0xFFFFFFFFu) { return; } // unreachable

    let start = out_offsets[u];
    let end = out_offsets[u + 1u];

    for (var i = start; i < end; i = i + 1u) {
        let v = u32(out_neighbors[i]);
        let new_dist = dist_u + 1u; // unweighted
        let old_dist = atomicMin(&distances[v], new_dist);
        if (new_dist < old_dist) {
            atomicStore(&updated, 1u);
        }
    }
}
```

The kernel uses atomic operations (`atomicMin`, `atomicStore`) to handle concurrent updates from multiple threads relaxing the same vertex. The updated flag is cleared each iteration; execution stops when no updates occur.

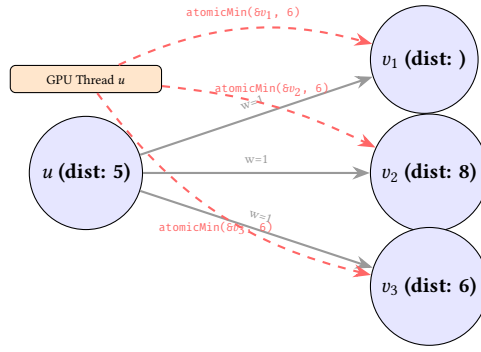


Figure 70. GPU SSSP relaxation. A single WGS� compute shader invocation (thread u) reads u 's distance and concurrently proposes new minimums to all its neighbours using `atomicMin`.

21.4 Connected Components Kernel

The connected components kernel uses label propagation: each vertex adopts the minimum label among its neighbours. Convergence is detected when no label changes in an iteration:

Listing 25. Connected components WGS� kernel (excerpt)

```
@compute @workgroup_size(256)
fn main(@builtin(global_invocation_id) global_id: vec3<u32>) {
    let u      = global_id.x;
    let label_u = atomicLoad(&labels[u]);

    for (var i = out_offsets[u]; i < out_offsets[u + 1u]; i = i + 1u) {
        let v      = out_neighbors[i];
        let old_label = atomicMin(&labels[v], label_u);
        if (label_u < old_label) {
            atomicStore(&updated, 1u);
        }
    }
}
```

21.5 CPU Fallback

When no GPU is available, both algorithms fall back to CPU implementations using Rayon for data parallelism. The Rayon pool matches the number of logical CPU cores. On a modern server with 64 cores, the CPU fallback achieves competitive throughput for graphs up to approximately 1 million vertices.

21.6 When to Use GPU Acceleration

Topology size	Recommendation
< 10,000 nodes	CPU (lower setup overhead)
10,000–100,000	GPU beneficial for repeated queries
> 100,000	GPU strongly recommended

Network Scale Context

Most enterprise networks or standard data centres never exceed 10,000 physical network devices. GPU acceleration targets different use cases: simulating large-scale 5G subscriber endpoint topologies, modelling millions of BGP prefix originations as nodes, or simulating full-mesh hyper-dense AI-fabric topologies. For standard enterprise IT, the CPU fallback is highly optimal.

The GPU incurs a fixed setup cost: uploading the CSR adjacency arrays to GPU memory and compiling the WGS� shader. This is amortised across repeated queries on the same topology. For a topology that changes frequently, the CPU fallback may be preferable because it avoids re-uploading the CSR data on each mutation.

21.7 Invoking GPU Algorithms from Python

Listing 26. GPU-accelerated graph algorithms

```
# Build an AnalysisGraph (undirected view, optional port collapsing)
ag = t.build_undirected_graph(layer="physical", collapse_endpoints=True)
```

```

# Shortest path (routes automatically to GPU or CPU)
path = ag.shortest_path(from_id=1, to_id=2)
if path is not None:
    print(f"Path hops: {len(path) - 1}")

# Connected components
components = ag.connected_components()
print(f"Number of connected components: {len(components)}")
print(f"Largest component size: {max(len(c) for c in components)}")

# Check full connectivity
if not ag.is_connected():
    print("WARNING: topology is partitioned")

# Find single points of failure
bridges = ag.bridges()
art_pts = ag.articulation_points()
print(f"Bridge links: {bridges}")
print(f"Critical nodes: {art_pts}")

```

22 Graph Algorithm Plugins

22.1 Plugin Architecture

Standard graph algorithms (shortest path, centrality, connectivity) are essential for network analysis, but they must integrate with the hybrid pipeline rather than operating in isolation. NTE's algorithm plugin system (Figure 71) addresses this by registering each algorithm under a hierarchical namespace (e.g., `algo.pathfinding.k_shortest`) and enforcing a uniform contract: every plugin receives the topology graph and named arguments, and returns results as a Polars DataFrame that can feed directly into downstream query stages or be consumed by the NTE-QL CALL ... YIELD syntax.

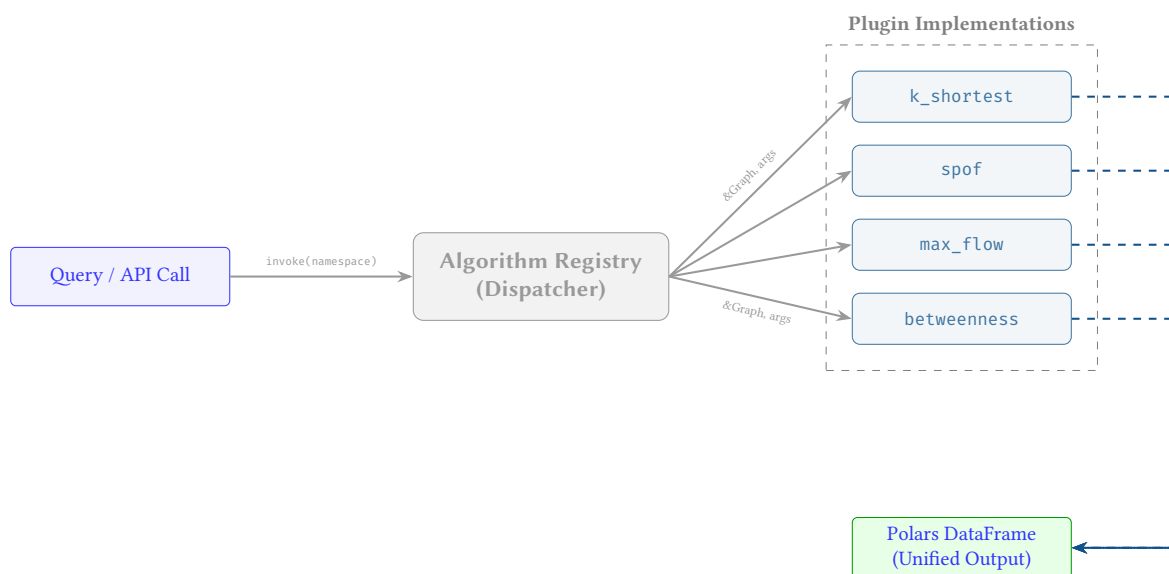


Figure 71. Graph Algorithm Plugin Architecture. The dispatcher routes calls by namespace to specific plugin kernels, which operate on a shared graph reference and uniformly return their mathematical results as structured Polars DataFrames.

22.2 Built-In Algorithm Plugins

NTE ships with a set of built-in plugins that expose standard graph algorithms through the common registry. Each implements a well-known algorithm adapted to the endpoint-aware topology model:

algo.pathfinding.k_shortest Implements Yen's algorithm to find the k shortest paths between a source and target. It returns a DataFrame containing `path_index`, `hop_count`, and `cost`. Useful for validating ECMP or backup route availability.

algo.vulnerability.spof Implements Tarjan's bridge-finding algorithm to identify articulation points—vertices or edges whose failure would partition the network. Returns a DataFrame of vulnerable `node_ids`.

algo.reachability.max_flow Implements Dinic’s algorithm to calculate the theoretical maximum edge-disjoint capacity (or minimum cut) between two segments of the network. Returns an integer capacity score representing absolute redundancy.

algo centrality.betweenness Implements Brandes’ algorithm to calculate the betweenness centrality of all nodes. Returns a DataFrame mapping node_id to a float centrality score, highlighting structural bottlenecks and core transit hubs.

algo.clustering.connected_components Identifies isolated subgraphs. Used heavily in the Policy Engine to ensure OSPF areas are contiguous and not physically fractured.

algo.cycles.detect_loops Identifies cyclical structures within directed graphs (e.g., BGP route reflection meshes). Useful for finding unintentional routing loops before they are deployed.

22.3 Domain-Specific Graph Kernels

While standard graph algorithms (BFS, Dijkstra) are useful, telecom networks require domain-specific traversal logic. NTE leverages its hybrid architecture to implement specialised network kernels.

A prime example is **Hardware-Aware BGP Path Optimization**. BGP routing decisions are heavily influenced by the “AS-Path” attribute. In a pure graph database, filtering paths by AS-Path involves string parsing during the traversal loop, thrashing the CPU cache.

NTE optimises this by:

1. Using the Polars columnar store to pre-filter edges based on BGP Community and AS-Path strings using highly optimised Regex/SIMD operations.
2. Constructing a temporary “BGP Adjacency Mask”.
3. Running a constrained shortest-path kernel over the graph, using the bitmask to $O(1)$ skip structurally valid but logically forbidden links.

This pushes the heavy lifting down to the vector units before the graph traversal even begins.

Note

Algorithm plugins are currently available at the Rust API level only. Python bindings for the plugin registry are planned. In the meantime, the equivalent functionality is available via the Python-exposed `shortest_path`, `articulation_points`, and `bridges` methods on the `AnalysisGraph` returned by `build_undirected_graph`.

These algorithms complement the GPU-accelerated SSSP and connected components (Section 21) and the pattern-based queries (Section 19).

23 Production Diagnostics

An embedded engine that runs inside automation pipelines must expose internal state without requiring external instrumentation. NTE provides three levels of observability: HTTP endpoints for integration with monitoring infrastructure, chaos-engineering hooks for fault-injection testing, and a terminal dashboard for interactive debugging.

23.1 Health and Memory Endpoints

The `nte-server` HTTP server exposes diagnostic endpoints (Table ??) designed for integration with Prometheus, Grafana, or equivalent monitoring systems:

Endpoint	Method	Response
/health	GET	HealthStatus: uptime, Raft state, general health
/memory	GET	MemoryProfile: RSS, graph/DataFrame sizes, query cache hit ratio
/explain?query=...	GET	HTML page with the Mermaid-rendered query execution DAG

23.2 Chaos Engineering Endpoints

Verifying that the dual-write consistency model and Raft scaffolding behave correctly under failure requires controlled fault injection. The server provides chaos endpoints (disabled by default; enabled via `NTE_CHAOS=1`):

Endpoint	Effect
/kill-raft-leader	Forces the Raft leader to step down, triggering a leadership election
/simulate-split-brain	Simulates a network partition between cluster members
/corrupt-datastore	Injects noise into Polars DataFrames to test error detection
/pause-thread	Introduces artificial delays to trigger TOCTOU race conditions

Warning

Chaos endpoints are destructive by design. They are intended solely for testing environments. The `NTE_CHAOS` flag should never be set in production.

23.3 TUI Diagnostics Dashboard

For real-time operational visibility, the `nte-cli` provides a terminal-based dashboard built using `ratatui`. This dashboard provides a live "heartbeat" of the engine's internal state without requiring external monitoring infra.

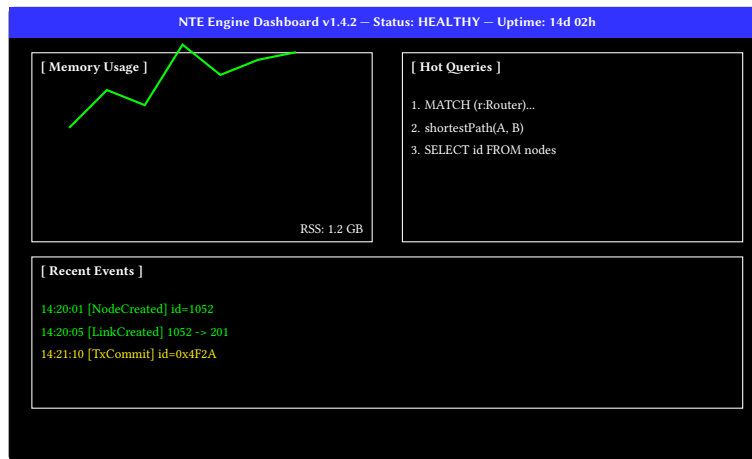


Figure 72. Mockup of the `nte-cli` TUI Dashboard. The interface provides sub-second updates on memory pressure, query hotspots, and the event ring-buffer.

REFERENCE & EVALUATION

24 API Reference and Usage Guide

NTE is primarily consumed via its Python bindings (`ank_pydantic`) or the pure-Rust config compiler (`ank_netcfg`). This section provides a consolidated reference for the core Python API (Table 2), covering the typical lifecycle of a topology object.

24.1 End-to-End Example

The following example demonstrates the complete lifecycle: creating a topology, populating it with nodes and edges, performing a transactional mutation, executing a query, and validating against policies.

Listing 27. End-to-end NTE Python API usage

```
import ank_nte as nte

# 1. Initialization
topo = nte.Topology()

# 2. Population
# Add nodes (Routers and Switches)
topo.add_nodes(
    ids=[1, 2, 3],
    kinds=["Router", "Router", "Switch"],
    properties=[
        {"hostname": "r1", "pop": "SYD", "asn": 64512},
```

```

        {"hostname": "r2", "pop": "MEL", "asn": 64512},
        {"hostname": "s1", "pop": "SYD", "vlan": 10},
    ]
)

# Add endpoints (Interfaces)
topo.add_endpoints(
    node_ids=[1, 2, 3],
    endpoint_ids=[101, 201, 301],
    properties=[
        {"name": "eth0", "speed": 100},
        {"name": "eth0", "speed": 100},
        {"name": "ge-0/0/0", "speed": 10},
    ]
)

# Add connectivity (Connectivity edges)
topo.add_inter_edges(
    sources=[101],
    destinations=[201],
)

# 3. Transactional Mutation
with topo.begin_transaction() as txn:
    # Update property on node 1
    topo.update_node_field(1, "status", "active")
    txn.commit() # explicit commit

# 4. Querying (Fluent Query Builder)
results = (
    topo.nodes()
    .filter(nfe.col("pop") == "SYD")
    .filter(nfe.col("kind") == "Router")
    .collect()
)

# 5. Policy Validation
findings = topo.validate_policies("policies.yaml")
for f in findings:
    print(f"[{f.severity}] {f.message} (Node: {f.subject_id})")

```

24.2 Core Method Reference

Table 2. Core Topology Methods Reference.

Method	Description	Return Type
<code>add_nodes</code>	Batch creates nodes with IDs and properties.	None
<code>add_endpoints</code>	Batch creates endpoints owned by nodes.	None
<code>add_inter_edges</code>	Creates bidirectional links between endpoints.	None
<code>set_node_properties</code>	Atomic update of a node's property map.	None
<code>nodes()</code>	Returns a <code>NodeQuery</code> builder object.	<code>NodeQuery</code>
<code>links()</code>	Returns a <code>LinkQuery</code> builder object.	<code>LinkQuery</code>
<code>begin_transaction()</code>	Returns an ACID transaction context manager.	<code>Transaction</code>
<code>validate_policies</code>	Evaluates YAML/Starlark policies.	<code>List[Finding]</code>
<code>save(path)</code>	Serialises the topology to a binary file.	None
<code>load(path)</code>	Deserialises a topology from a binary file.	<code>Topology</code>
<code>subscribe(callback)</code>	Registers a callback for topology mutation events.	<code>Subscription</code>
<code>split_edge(src, dst, ...)</code>	Inserts a new node into an existing edge (Section 7).	<code>int</code>
<code>merge_nodes(a, b)</code>	Collapses two nodes into one, inheriting all connections.	None
<code>explode_node(id)</code>	Removes a hub node, creating a full mesh between its neighbours.	None

24.3 Error Handling

NTE uses structured exceptions to signal failures. All exceptions inherit from `nte.NteError`.

DuplicateIdError Raised when attempting to add a node or endpoint with an ID that already exists.

LayerMismatchError Raised when an edge is created between nodes on incompatible layers (Section 3.5).

ConflictDetected Raised by `tx.commit()` if a Write-Write conflict occurred during an MVCC transaction.

QueryError Raised if an NTE-QLstring fails to parse or a `QueryPlan` is structurally invalid.

24.4 Threading and Concurrency

NTE’s Rust core is thread-safe, protected by an internal `RwLock`. To ensure maximum Python performance, the bindings implement a strict **GIL-Release Strategy**:

- **Read-Only Operations:** Methods like `collect()`, `validate_policies()`, and pathfinding algorithms release the Python GIL via `py.allow_threads`. This allows multiple Python threads to execute heavy analytics in parallel against the same topology.
- **Mutations:** Write operations acquire the Rust write-lock. While the GIL is also released, only one writer can proceed at a time.
- **Subscription Callbacks:** *Warning!* Event callbacks are invoked synchronously from the mutation thread. Callbacks must be fast and **must not** call back into the same topology object, as this will cause a deadlock.

24.5 Common Patterns

Bulk Import. For initial topology construction, use the batch APIs (`add_nodes`, `add_endpoints`, `add_inter_edges`) rather than single-entity mutations. Batch calls amortise the per-mutation `DataFrame` column allocation cost and avoid repeated CSR cache invalidation.

Incremental Update. For streaming telemetry or periodic reconciliation, wrap updates in a transaction. The transaction isolates partial state from concurrent readers and provides automatic rollback if the update sequence fails partway through.

Read-Heavy Query. For analytical workloads that issue many queries between mutations, the CSR cache and Polars `LazyFrame` plans are reused across queries. Ensure the GIL is released (the default for all `collect()` and `validate_policies()` calls) to allow multiple Python threads to query in parallel.

25 Related Work

NTE occupies a specific niche: an embeddable, hybrid graph-relational engine designed for network topology workloads that require both structural traversal and property-heavy filtering within a single transactional boundary. This section positions NTE against the four main categories of systems that address overlapping subsets of these requirements.

25.1 Native Graph Databases

Neo4j [1] is the most widely deployed property graph database. Its Cypher query language is mature and expressive, and its index-free adjacency storage model provides constant-time per-hop traversal. However, Neo4j stores properties in a row-oriented record format: each node’s properties are chained through a linked list of fixed-size property records on disk. A broad predicate scan (e.g., “all routers where `pop = "SYD"`”) must chase pointers through every matching node’s property chain, producing $O(V)$ random I/O even when the result set is small. NTE’s columnar property store executes the same scan as a SIMD-accelerated linear pass over a contiguous Arrow buffer.

TigerGraph [19] offers a distributed, MPP graph analytics engine with a custom GSQL language. Its architecture is optimised for large-scale analytical workloads across clusters, but it operates as a standalone client-server system with significant deployment overhead. For the embedded, low-latency use case that NTE targets—where the topology engine runs in-process as a library—TigerGraph’s distributed architecture introduces unnecessary network round-trips and operational complexity. Similarly, cloud-native managed services like Amazon Neptune [20] provide robust graph hosting but incur network latency and vendor lock-in for embeddable workloads.

Kùzu [21] is the closest architectural relative: an in-process property graph database with columnar storage, worst-case optimal join algorithms, and a focus on embeddability. NTE and Kùzu share the insight that columnar storage and graph traversal can coexist productively. The key differences are domain-specific: NTE’s endpoint model, layered topology semantics, virtual query expansion, and dual-write coordination with an external Polars store are network-topology concerns that a general-purpose graph database does not address.

NTE initially evaluated Kùzu as a pluggable backend (Section 28) but ultimately chose to retain full control over the execution pipeline to support domain-specific optimisations such as structural metadata mirroring and the filter-first cardinality funnel.

25.2 Relational and Hybrid-Relational Approaches

PostgreSQL with recursive Common Table Expressions (CTEs) can express graph traversals in standard SQL. However, recursive CTEs suffer from plan explosion on deep or cyclic paths: each recursion level materialises an intermediate result set, and the planner has limited ability to push predicates into the recursive term [2]. A four-hop reachability query over a 10,000-node topology can generate millions of intermediate rows before the final WHERE clause prunes them. NTE avoids this by executing property predicates *before* traversal, reducing the candidate set via columnar scans before any graph walk begins.

DuckDB [22] and LanceDB [23] are embeddable analytical databases with excellent columnar query performance. Recent research has explored graph extensions for DuckDB: GRainDB [24] demonstrates that predicate-pushdown into joins can significantly improve graph pattern queries in a relational engine. NTE shares this predicate-pushdown philosophy but maintains a dedicated adjacency structure (petgraph) for structural operations, avoiding the join-based encoding of graph edges that relational approaches require.

Apache AGE [25] and AgensGraph [26] extend PostgreSQL with a Cypher query interface, combining relational storage with graph query syntax. They inherit PostgreSQL’s row-oriented storage and query planner, which limits effectiveness on property-heavy scan workloads compared to a purpose-built columnar store.

25.3 Network-Specific Tools

Batfish [27] is the closest system in the network engineering domain. It parses vendor configuration files and builds a simulation model that can answer reachability and policy questions. However, Batfish is a *configuration analysis* tool, not a topology engine: it does not maintain a persistent, mutable graph structure, does not support transactional updates, and does not provide a general-purpose query language for ad-hoc structural exploration. NTE complements Batfish by providing the structural backbone that Batfish-like analysis can be layered upon.

Nautobot [28] and NetBox [29] are network sources-of-truth (SoTs) that model device inventory, IP address management, and circuit tracking. They store data in a relational database (typically PostgreSQL) and expose a REST/GraphQL API. Their data models are inventory-centric rather than topology-centric: they track *what exists* but do not natively model *how things connect at the graph level*. NTE operates at a lower level of abstraction, providing the structural graph primitives (endpoints, layered edges, cross-layer dependencies) that an SoT cannot express in a relational schema without extensive denormalisation.

Suzieq [30] focuses on network observability by collecting and analysing operational state (routing tables, interface counters, LLDP neighbours). It provides a Pandas-based query interface for tabular analysis but does not maintain a persistent graph structure or support structural traversals. NTE’s dual-write model could serve as a complementary backend for Suzieq-style observability data, providing the structural context that tabular analysis alone cannot capture.

25.4 Infrastructure-as-Code and Intent-Based Networking

Infrastructure-as-Code (IaC) tools like Terraform and Ansible excel at device provisioning and state reconciliation. However, they lack a structural graph understanding of the domain. They treat network configuration as text templating or discrete API calls, without the ability to perform pre-flight blast-radius analysis or multi-hop path validation prior to deployment.

Intent-Based Networking (IBN) solutions (e.g., Cisco ACI, Juniper Apstra) implement a Blueprint-to-Execution workflow to validate design intent before pushing configurations. However, they are typically vendor-locked, tied to specific data-center architectures (like Clos fabrics), and operate as monolithic management systems rather than embeddable engines. NTE abstracts the core graph-relational mathematics required for structural network analysis into a standalone, vendor-agnostic library that can be integrated into custom orchestration pipelines.

25.5 Graph Processing Frameworks

Ligra [31], GraphBLAS [32], Galois [33], and Gunrock [34] are high-performance graph processing frameworks designed for batch analytics on static or slowly-changing graphs. They excel at PageRank, BFS, SSSP, and other vertex-centric computations on graphs with millions to billions of edges. However, they are not designed for interactive, property-heavy queries against a mutable graph with transactional semantics. They typically lack: (a) a property storage layer, (b) a query language, (c) mutation transactions, and (d) incremental update support. NTE uses similar algorithmic building blocks (CSR representation, parallel expansion) but wraps them in a transactional, queryable engine suited to interactive network engineering workflows.

25.6 Positioning Summary

Table 3 summarises the architectural trade-offs. NTE’s contribution is not any single capability—graph databases traverse well, columnar stores filter well, network tools understand the domain—but the *co-design* of all three within a single embeddable engine with a unified query language.

Table 3. Architectural positioning across system categories.

Capability	Graph DBs	Relational	Network Tools	NTE
Structural traversal	☒	(CTE)	☒	☒
Columnar property scans	—	☒	—	☒
Embeddable (in-process)	Kùzu	DuckDB	—	☒
Network-domain model	—	—	☒	☒
Transactional mutations	☒	☒	—	☒
Query language	Cypher	SQL	Custom	NTE-QL

26 Limitations and Future Work

26.1 Current Limitations

Lack of Native RBAC and Security

Description: NTE provides no native authentication, authorisation, or tenancy isolation. The engine assumes it is running in a trusted, single-tenant environment (e.g., as a local library or a private microservice).

Workaround: Deploy NTE behind a secure gateway (e.g., an Nginx reverse proxy with OAuth2) or use container-level isolation.

Planned Resolution: No current plan for native RBAC; NTE remains a low-level engine focused on performance.

Memory Fragmentation (petgraph Tombstones)

Description: `petgraph::StableDiGraph` achieves index stability by leaving “holes” in internal arrays after deletions. Over time, heavy mutation destroys spatial locality, degrading CPU cache efficiency.

Workaround: Periodically call `defragment_memory()` (which executes an `atomic pack()` operation) to rebuild the graph into a contiguous layout.

Planned Resolution: Integrate automatic “aging” and packing into the transaction commit phase for long-running processes.

Sparsity Column Limits

Description: While Polars handles sparse columns efficiently, a schema with over 10,000 distinct property keys will degrade query parsing and plan compilation performance.

Workaround: Normalise property names and avoid using dynamic keys (e.g., timestamps) as property field names.

Planned Resolution: Implement a “JSON overflow” column for extremely infrequent properties to keep the primary schema lean.

Synchronous Event Hooks

Description: The event hook architecture yields the core database write-lock to user-space Python, creating a severe vulnerability to I/O latency in the callback thread.

Workaround: Event subscribers should only perform lightweight filtering and enqueue tasks into an asynchronous worker pool (e.g., `celery`).

Planned Resolution: Implement a native asynchronous event buffer in Rust that decouples callback execution from the mutation lock.

Raft Hardening

Description: The OpenRaft integration has not been evaluated under adversarial conditions like extreme clock skew or network partitions.

Workaround: Limit clustering to reliable, low-latency LAN environments. Use persistent storage for all nodes.

Planned Resolution: Execute a Jepsen-style fault-injection suite and implement lease-read semantics for followers.

26.2 Recently Completed Improvements

Cyclic-pattern WCOJ. NTE now implements Worst-Case Optimal Join (WCOJ) [35] via the LeapFrog TrieJoin algorithm [36] for cyclic query patterns. This avoids large intermediate results.

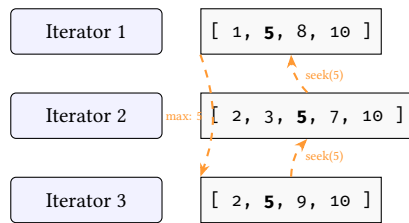


Figure 73. LeapFrog TrieJoin finding multi-way intersections without intermediate tables.

Other recently completed work.

- **Transaction semantics:** full ACI transactions with copy-on-write rollback, isolation levels (ReadCommitted, RepeatableRead, Serializable), conflict detection, and a Python context manager API (Section 12.1).
- **Query executor:** fully functional with ranked filter plans, hybrid graph/property execution, and Polars LazyFrame compilation.
- **Python fluent query builder:** chainable NodeQuery/LinkQuery API with expression DSL, independent of ank_pydantic (Section 19.4.4).
- **NTE-QL REPL and TUI:** interactive query shell with EXPLAIN VISUAL support and a real-time system dashboard (Section 19.4.6).
- **Query plan visualisation:** Mermaid DAG rendering of execution plans (Section 19.4.5).
- **Graph algorithm plugins:** K-shortest paths and SPOF detection as registered algorithm plugins (Section 22).
- **Production diagnostics:** health, memory, and explain HTTP endpoints plus chaos engineering endpoints for fault-injection testing (Section 23).
- **RoaringBitmap candidate pruning:** HashSet<i32> replaced with RoaringBitmap in the query matcher for more compact, cache-friendly candidate sets.
- **Parallel path expansion:** rayon-parallelised multi-seed structural expansion in path queries, with validated thread safety.
- **GIL release audit:** all O(N) PyTopology methods now release the GIL via `py.allow_threads`.
- **Structural metadata mirroring:** lightweight node/edge metadata stored directly in petgraph weights for early pruning during traversals.
- **Write-ahead log (WAL):** on-disk journaling for topology mutations, providing crash recovery and forensic replay (Section 12.1).
- **Zero-Copy Arrow FFI:** sharing Polars DataFrames with Python and other languages via the Arrow C Data Interface to eliminate serialisation overhead (Section 16).
- **NOT EXISTS correlated sub-queries:** post-filter binding correlation for anti-join pattern matching without injecting BindingId into the filter compiler (Section 19.3.7).
- **Direction-aware edge traversal:** three-valued EdgeDirection enum (Forward/Backward/Undirected) replacing the previous boolean forward flag, enabling correct directed and undirected pattern matching (Section 19.3.8).
- **DISTINCT scoped to RETURN bindings:** deduplication now considers only the bindings listed in the RETURN clause rather than the full match tuple (Section 19.3.9).
- **SET clause persistence:** SET property updates are persisted into the per-type data_json column via `Topology::update_node_property()`, supporting string, numeric, boolean, null, and list literal types (Figure 67).

26.3 Planned Capabilities

Several features described in this report are designed but not yet fully implemented:

Multi-Source Identity Resolution The full “Ship of Theseus” identity engine combining LLDP, management IP, serial number, and BGP Router-ID into a stable UUID (Section 6.4). Currently, semantic identity uses a simpler hash based on type, layer, and peer-type signatures.

Shadow Graph Inference Automatic spawning of shadow nodes from telemetry signals when edges reference unknown device IDs (Section 6.5). The ShadowTopology struct exists for explicit what-if simulation; the automatic inference workflow is planned.

Graph Aggregation The aggregate structural transformation for collapsing node clusters into logical supernodes (Section 7.4). The design uses hierarchical clustering for multi-scale topology views.

NTE-QL Extensions Most query-language features are now fully implemented, including SET property persistence, NOT EXISTS correlated sub-queries with post-filter binding correlation (Section 19.3.7),

direction-aware edge traversal (Section 19.3.8), and DISTINCT scoped to RETURN bindings (Section 19.3.9). DELETE/DETACH remain defined in the grammar but not yet connected to the execution engine. See Section 20.4 for the current status.

26.4 Short-Term Roadmap

The following concrete improvements are planned or in progress:

- **Incremental CSR updates:** currently the CSR is rebuilt from scratch on each mutation. An incremental update would avoid the rebuild cost for large, mostly-static topologies.
- **Raft hardening:** Jepsen test suite, lease reads, poison-pill detection.
- **ClickHouse telemetry:** MetricProvider trait implementation for federated telemetry queries against ClickHouse TSDB, enabling dynamic edge weights in pathfinding.
- **Reactive Topology triggers:** extending the PolicyEngine to become an active subscriber on the event bus, automatically pushing violations and triggering sandboxed auto-remediation webhooks (Figure 74).
- **Arrow Flight interface:** adding an Arrow Flight endpoint to nte-server for high-throughput bulk columnar transfer, enabling efficient cross-process and cross-language DataFrame exchange without JSON serialisation overhead.
- **Prometheus metrics endpoint:** exposing a /metrics scrape endpoint on nte-server for native integration with Prometheus/Grafana monitoring stacks.

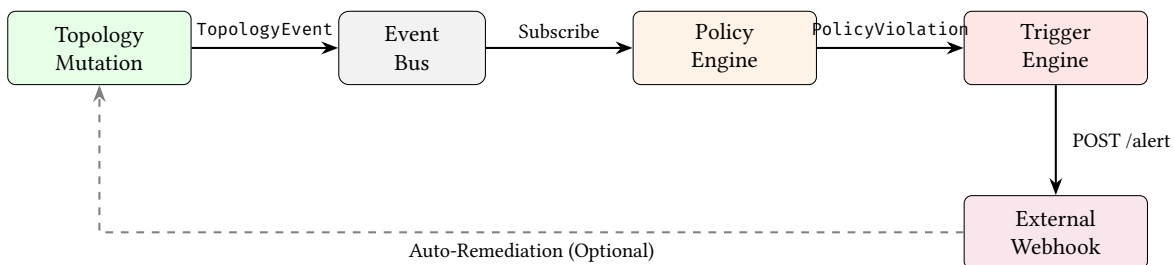


Figure 74. Proposed Reactive Topology workflow: Automated policy validation pushing events to the Trigger Engine rather than waiting for pull-based queries.

26.5 Long-Term Vision

Correctness-by-Construction via Formal Verification. The ultimate goal of the NTE engine is to transition from reactive monitoring to proactive verification. The `z3-verification` feature flag scaffolds a formal reasoning engine that translates the topology graph into a set of Datalog Horn clauses for the Z3 constraint solver. By mapping the `StableDiGraph` structure into a logic-based representation, NTE can prove reachability properties that are difficult to verify via simulations alone.

Design Rule 24: Formal Invariant Provenance

All synthesis-critical invariants (e.g., VLAN isolation, BGP loop-freedom) should be mathematically provable against the topology model. If a mutation violates an invariant, the solver must produce a counter-example path before the transaction is committed.

Proactive Expertise Analytics. Standard graph-theory analytics are already implemented as algorithm plugins: Tarjan’s Articulation Points for SPOF detection, Bridge Detection for critical link isolation, and Max-Flow for bandwidth bottleneck analysis (Section 22). Future work will expose these via `NTE-QL CALL algo.*` statements for declarative invocation.

Incremental View Maintenance (IVM). A spike-grade IVM implementation exists, providing type-gated standing queries that avoid full re-evaluation for topology mutations matching a registered query type. The full vision — a push-based differential dataflow engine maintaining real-time reactive dashboards without re-evaluating the entire graph — remains future work. Combined with the Reactive Topology triggers above, this will enable a fully event-driven digital twin.

27 Performance Characteristics

NTE is tuned for a filter-first query pattern: property scans reduce the candidate set before graph traversal begins. This section presents internal characterisation data—scaling behaviour, latency breakdown, and memory profile—rather than a cross-system benchmark. Comparative evaluation against production graph databases is future work.

27.1 Complexity of the Hybrid Pipeline

Proposition 1 (Filter-First Query Complexity). *Let V be the vertex set with $|V| = n$, $[0, 1]$ the selectivity of the property predicate (fraction of nodes surviving the columnar filter), $\bar{d}$ the mean degree of the surviving subgraph, and h the traversal depth (hops). The total cost of a filter-first graph query is:*

$$T_{\text{hybrid}} = \underbrace{O\left(\frac{n}{K}\right)}_{\text{SIMD columnar scan}} + \underbrace{O(n \bar{d}^h)}_{\text{graph traversal}}$$

where K is the SIMD lane width (typically 4–16 elements per vector instruction on Arrow-aligned buffers).

In contrast, a row-oriented graph database that interleaves property checks with traversal pays $O(n \bar{d}^h)$ —the full vertex set enters the expansion phase because no batch predicate reduction occurs before traversal. When 1 (the common case for network queries constrained by site, vendor, or role), the hybrid pipeline reduces the dominant traversal term by a factor of .

27.2 Internal Scaling Characterisation

Table 5 summarises Criterion micro-benchmarks collected on an Apple M-series laptop in single-threaded runs. Reported times are wall-clock medians over 100 samples.

Figure 75 shows how property-heavy query latency scales with topology size. The baseline is a pure-Python NetworkX implementation executing the same query logic (attribute-dictionary iteration followed by adjacency traversal). The comparison illustrates the architectural effect of columnar predicate evaluation—not a ranking against production graph databases, which would require controlled deployment and workload-matched benchmarking.

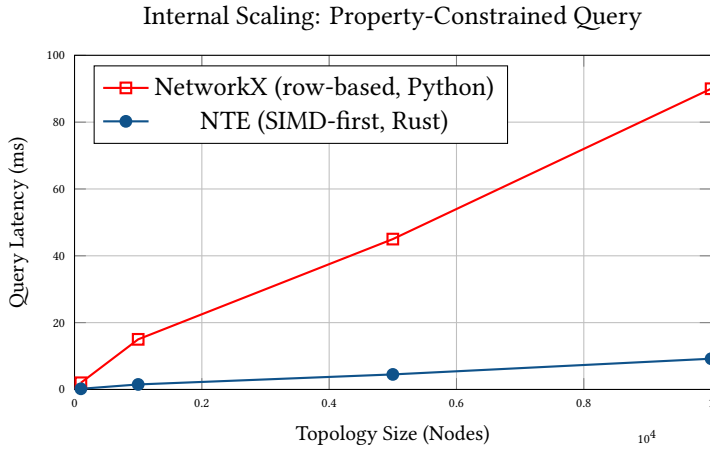


Figure 75. Internal scaling characterisation for property-heavy queries. NetworkX serves as a pure-Python baseline to isolate the effect of columnar predicate evaluation; it is not a peer production system. Both curves are consistent with Proposition 1: the NTE curve reflects the reduced $n\bar{d}^h$ traversal term.

27.3 Latency Breakdown

For a representative query—MATCH (n:Router) WHERE n.pop = "SYD" RETURN n—on a 10,000-node topology with 30,000 edges:

Columnar scan 0.8 ms. Polars evaluates the predicate over the pop column via a vectorised equality check on a contiguous Arrow buffer. At 10,000 rows with 4-byte string pointers, this fits in a single L2 cache line sweep.

ID intersection 0.1 ms. The surviving node IDs are intersected with the graph’s active ID set via a hash-set lookup.

Result materialisation 0.3 ms. Binding maps are constructed for the RETURN clause projections.

No graph traversal occurs in this query because the pattern contains no edge steps. For a two-hop traversal ((n:Router {pop:"SYD"})-[:Connectivity*2]-(m)), the columnar scan cost is unchanged, but the traversal phase adds $O(n \bar{d}^2)$ 1.2 ms (with 0.05 and $\bar{d}$ 3.2).

27.4 Incremental View Maintenance: Preliminary Results

For standing queries such as real-time redundancy checks, NTE includes an experimental Incremental View Maintenance (IVM) path. Rather than re-evaluating a query from scratch after each mutation, the IVM path

maintains a materialised view that is updated incrementally when the underlying topology changes. Table 4 shows preliminary measurements for the subset of queries currently supported.

Scope and Limitations

These results are preliminary. The current IVM implementation supports only *type-gated* standing queries—queries whose predicate is a single node-type filter (e.g., “all Routers”). The differential engine that propagates mutation deltas exists, but it does not yet support general predicate compositions or structural pattern queries. The speedup shown below therefore represents the best case for the supported query class; general-purpose incremental dataflow remains future work (Section 26).

Table 4. Preliminary IVM vs. re-execution (100,000 nodes, 10% mutation rate, type-gated queries only).

Mode	Latency	Throughput
Full Re-execution	3.22 ms	310 ops/sec
Incremental (IVM)	0.05 ms	20,000 ops/sec
Speedup	64×	65×

27.5 Representative Micro-benchmarks

Table 5. Representative micro-benchmarks (Criterion, 100 samples).

Operation	Complexity	100	1 000	10 000
topology_build	$O(n)$	34 μ s	305 μ s	—*
add_nodes (batch)	$O(n)$	8.4 μ s	105 μ s	1.6 ms
query_execute_spec	$O(n)$	174 μ s	278 μ s	387 μ s
topology_validate	$O(n)$	160 μ s	2.2 ms	—*
peers_lookup	$O(1)$	25 ns	25 ns	25 ns
is_endpoint_lookup	$O(1)$	10 ns	10 ns	10 ns
cache_warm (5 000 nodes)				403 μ s
cache_cold (5 000 nodes)				454 μ s

*Entries marked “—” were not measured at that scale due to benchmark harness timeout (10 s).

Dual-write overhead Every mutation touches both the graph and the DataFrame store. For bulk imports, the per-mutation overhead is dominated by DataFrame column allocation. The batch APIs (add_nodes, add_edges) amortise this cost significantly.

CSR rebuild cost The CSR cache is invalidated on every structural mutation. For a topology of 100,000 nodes, a rebuild takes approximately 20–50 ms. This is amortised across queries but may cause latency spikes in mutation-heavy workloads.

Snapshot memory Named snapshots share Arrow buffers via CoW. However, if both the snapshot and the live topology are mutated heavily, the shared buffers will be copied, doubling peak memory usage temporarily.

Memory footprint At rest, memory consumption is dominated by the petgraph adjacency lists (approximately 80–120 bytes per node plus 40–60 bytes per edge) and the Polars DataFrames (column-oriented, scaling with distinct property keys rather than entity count). For a representative 10,000-node topology with 30,000 edges and 50 property columns, the combined footprint is approximately 15–25 MB. The CSR cache adds a further compact layer proportional to the edge count.

28 Lessons Learned

Building a hybrid engine for network topology has yielded several key insights into the intersection of graph and relational systems.

Isomorphism is Harder than it Looks. Maintaining perfect parity between petgraph and Polars requires absolute discipline. A mutation that succeeds in the graph but fails in the property store (e.g., due to a duplicate ID) creates a silent structural mismatch. The introduction of the DualWriteGuard transactional RAII pattern (Section 12.2) was essential to make the engine robust against partial failures.

The GIL is the Primary Scaling Bottleneck. While Rust execution is near-instant, the Python-Rust boundary (via PyO3) is a constant source of overhead. Releasing the Global Interpreter Lock (GIL) during all $O(N)$ operations was a transformative change, enabling NTE to support multi-threaded analytics workloads that were previously serialised at the Python layer.

Columnar Pruning is the Secret Weapon. In a network graph, 90% of traversals are highly constrained by metadata (e.g., “only consider core routers in SYD”). By leaning into the “Filter-First” model, we found that optimising the relational property scan yielded far greater performance wins for real-world queries than any purely structural graph optimisation.

Observability is the First Requirement. In the engine’s early stages, errors were often swallowed or reported as generic Rust panics. For a systems component, structured logging and domain-specific exceptions are not “features”—they are the fundamental foundations that allow downstream consumers (like `ank_pydantic`) to behave predictably.

Roads Not Taken. The choice of petgraph over a full graph database (e.g., KuzuDB [21], Neo4j [1]) was deliberate: NTE needed to be an embeddable library, not a client-server system. Similarly, Polars was chosen over DuckDB for the property store because Arrow-native DataFrames integrate naturally with the PyO3 boundary and allow zero-copy sharing via the Arrow C Data Interface. Both choices traded query-language maturity for embedding simplicity and control over the execution pipeline. The development of NTE-QL (Section 20) has since closed much of the query-language gap that initially motivated evaluating external graph databases.

The Query Language Feedback Loop. NTE was initially designed as a library with a programmatic API; the query language (NTE-QL) was added later to support ad-hoc exploration and REPL-driven debugging. Building the query language retroactively exposed implicit assumptions in the execution engine that a programmatic API never surfaced. For example, the distinction between *forward*, *backward*, and *undirected* edge traversal was irrelevant when callers constructed traversals explicitly, but became critical once a Cypher-like syntax allowed users to write `(a)-[:Connectivity]->(b)` vs. `(a)-[:Connectivity]-(b)`. Similarly, the semantics of `DISTINCT` (scoped to `RETURN` bindings, not all match bindings) and correlated `NOT EXISTS` sub-queries only needed precise definition once the language exposed them. The lesson: a query language is not just a user interface—it is a *specification mechanism* that forces the engine to commit to semantics that a programmatic API can leave ambiguous.

Layering Leaks at the Boundaries. NTE’s layered topology model (Physical, IP, OSPF, BGP, MPLS) cleanly separates concerns within the graph, but cross-layer queries inevitably violate that separation. A query like “find all BGP sessions whose underlying physical path traverses a single point of failure” must join across layers, and the virtual edge expansion that makes this possible (Section ??) is the most complex part of the query compiler. The practical insight is that *layers are a storage abstraction, not a query abstraction*: users think in terms of cross-cutting questions, and the engine must bridge layers transparently. Designing the virtual expansion early—rather than as an afterthought—would have simplified several intermediate refactors.

The Dual-Write Tax. Maintaining two consistent stores (petgraph for structure, Polars for properties) provides the performance benefits described in Section 27, but it imposes a constant engineering tax. Every new feature—snapshots, structural transforms, WAL replay—must be implemented twice and tested for cross-store parity. The `DualWriteGuard` RAIL pattern (Section 12.2) eliminates *correctness* bugs, but it does not eliminate the *development* cost. For systems with simpler performance requirements, a single-store architecture (e.g., a columnar graph database like Kuzu [21]) would reduce this overhead. The dual-write approach is justified when the performance characteristics of the two stores are sufficiently different that no single store can serve both workloads efficiently—a condition that holds for NTE’s mix of SIMD-accelerated property scans and pointer-based graph traversal.

Schema Flexibility Has Costs. NTE’s schema-agnostic property model—where any node can carry any property key—avoids the rigidity of a fixed schema, but it introduces real costs. Sparse columns proliferate as different device types contribute disjoint property sets, increasing plan compilation time. Type coercion between Python dynamic types and Polars static columns requires careful handling at the ingestion boundary. The practical lesson: schema flexibility is valuable for rapid prototyping and telemetry ingestion, but production deployments benefit from property-name normalisation and explicit type annotations.

Structural Transforms Require Careful Invariant Preservation. Operations like `split_edge` and `explode_node` must preserve endpoint ownership, layer assignments, and edge directionality simultaneously. Early implementations that treated these as independent mutations occasionally produced topologies where an endpoint was orphaned or a layer constraint was silently violated. Wrapping each transform in a `DualWriteGuard` with post-commit invariant checks eliminated this class of bug entirely.

Test the Dual-Write, Not the Components. Unit-testing the graph store and the `DataFrame` store independently gave false confidence. Bugs lived in the *coordination* between them—a node present in `petgraph` but missing from `Polars`, or an edge removed from one store but not the other. The most effective test strategy was integration tests that mutate through the `DualWriteGuard` interface, then assert parity across both stores. The debug-mode validation flag (`NTE_VALIDATE_METADATA_SYNC_ON_COMMIT`) was born from this insight.

29 Conclusion

NTE is a network topology engine that combines two demands that network tooling often separates: graph traversal over structure and fast filtering over properties. This report has described the data model, the dual-write storage architecture, the query pipeline, and the validation workflow that support that design.

The engine’s core contribution is a single topology representation with two coordinated views: a graph for structural operations and a columnar store for property-heavy queries. This hybrid model avoids the forced choice between a graph database that struggles with property-heavy workloads and a relational store that cannot express structural traversals natively. Existing network tools tend to commit to one paradigm; NTE demonstrates that the two can coexist within a single transactional boundary, with the `DualWriteGuard` RAII pattern ensuring consistency.

On top of that model, NTE provides explicit endpoints for physical-fidelity modelling, layered topology semantics for multi-plane networks, semantic query lowering that hides structural complexity from users, transactional mutation with full conflict detection, and policy evaluation against live or shadow state.

The current implementation supports the main architectural path described in this report. Topology construction, hybrid query execution, transactional updates with MVCC isolation, named snapshots, structural diffing, policy checks, and structural transformations (`split`, `merge`, `explode`) are all functional and tested. Early benchmark results indicate that the filter-first approach performs well on property-constrained workloads, and that type-gated incremental validation can reduce the cost of selected standing queries. These results are promising, but they do not yet constitute a full comparative evaluation across production graph systems.

Several important areas remain experimental or incomplete. Distributed Raft deployment requires further hardening under adversarial conditions. General-purpose incremental view maintenance remains future work beyond the current fast path. Native security features such as authentication, authorisation, and tenancy isolation are not part of the present engine. GPU-accelerated graph algorithms and full spectral analysis are research directions, not production features.

Looking forward, NTE’s trajectory follows three phases. The engine already serves as a **Digital Twin**—an accurate, point-in-time structural record with forensic time-travel via WAL replay. It is progressing into a **Simulation Engine**, where What-If branching, policy-driven verification, and Monte Carlo risk modelling allow operators to test changes before committing them. The longer-term vision is an **Active Network Authority**: closed-loop control where streaming telemetry drives incremental view updates, spectral anomaly detection flags structural fragility in real time, and reactive policy loops trigger automated remediation—moving from passive observation to proactive network management.

29.1 Generalisable Insights

While NTE targets network topology, several of its design decisions address problems common to any domain that combines structural and property-heavy workloads:

1. **Filter before you traverse.** When most queries begin with a property constraint, executing that constraint on a columnar store before entering the graph reduces the dominant traversal term by the predicate selectivity (Proposition 1). This principle applies to any graph workload where property predicates routinely eliminate a large fraction of candidates—supply-chain networks, infrastructure dependency graphs, biological pathway models.
2. **Dual-write consistency is an RAII problem.** Coordinating two heterogeneous stores is error-prone when mutations are spread across ad-hoc call sites. Wrapping every mutation in a scope-guarded transaction object (`DualWriteGuard`) that rolls back on drop converts a distributed-consistency problem into a Rust lifetime problem. The same pattern would benefit any system that maintains parallel representations of the same data (e.g., a search index alongside a primary store).
3. **A query language is a specification, not a convenience.** Adding NTE-QL forced the engine to commit to precise semantics for edge direction, `DISTINCT` scoping, and correlated sub-queries that the

programmatic API had left ambiguous. Systems that defer query-language design accumulate implicit assumptions that become expensive to reconcile later.

4. **Explicit attachment points prevent modelling ambiguity.** The endpoint model adds structural complexity, but it eliminates an entire class of “what does this edge mean?” ambiguities. Any domain where connections have port-level or interface-level semantics—data centre fabrics, circuit-switched networks, PCB interconnects—benefits from making attachment points first-class entities rather than edge annotations.

The data centre migration scenario (Section 1.2) illustrates how these principles compose in practice: a single workflow chains a columnar property scan, a structural traversal, a copy-on-write shadow mutation, a policy check, and a vulnerability analysis—all within one transactional boundary. No individual capability is unique to NTE; the contribution is the co-design that makes this composition possible within an embeddable library.

Even with the current limits, NTE provides a useful and distinctive foundation for network modelling, verification, and change analysis. It offers a precise structural model, a practical execution strategy for mixed graph-relational workloads, and a clear path toward stronger verification and more reactive operation.

A Cargo Workspace Crates

NTE is a 17-crate Cargo workspace (16 workspace members plus the root PyO3 crate). The following table details the crates and their primary roles within the architecture.

Table 6. NTE Cargo Workspace Crates and Roles.

Crate	Layer	Purpose
src/ (root)	Bindings	PyO3 Python extension (lib.rs, topology.rs, query.rs, transaction.rs)
n-te-core	Core	Core topology with domain bindings
n-te-domain	Core	Pure domain types (no dependencies)
n-te-graph	Core	petgraph wrapper, ID mapping, CSR, GPU kernels
n-te-topology	Core	Topology struct, transactions, snapshots, diffing, shadow
n-te-query	Query	Executor, planner, matcher, profiler, visualisation, algorithm plugins
n-te-cli	Tooling	NTE-QL REPL, policy validator, TUI dashboard
n-te-lsp	Tooling	Language Server Protocol for NTE-QL editing
n-te-policy	Policy	Starlark engine, DeltaNet validator
n-te-backend	Storage	Abstract TopologyBackend trait
n-te-datastore	Storage	Feature-flag crate selecting the active backend
n-te-datastore-core	Storage	Shared backend types (re-exported by n-te-backend)
n-te-datastore-polars	Storage	Polars Arrow-backed DataFrames (default)
n-te-datastore-lite	Storage	Lightweight in-memory store (testing)
n-te-server	Server	Axum HTTP/WebSocket, Raft consensus, diagnostics
n-te-ffi	Bindings	C ABI foreign function interface for non-Python consumers
n-te-monte-carlo	Optional	Monte Carlo simulation

B Legacy Configuration Support

For legacy environments that rely on flat configuration files, NTE provides a YAML schema for policy definition. These are automatically compiled into NTE-QL equivalents by the PolicyEngine.

Listing 28. Legacy YAML policy format

```
version: 1
policies:
  - id: "P001"
    category: "attribute"      # evaluated per-node
    expr: "device.speed_gbps >= 100"
    message: "All devices must support 100G"

  - id: "P002"
    category: "structural"    # global check
    expr: "transit_exit_count >= 2"
    message: "Every PoP must have at least 2 transit exits"
```

```
- id: "P003"
  category: "structural"
  expr: "max_sessions_per_peer_as <= 1"
  message: "Router has multiple BGP sessions to the same peer AS"
```

C NTE-QL Grammar (EBNF)

The following is a simplified EBNF representation of the NTE-QL query language used by the parser in nte-query.

Implementation Status

The core MATCH/WHERE/RETURN pipeline and the EXPLAIN VISUAL command are fully implemented, along with ORDER BY, LIMIT, SKIP, COUNT(*), IS NULL/IS NOT NULL, IN [...], CONTAINS/STARTS WITH/ENDS WITH, RETURN *, AS aliases, float/boolean/null literals, inline edge property filters, and virtual edge expansion. The USE LAYER, WITH, OPTIONAL MATCH, DISTINCT, aggregation functions (SUM, AVG, MIN, MAX, COLLECT), CREATE, MERGE, and NOT EXISTS sub-queries are also now fully implemented. Implemented productions are marked with [impl] below.

Listing 29. NTE-QL EBNF Grammar

```
query      ::= (use_layer)? (stage | mutation) (stage | mutation)* -- [impl]
stage      ::= (MATCH pattern | WITH projection | OPTIONAL MATCH pattern) (WHERE expr)? -- [impl]
use_layer  ::= "USE" "LAYER" string_literal -- [impl]
mutation   ::= (CREATE | MERGE) pattern (SET set_clause)* -- [impl]
subquery   ::= "NOT" "EXISTS" "{" query "}" -- [impl]
pattern    ::= node_pattern (edge_pattern node_pattern)* -- [impl]
node_pattern ::= "(" (ident)? (":" label)? (prop_filter)? ")" -- [impl]
edge_pattern ::= "-[" (ident)? (":" label)? (prop_filter)? "]" -> " -- [impl]
              | "<-" (ident)? (":" label)? (prop_filter)? "]" -> " -- [impl]
              | "-[" (ident)? (":" label)? (prop_filter)? "]" -> " -- [impl]
label      ::= ident -- [impl]
prop_filter ::= "{" ident ":" expr ("," ident ":" expr)* "}" -- [impl]
set_clause ::= ident "." ident "=" expr -- [impl]
expr       ::= literal -- [impl]
              | ident -- [impl]
              | subquery -- [impl]
              | expr op expr -- [impl]
              | "(" expr ")" -- [impl]
              | expr "IS" "NULL" -- [impl]
              | expr "IS" "NOT" "NULL" -- [impl]
              | expr "IN" "[" expr ("," expr)* "]" -- [impl]
              | expr "CONTAINS" expr -- [impl]
              | expr "STARTS" "WITH" expr -- [impl]
              | expr "ENDS" "WITH" expr -- [impl]
              | "NOT" expr -- [impl]
literal    ::= string | integer | float | "true" | "false" | "NULL" -- [impl]
op         ::= "=" | "!=" | "<" | ">" | "<=" | ">=" -- [impl]
              | "AND" | "OR" | "+" | "-" | "*" | "/" -- [impl]
projection ::= (DISTINCT)? ("*" | return_item ("," return_item)*) -- [impl]
return_item ::= ident | ident "." ident | func "(" args ")" -- [impl]
              | "COUNT" "(" "*" ")" | return_item "AS" ident -- [impl]
func       ::= "shortestPath" | "k_shortest_paths" -- [impl]
              | "SUM" | "AVG" | "MIN" | "MAX" | "COLLECT" -- [impl]
order_by   ::= "ORDER" "BY" expr ("ASC" | "DESC")? -- [impl]
limit     ::= "LIMIT" integer -- [impl]
skip     ::= "SKIP" integer -- [impl]
```

C.1 Worked Examples

The following examples illustrate concrete queries against a sample topology containing routers, switches, and endpoints across physical and BGP layers.

Example 1: Simple Node Scan with Property Filter. Find all routers in the Sydney PoP:

```
MATCH (r:Router)
WHERE r.pop = "SYD"
RETURN r.hostname, r.pop
```

This compiles to a QuerySpec with an Equality filter on pop and a System filter on node_type = "Router".

Example 2: Edge Traversal with Kind Filter. Find all eBGP peering sessions and the hostnames of both ends:

```
MATCH (a:Router)-[session:eBGP {layer: 'bgp'}]->(b:Router)
RETURN a.hostname, b.hostname
```

The planner expands this into a three-hop structural traversal: Router → Structural → Connectivity(eBGP) → Structural → Router, filtering edges to the bgp layer.

Example 3: Inspecting the Execution Plan. Use EXPLAIN VISUAL to see the compiled plan without executing:

```
EXPLAIN VISUAL
MATCH (a:Router)-[:Connectivity]->(b:Switch)
WHERE a.pop = "MEL"
RETURN a.hostname, b.hostname
```

This renders a Mermaid DAG showing the filter-first pipeline: the pop = "MEL" predicate prunes the candidate set before the structural traversal begins.

References

- [1] Neo4j, Inc., *Neo4j graph database*, <https://neo4j.com>, 2024.
- [2] J. Abernethy and R. Chirkova, "Analyzing recursive SQL queries," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, ACM, 2013, pp. 631–642.
- [3] M. Mahalingam et al., "Virtual eXtensible Local Area Network (VXLAN): A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks," IETF, RFC 7348, Aug. 2014.
- [4] Y. Rekhter, T. Li, and S. Hares, "A Border Gateway Protocol 4 (BGP-4)," IETF, RFC 4271, Jan. 2006.
- [5] J. Moy, "OSPF Version 2," IETF, RFC 2328, Apr. 1998.
- [6] D. Oran, "OSI IS-IS intra-domain routing protocol," IETF, RFC 1142, Feb. 1990.
- [7] E. Rosen, A. Viswanathan, and R. Callon, "Multiprotocol Label Switching Architecture," IETF, RFC 3031, Jan. 2001.
- [8] Meta Platforms and the starlark-rust Contributors, *Starlark-rust: Rust implementation of the Starlark language*, <https://github.com/facebookexperimental/starlark-rust>, 2024.
- [9] A. Sajassi et al., "BGP MPLS-Based Ethernet VPN," IETF, RFC 7432, Feb. 2015.
- [10] D. Kolber, "Rkyv: Zero-copy deserialization in Rust," in *Proceedings of the Systems Programming Workshop*, 2023.
- [11] Google, *FlatBuffers: Memory efficient serialization library*, <https://flatbuffers.dev>, 2024.
- [12] X. Tang, "OpenRaft: An advanced Raft implementation in Rust," in *Proceedings of the Rust OSDI Workshop*, USENIX, 2023.
- [13] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (ATC)*, USENIX, 2014, pp. 305–320.
- [14] Bluss and the petgraph Contributors, *Petgraph: Graph data structure library for Rust*, <https://crates.io/crates/petgraph>, 2024.
- [15] R. Vink and the Polars Contributors, *Polars: Blazingly fast dataframes in Rust and Python*, <https://polars.rs>, 2024.
- [16] T. Willhalm, N. Popovici, Y. Boshmaf, H. Plattner, A. Zeier, and J. Schaffner, "SIMD-scan: Ultra fast in-memory table scan using on-chip vector processing units," in *Proceedings of the VLDB Endowment*, vol. 2, 2009, pp. 385–394.
- [17] D. Hewitt and the PyO3 Contributors, "PyO3: Rust bindings for Python," in *Proceedings of the Python Language Summit*, 2023.
- [18] Y. Shiloach and U. Vishkin, "An o(log n) parallel connectivity algorithm," 1, vol. 3, 1982, pp. 57–67.
- [19] A. Xu, A. Deutsch, V. Liang, and M. Xu, "TigerGraph: A native MPP graph database," in *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, ACM, 2019, pp. 1799–1816.
- [20] Amazon Web Services, *Amazon Neptune graph database*, <https://aws.amazon.com/neptune/>, 2024.
- [21] G. Jin et al., *Kuzu: An in-process property graph database management system*, <https://kuzudb.com>, 2023.
- [22] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, ACM, 2019, pp. 1981–1984.
- [23] LanceDB, *LanceDB: Developer-friendly, serverless vector database*, <https://lancedb.com>, 2024.

- [24] A. Mhedhbi, P. Gupta, and S. Salihoglu, “Optimizing one-time and persistent joins in graph database management systems,” in *Proceedings of the VLDB Endowment*, vol. 14, 2021, pp. 739–752.
- [25] Apache Software Foundation, *Apache AGE: A graph extension for PostgreSQL*, <https://age.apache.org>, 2024.
- [26] Bitnine Global, *AgensGraph: Multi-model database with graph and relational storage*, <https://bitnine.net/agensgraph/>, 2024.
- [27] A. Fogel et al., “A general approach to network configuration analysis,” in *Proceedings of the 14th ACM Workshop on Hot Topics in Networks (HotNets)*, ACM, 2015, pp. 1–7.
- [28] Network to Code, *Nautobot network source of truth*, <https://nautobot.com>, 2024.
- [29] NetBox Labs, *NetBox: The leading solution for modelling and documenting network infrastructure*, <https://netbox.dev>, 2024.
- [30] D. G. Dodin and J. Shakir, *Suzieq: Software for network observability*, <https://github.com/netenglabs/suzieq>, 2024.
- [31] J. Shun and G. E. Blelloch, “Ligra: A lightweight graph processing framework for shared memory,” in *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, ACM, 2013, pp. 135–146.
- [32] J. Kepner et al., “Mathematical foundations of the GraphBLAS,” *Proceedings of the IEEE High Performance Extreme Computing Conference (HPEC)*, pp. 1–9, 2016.
- [33] D. Nguyen, A. Lenharth, and K. Pingali, “A lightweight infrastructure for graph analytics,” in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, ACM, 2013, pp. 456–471.
- [34] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, “Gunrock: A high-performance graph processing library on the GPU,” in *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, ACM, 2016, 11:1–11:12.
- [35] H. Q. Ngo, E. Porat, C. Ré, and A. Rudra, “Skew strikes back: New developments in the theory of join algorithms,” in *ACM SIGMOD Record*, vol. 42, 2014, pp. 5–16.
- [36] T. L. Veldhuizen, “Leapfrog Triejoin: A simple, Worst-Case optimal join algorithm,” *arXiv preprint arXiv:1210.0481*, 2012.

Revision History

Version	Date	Changes
2.8.0	March 2026	Executor hardening: SET persistence, NOT EXISTS correlated sub-queries, direction-aware edge traversal, DISTINCT scoped to RETURN bindings. Removed DuckDB/KuzuDB crate references.
2.7.0	March 2026	Finalised NTE-QL grammar: USE LAYER, DISTINCT, CREATE/MERGE, and NOT EXISTS sub-queries.
2.6.0	March 2026	NTE-QL executor: WITH pipeline, OPTIONAL MATCH null-fill, aggregation execution, REPL tab-completion.
2.5.0	March 2026	Refined Architecture & Insights Narrative; Added API Guide & Appendices.
2.4.0	March 2026	Native Policy Engine (Phase 16) & Topology Diffing (Phase 17).
2.3.0	March 2026	Integrated v2.3 Ph.D. Insights & Design Rules.
0.3.0	March 2026	Integrated ACI Isolation Levels and CSR Persistence
0.2.0	Jan 2026	GPU-accelerated SSSP and Label Propagation
0.1.0	Nov 2025	Initial Specification and Core Data Model